

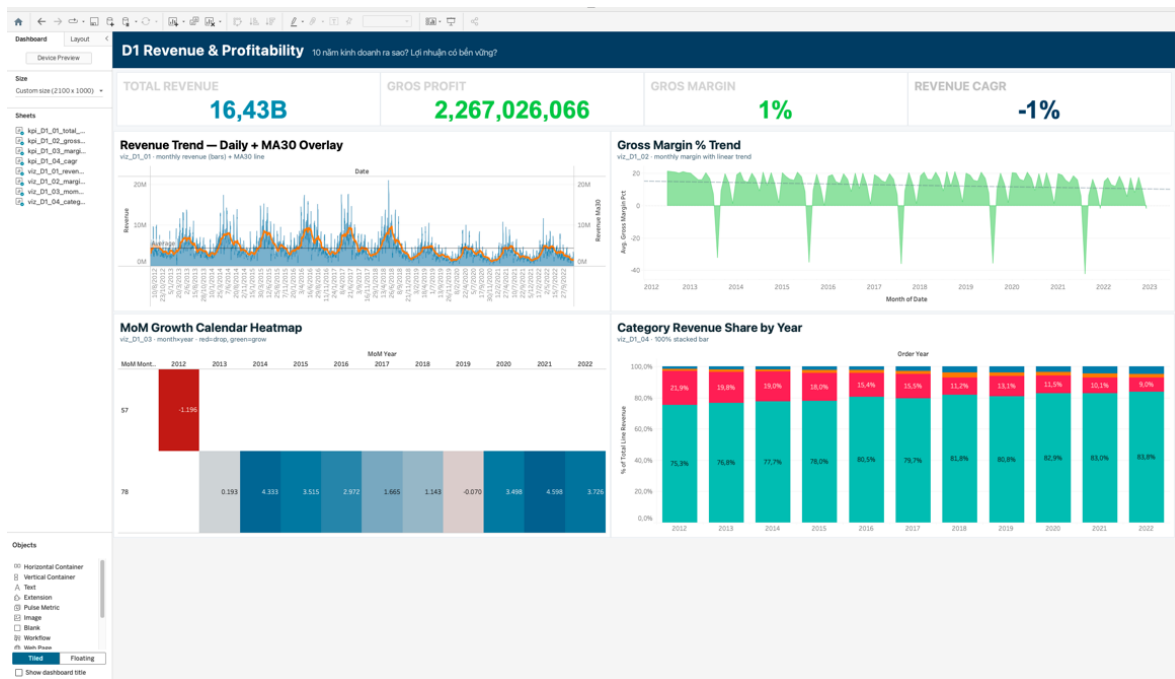
Retail Analytics & Forecasting

Phân tích trực quan và Dự báo chuỗi thời gian trên dữ liệu E-commerce

Nguyễn Quốc Thái (MSc) Nguyễn Phúc Nguyên Đinh Thị Thu Hà
Nguyễn Thảo Nhi Đinh Quang Vinh (PhD)
Proofreader: Bùi Quốc Huy, Đinh Tiến Anh

A. Giới thiệu chung

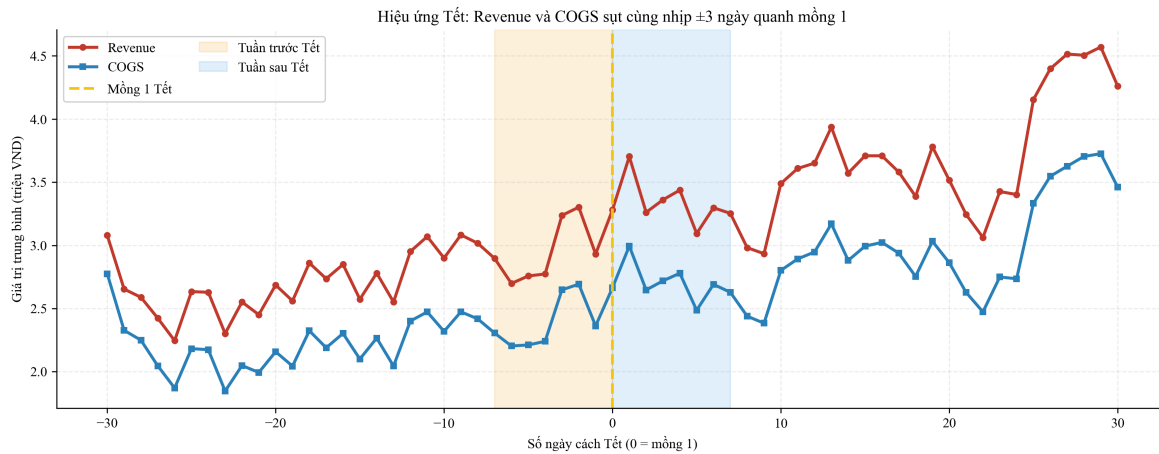
VinDatathon 2026 (Round 1) xoay quanh một bộ dữ liệu thương mại điện tử mô phỏng một doanh nghiệp Việt Nam, trải dài 11 năm (04/07/2012–31/12/2022) với 15 bảng phủ đủ các khía cạnh: đơn hàng, sản phẩm, khách hàng, giao vận, tồn kho và web traffic. Dựa trên dữ liệu này chúng ta sẽ phân tích đưa ra đặc điểm cụ thể, xây dựng mô hình dự đoán và giải thích thông qua phân tích để nắm rõ những câu chuyện từ dữ liệu. Trong phần này chúng ta sẽ đi tìm hiểu hai khía cạnh như sau:



Hình 1: Phần A — Dashboard & Analysis: trực quan hoá và phân tích dữ liệu bán lẻ.

Phần A — Dashboard & Analysis (DA) trả lời câu hỏi *điều gì đã xảy ra và vì sao*. Phần này trực quan hoá doanh thu và lợi nhuận, phân khúc khách hàng theo RFM, phân tích cohort retention và hiệu quả các kênh marketing; mỗi dashboard đi đủ bốn cấp phân tích Descriptive → Diagnostic → Predictive → Prescriptive và kết thúc bằng đề xuất hành động cụ thể, định lượng.

Phần B — Dự báo chuỗi thời gian (DS) chuyển từ mô tả sang *dự đoán*. Từ các quy luật quan sát được, phần này xây dựng một pipeline dự báo **Revenue** và **COGS** cho 548 ngày kế tiếp (01/01/2023–01/07/2024): feature engineering thuần lịch, ba mô hình nền (Ridge, LightGBM, Prophet), bốn mô hình chuyên biệt theo quý, một tầng ensemble, và cuối cùng là phần giải thích mô hình (explainability).



Hình 2: Phần B — Dự báo chuỗi thời gian: hiệu ứng Tết được khai thác cho mô hình.

Hai phần bổ trợ chặt chẽ cho nhau: các insight và giả định rút ra ở Phần A — doanh thu đã qua đỉnh với CAGR âm, mùa vụ bền vững qua các năm, margin biến động mạnh theo quý — chính là điểm để đưa ra các nhận định và những quyết định thiết kế mô hình ở Phần B.

Mục lục

A	Giới thiệu chung	1
B	Phân tích dữ liệu	7
I.	Giới thiệu	7
I.1.	Cấu trúc đề thi	8
I.2.	Rubric: Ban giám khảo chấm những gì?	8
I.3.	Tổng quan quy trình	9
II.	Khám phá dữ liệu	10
II.1.	Bản đồ dữ liệu	10
II.2.	Quan hệ giữa các bảng	10
II.3.	Ba nhận xét quan trọng	11
III.	Thiết kế hướng phân tích	12
III.1.	Mapping: Data → Câu hỏi → Dashboard	12
III.2.	Narrative arc (mạch chuyện)	12
III.3.	Tại sao cần Python trước khi mở Tableau?	12
IV.	Chuẩn bị dữ liệu bằng Python	14
IV.1.	Cấu hình và helper functions	14
IV.2.	Tổng quan 14 bước xử lý	14
IV.3.	Step 1: Load dữ liệu gốc	15
IV.4.	Step 2: dim_products (Sản phẩm enriched)	15
IV.5.	Step 3–4: dim_geography, dim_promotions	16
IV.6.	Step 5: fact_orders_enriched (Bảng fact trung tâm)	16
IV.7.	Step 6: fact_returns_enriched	18
IV.8.	Step 7: RFM Segmentation (phức tạp nhất)	18
IV.9.	Step 8: Shipments enriched	20
IV.10.	Step 9: fact_sales_daily (Revenue + trends)	21
IV.11.	Step 10: Inventory + health labels	21
IV.12.	Step 11: Web traffic	22

IV.13.	Step 12: Cohort Retention	22
IV.14.	Step 13–14: Bảng tổng hợp	23
V.	Thiết lập Tableau Public	25
V.1.	Tạo Data Source	25
V.2.	Kiểm tra Data Types	25
V.3.	Template layout chung	25
V.4.	Dashboard Actions	26
V.5.	KPI Delta: cách tính “vs prior year”	26
VI.	Dashboard 1: Revenue & Profitability	28
VI.1.	4 KPI Cards	28
VI.2.	Chart 1: Revenue Trend (Descriptive)	29
VI.3.	Chart 2: Gross Margin Trend (Diagnostic)	29
VI.4.	Chart 3: MoM Growth Heatmap (Predictive)	30
VI.5.	Chart 4: Category Revenue Contribution (Prescriptive)	30
VI.6.	Ghép Dashboard 1	31
VII.	Dashboard D2	
	Customer Segmentation & Lifecycle	32
VII.1.	Tổ chức & Chuẩn bị Dữ liệu	32
VII.2.	Xây dựng dashboard	33
VII.3.	Chiến lược ghi điểm cho Dashboard 2	35
VIII.	Dashboard D4	
	Marketing & Channel Effectiveness	35
VIII.1.	Tổ chức & Chuẩn bị Dữ liệu	36
VIII.2.	Xây dựng dashboard	37
IX.	Gợi mở: Dashboard tiếp theo	40
IX.1.	Dashboard D3: Product Analysis	40
IX.2.	Dashboard 5: Operations & Supply Chain	40
IX.3.	Story: CEO Narrative (6 Story Points)	40
X.	Phân tích	41
X.1.	Dashboard D1 Revenue & Profitability Overview	41

X.2.	Dashboard D2: Customer Segmentation & Lifecycle	42
X.3.	Dashboard D4: Marketing & Channel Effectiveness	44
XI.	Tổng kết	46
C	XÂY DỰNG MÔ HÌNH DỰ ĐOÁN	47
XII.	Giới thiệu	47
XIII.	Phát biểu bài toán	48
XIII.1.	Input và Output	48
XIII.2.	Hàm mục tiêu	48
XIV.	Phân tích dữ liệu	49
XIV.1.	Bức tranh tổng thể: Biểu đồ qua các năm	49
XIV.2.	Cấu trúc thời gian: Tính mùa vụ có bền vững không?	50
XIV.3.	Các pattern lịch biểu chi tiết	51
XIV.4.	Phân phối và mối liên hệ giữa hai biến mục tiêu	53
XIV.5.	Scatter Revenue–COGS: cơ sở cho hướng dự báo qua tỉ số	55
XIV.6.	Tóm tắt: từ EDA đến các quyết định mô hình	56
XV.	Feature Engineering	57
XVI.	Chiến lược validation	60
XVI.1.	Ba fold theo thời gian	60
XVII.	Lựa chọn và huấn luyện mô hình	61
XVII.1.	Lựa chọn mô hình như thế nào?	61
XVII.2.	M1: Ridge Regression	61
XVII.3.	M2: LightGBM với sample weighting	62
XVII.4.	M3: Prophet	63
XVII.5.	Mô hình huấn luyện theo quý	64
XVIII.	Chiến lược ensemble learning	66
XVIII.1.	Hướng 1: Ensemble đơn giản	66
XVIII.2.	Hướng 2: Ensemble phân tầng	66
XVIII.3.	Mã nguồn ensemble phân tầng	68

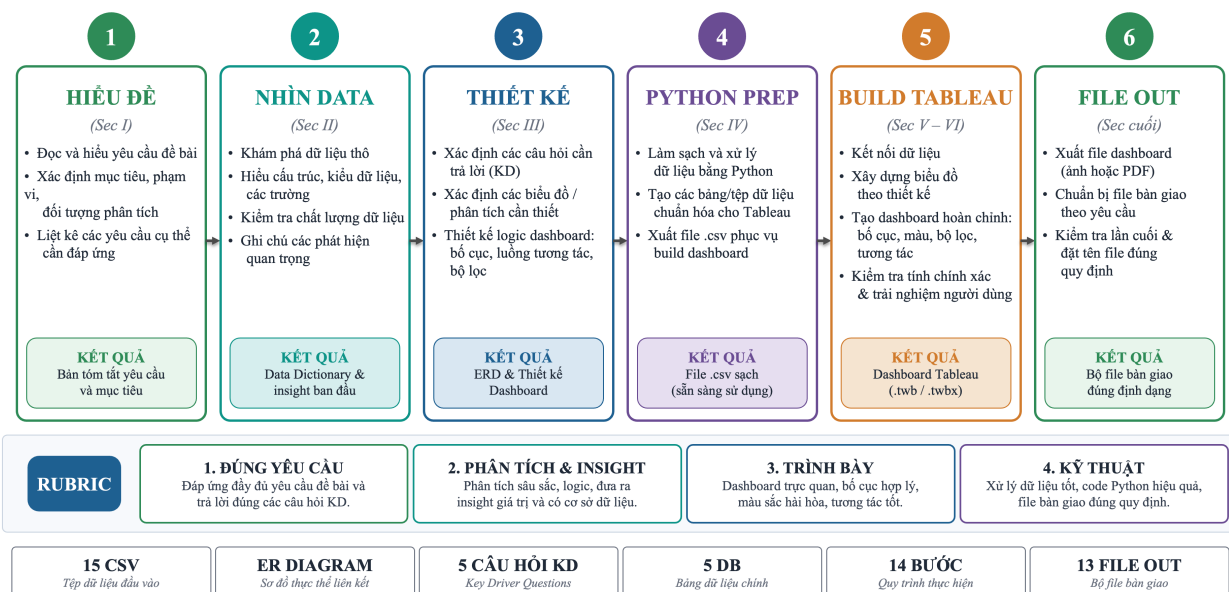
XIX. Parameters Tuning	69
XIX.1. Tổng quan: ba nhóm tham số và công cụ tuning tương ứng	69
XIX.2. Chiến lược tuning cho từng nhóm	69
XX. Khả năng giải thích mô hình	
(Explainability)	71
XX.1. Đối tượng và công cụ giải thích	71
XX.2. Global importance	72
XX.3. Local effect	74
XX.4. Đối chứng định lượng với Prophet	77
Phụ lục	81

B. Phân tích dữ liệu

I. Giới thiệu

Datathon 2026 Round 1 đặt ra một bài toán rất thực tế: chúng ta nhận được 15 file CSV mô phỏng toàn bộ hoạt động của **một doanh nghiệp thời trang thương mại điện tử** tại Việt Nam, giai đoạn 04/07/2012 đến 31/12/2022. Dữ liệu trải rộng từ đơn hàng, sản phẩm, khách hàng, giao vận, cho đến web traffic và inventory. Nhiệm vụ của chúng ta: biến đổi dữ liệu thô đó thành một bộ dashboard có khả năng kể chuyện, đủ sâu để thuyết phục CEO hành động. Phần 2 (Trực quan hoá & Phân tích) chiếm **60/100 điểm** của Round 1, quyết định thứ hạng. Câu hỏi đặt ra là: bắt đầu từ đâu khi trước mặt là 15 file, hàng trăm nghìn dòng, và một rubric chấm điểm khá chi tiết?

Quy Trình Xây Dựng Dashboard Tableau



Hình 3: Quy trình phân tích dữ liệu.

💡 Dataset là synthetic (mô phỏng)

Đơn vị tiền không xác định rõ. Khi phân tích, tập trung vào **tỷ lệ và ranking** hơn là giá trị tuyệt đối. Ví dụ: “Category A chiếm 35% doanh thu” thuyết phục hơn “Category A đạt 500 triệu”.

Bài hướng dẫn này sẽ đưa chúng ta qua toàn bộ quy trình, từ phân tích đề bài cho đến build

hoàn chỉnh Dashboard đầu tiên (Revenue & Profitability). Sau khi nắm vững quy trình qua Dashboard 1, chúng ta sẽ có đủ nền tảng để tự build 4 Dashboard còn lại theo cùng một pattern.

I.1. Cấu trúc đề thi

Datathon 2026 Round 1 chia thành 3 phần, trong đó Phần 2 chiếm tỷ trọng lớn nhất:

Phần	Điểm	Nội dung
Phần 1: Trắc nghiệm	20đ	10 câu hỏi tính toán từ dữ liệu
Phần 2: Visualization & Analysis	60đ	EDA, dashboard, insight, đề xuất hành động
Phần 3: Sales Forecasting	20đ	Dự báo Revenue trên Kaggle

Chỉ riêng Phần 2 đã chiếm quá nửa tổng điểm. Một bộ dashboard tốt có thể bù cho phần Forecasting chưa hoàn hảo, nhưng ngược lại thì khó.

I.2. Rubric: Ban giám khảo chấm những gì?

Hiểu rubric trước khi bắt tay làm giúp chúng ta biết đâu là nơi “ghi điểm” hiệu quả nhất:

Tiêu chí	Điểm	Ghi điểm cao khi...
Chất lượng trực quan hoá	15đ	Mọi biểu đồ có tiêu đề, nhãn trục, chú thích đầy đủ. Loại chart phù hợp dữ liệu
Chiều sâu phân tích	25đ	Đạt cả 4 cấp: Descriptive → Diagnostic → Predictive → Prescriptive
Insight kinh doanh	15đ	Đề xuất cụ thể, định lượng, áp dụng được ngay
Tính sáng tạo & kể chuyện	5đ	Góc nhìn độc đáo, kết nối nhiều bảng dữ liệu

Tiêu chí “Chiều sâu phân tích” chiếm đến 25 điểm, nặng nhất. Muốn đạt 21–25đ, bài nộp cần nhất quán ở cấp **Prescriptive** (đề xuất hành động cụ thể) trên nhiều phân tích.

4 cấp độ phân tích dữ liệu

Mỗi cấp trả lời một loại câu hỏi khác nhau:

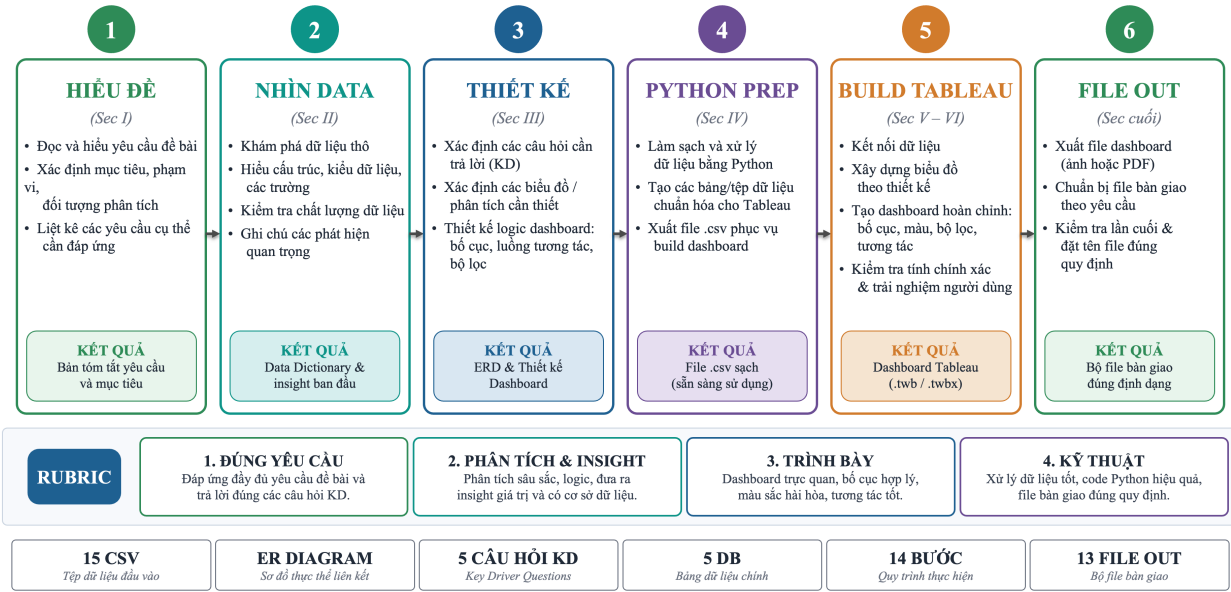
- **Descriptive** (Chuyện gì đã xảy ra?): “Revenue giảm 3.8%/năm.”
- **Diagnostic** (Tại sao xảy ra?): “Margin giảm vì COGS tăng nhanh hơn revenue.”
- **Predictive** (Điều gì sẽ xảy ra?): “Q1 mỗi năm đều giảm, dự kiến Q1/2023 tương tự.”
- **Prescriptive** (Nên làm gì?): “Giảm tỷ trọng category margin thấp, tăng đầu tư Stars-quadrant.”

Mỗi dashboard trong bài hướng dẫn đều có ít nhất 1 chart ở mỗi cấp.

I.3. Tổng quan quy trình

Bài hướng dẫn chia thành 6 giai đoạn, mỗi giai đoạn xây trên giai đoạn trước:

Quy Trình Xây Dựng Dashboard Tableau



Hình 4: Quy trình phân tích dữ liệu.

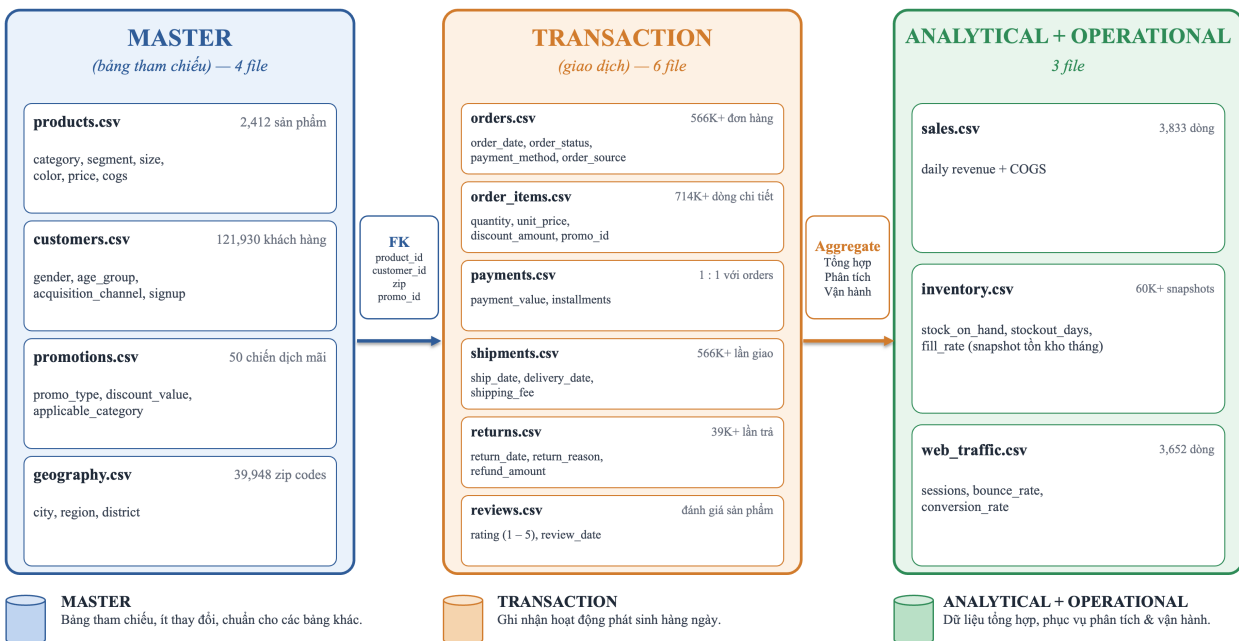
Hiểu rubric giúp chọn đúng câu hỏi. Nhìn data giúp thiết kế dashboard sát thực tế. Python prep giúp Tableau chạy nhanh và công thức gọn.

II. Khám phá dữ liệu

Trước khi quyết định phân tích gì, chúng ta cần biết data có gì. Mở thư mục Data/ và phân loại 15 file theo 4 lớp.

II.1. Bản đồ dữ liệu

SƠ ĐỒ KHỐI BẢN ĐỒ DỮ LIỆU THEO NHÓM CHỨC NĂNG



Hình 5: Bản đồ dữ liệu 15 file CSV phân theo 3 lớp: Master (bảng tham chiếu), Transaction (giao dịch), và Analytical + Operational.

Ba lớp cho thấy dữ liệu phủ đủ các khía cạnh của một doanh nghiệp: bán hàng, khách hàng, sản phẩm, marketing, và vận hành.

II.2. Quan hệ giữa các bảng

Trước khi join dữ liệu, chúng ta cần hiểu **cardinality** (số lượng quan hệ) giữa các bảng:

Quan hệ	Cardinality	Ý nghĩa
orders ↔ payments	1:1	Mỗi đơn có đúng 1 thanh toán
orders ↔ order_items	1:N	1 đơn có nhiều sản phẩm
orders ↔ shipments	1:0..1	Chỉ đơn shipped/delivered có giao hàng
orders ↔ returns	1:0..N	1 đơn có thể trả nhiều sản phẩm
orders ↔ reviews	1:0..N	Chỉ ~20% đơn có review
order_items ↔ promotions	N:0..1	Mỗi line item có thể áp 0–2 promo
products ↔ inventory	1:N	1 sản phẩm có nhiều snapshot tháng

II.3. Ba nhận xét quan trọng

Dữ liệu đủ cho phân tích đa chiều. Có đủ 5 khía cạnh: tài chính (`sales`, `payments`), khách hàng (`customers`), sản phẩm (`products`, `reviews`, `returns`), marketing (`orders.order_source`, `web_traffic`), vận hành (`shipments`, `inventory`). Đây là lợi thế lớn cho rubric “Tính sáng tạo & kể chuyện”.

Nhiều bảng cần join mới có ý nghĩa. Muốn biết “category nào có return rate cao nhất” thì cần join `returns` → `products`. Muốn biết “kênh nào mang KH giá trị cao” thì cần join `orders` → `order_items` → `payments` → `customers`.

Grain khác nhau giữa các bảng. `sales.csv` có 1 dòng = 1 ngày, `order_items.csv` có 1 dòng = 1 sản phẩm trong 1 đơn. Không thể join trực tiếp hai bảng này trong Tableau mà không xử lý trước. Đây là lý do chúng ta cần Python prep.

Khái Niệm: Grain (mức chi tiết)

Grain là đơn vị nhỏ nhất mà mỗi dòng trong bảng đại diện. Ví dụ: grain của `order_items` là “một sản phẩm trong một đơn hàng”, grain của `sales` là “một ngày”. Hiểu đúng grain giúp tránh lỗi tính toán: `SUM(line_revenue)` cho đúng tổng doanh thu, nhưng `COUNT(order_id)` sẽ đếm trùng nếu 1 đơn có nhiều sản phẩm.

III. Thiết kế hướng phân tích

Giờ chúng ta ánh xạ data vào câu hỏi kinh doanh. Mỗi câu hỏi trở thành 1 dashboard:

III.1. Mapping: Data → Câu hỏi → Dashboard

#	Câu hỏi kinh doanh	Data cần	Tại sao?
1	Tăng trưởng ra sao? Lợi nhuận bền vững?	<code>sales + order_items</code>	3,833 ngày revenue, dễ vẽ trend
2	Ai là KH giá trị? Ai đang rời?	<code>customers + orders + payments</code>	Demographics + RFM
3	SP nào tốt? Tại sao bị trả?	<code>products + returns + reviews</code>	Price/cogs cho margin, reviews cho rating
4	Kênh marketing nào hiệu quả?	<code>orders + web_traffic + promotions</code>	Channel data + bounce/conversion
5	Delivery và inventory có vấn đề?	<code>shipments + inventory</code>	Delivery time + stockout metrics

III.2. Narrative arc (mạch chuyện)

5 dashboards xếp theo logic kinh doanh, từ tổng quan đến chi tiết:

```

DB1: Revenue Overview → "10 năm kinh doanh thế nào?"
    ↓
DB2: Customer → "Ai mua hàng? Ai đang rời đi?"
    ↓
DB3: Product → "Bán gì? Chất lượng ra sao?"
    ↓
DB4: Marketing → "Kênh nào mang KH đến?"
    ↓
DB5: Operations → "Giao hàng, tồn kho ổn không?"
    ↓
Story: "5 đề xuất hành động cho CEO"
  
```

Mỗi dashboard bao phủ đủ 4 cấp phân tích. Mỗi dashboard kết nối ít nhất 3 bảng dữ liệu. Mỗi dashboard kết thúc bằng đề xuất hành động cụ thể.

III.3. Tại sao cần Python trước khi mở Tableau?

Nhìn bảng mapping, mỗi dashboard cần join 2–5 bảng. Nếu join trực tiếp trong Tableau:

- **Performance kém:** Tableau Public xử lý chậm khi join bảng 714K dòng với 566K dòng

- **Logic phức tạp:** Tính RFM score, cohort retention, margin % cần code Python, không phải Tableau calc field
- **Lặp lại công việc:** 5 dashboards cùng cần phép join `orders + order_items + products`, sao không tính 1 lần?

Vì vậy, chúng ta dùng Python để: **join** (gộp nhiều bảng thành bảng phẳng), **compute** (tính sẵn RFM, margin, moving averages), và **aggregate** (tạo bảng tổng hợp cohort, monthly KPI).

IV. Chuẩn bị dữ liệu bằng Python

Script `prepare_tableau_data.py` là xương sống của toàn bộ pipeline. Chúng ta sẽ đi qua từng bước: đọc code, hiểu logic, và biết kết quả dùng ở đâu trong Tableau.

IV.1. Cấu hình và helper functions

```
1 import pandas as pd
2 import numpy as np
3 import os
4
5 DATA_DIR = r"Data"
6 OUT_DIR = r"tableau_data"
7 os.makedirs(OUT_DIR, exist_ok=True)
8
9 def load(name, **kw):
10     path = os.path.join(DATA_DIR, name)
11     df = pd.read_csv(path, **kw)
12     print(f" Loaded {name}: {df.shape[0]:,} rows x {df.shape[1]} cols")
13     return df
14
15 def save(df, name):
16     path = os.path.join(OUT_DIR, name)
17     df.to_csv(path, index=False)
18     size_mb = os.path.getsize(path) / 1024 / 1024
19     print(f" Saved {name}: {df.shape[0]:,} rows ({size_mb:.1f} MB)")
```

`load()` đọc CSV và in số dòng. `save()` ghi ra `tableau_data/`. Cần Python 3.8+:

```
1 python --version    # Can Python 3.8+
2 pip install pandas numpy
3 python prepare_tableau_data.py
```

Script mất khoảng 1–2 phút, tạo **13 file CSV** (tổng ~277 MB).

IV.2. Tổng quan 14 bước xử lý

Các bước đánh dấu ★ được phân tích code chi tiết:

Step	Output	Xử lý chính
1	(load vào RAM)	Parse dates, đọc 15 file
2 ★	dim_products	products + reviews + returns → margin, avg rating, return stats
3	dim_geography	Copy nguyên
4	dim_promotions	Copy nguyên
5 ★	fact_orders_enriched	JOIN 5 bảng + tính revenue, cost, margin, date dims
6	fact_returns_enriched	returns + products + orders → days_to_return
7 ★	dim_customers_rfm	RFM Segmentation (R, F, M scores + 8 segment labels)
8	fact_shipments	delivery_days, processing_days, delivery_bucket
9 ★	fact_sales_daily	Gross profit, margin, MA7/MA30, YoY growth
10	fact_inventory	health label, sell_through
11	fact_web_traffic	pages_per_session
12 ★	agg_cohort_retention	Ma trận retention 24 tháng
13	agg_monthly_summary	KPI rollup hàng tháng
14	agg_reviews_summary	Rating theo tháng × category

IV.3. Step 1: Load dữ liệu gốc

```

1 orders      = load("orders.csv",      parse_dates=["order_date"])
2 order_items = load("order_items.csv")
3 payments    = load("payments.csv")
4 products    = load("products.csv")
5 customers   = load("customers.csv",   parse_dates=["signup_date"])
6 geography   = load("geography.csv")
7 promotions  = load("promotions.csv",   parse_dates=["start_date", "end_date"])
8 shipments   = load("shipments.csv",    parse_dates=["ship_date", "delivery_date"])
9 returns     = load("returns.csv",      parse_dates=["return_date"])
10 reviews    = load("reviews.csv",      parse_dates=["review_date"])
11 inventory   = load("inventory.csv",    parse_dates=["snapshot_date"])
12 web_traffic = load("web_traffic.csv",   parse_dates=["date"])
13 sales       = load("sales.csv",        parse_dates=["Date"])

```

`parse_dates` chỉ định cột ngày để pandas parse thành `datetime64`. Nếu bỏ qua, `order_date` sẽ là chuỗi, không thể tính `.dt.year`.

IV.4. Step 2: dim_products (Sản phẩm enriched)

Bắt đầu từ `products.csv` (2,412 SP), bổ sung 3 nhóm thông tin:

```

1 # a) Margin tại cấp sản phẩm
2 dim_products = products.copy()
3 dim_products["gross_margin_pct"] = (
4     (dim_products["price"] - dim_products["cogs"]) / dim_products["price"] * 100
5 ).round(2)
6 dim_products["profit_per_unit"] = (dim_products["price"] - dim_products["cogs"]).round(
7     2)
8
9 # b) Join avg rating từ reviews
10 avg_rating = reviews.groupby("product_id")["rating"].agg(["mean", "count"]).reset_index(
11     )
12 avg_rating.columns = ["product_id", "avg_rating", "review_count"]
13
14 # c) Join return stats từ returns
15 ret_stats = returns.groupby("product_id").agg(
16     total_returns = ("return_id", "count"),
17     total_return_qty = ("return_quantity", "sum"),
18     total_refund = ("refund_amount", "sum")
19 ).reset_index()
20
21 dim_products = dim_products.merge(avg_rating, on="product_id", how="left")
22 dim_products = dim_products.merge(ret_stats, on="product_id", how="left")
23
24 # d) Xử lý NaN
25 dim_products[["review_count", "total_returns", "total_return_qty"]] = \
26     dim_products[["review_count", "total_returns", "total_return_qty"]].fillna(0).
27     astype(int)
28 dim_products["total_refund"] = dim_products["total_refund"].fillna(0)

```

.copy() tạo bản sao để không sửa DataFrame gốc. .merge(how="left") giữ nguyên tất cả 2,412 SP; SP chưa có review sẽ là NaN. .fillna(0) thay NaN bằng 0.

IV.5. Step 3–4: dim_geography, dim_promotions

```

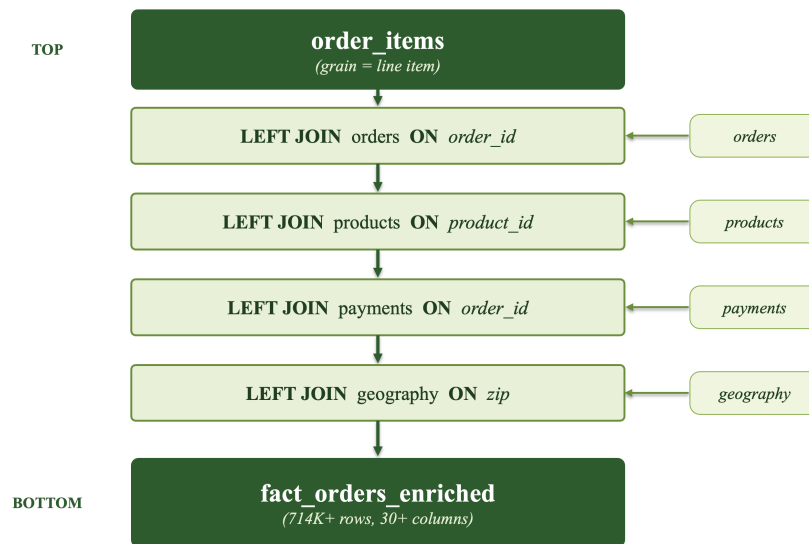
1 save(geography, "dim_geography.csv") # 39,948 dòng
2 save(promotions, "dim_promotions.csv") # 50 dòng

```

Copy nguyên, không xử lý. Tableau dùng làm bảng tham chiếu khi cần filter theo vùng hoặc loại promotion.

IV.6. Step 5: fact_orders_enriched (Bảng fact trung tâm)

Bước quan trọng nhất. Grain = từng line item.

JOIN Pipeline — *fact_orders_enriched*

Pipeline áp dụng LEFT JOIN lần lượt lên bảng fact chính để làm giàu thuộc tính mà không làm thay đổi grain (line-item level).

Hình 6: Pipeline JOIN 5 bảng để tạo *fact_orders_enriched*.

```

1 fact = order_items.copy()
2 fact = fact.merge(
3     orders[["order_id", "order_date", "customer_id", "zip",
4             "order_status", "payment_method", "device_type", "order_source"]],
5     on="order_id", how="left")
6 fact = fact.merge(
7     products[["product_id", "product_name", "category", "segment",
8               "size", "color", "price", "cogs"]],
9     on="product_id", how="left")
10 fact = fact.merge(
11     payments[["order_id", "payment_value", "installments"]],
12     on="order_id", how="left")
13 fact = fact.merge(
14     geography[["zip", "city", "region"]].drop_duplicates(subset="zip"),
15     on="zip", how="left")
  
```

💡 Tại sao dùng LEFT JOIN?

Một số đơn hàng có thể thiếu geography hoặc payment. LEFT JOIN giữ nguyên tất cả line items; cột thiếu sẽ là NULL. Nếu dùng INNER JOIN, chúng ta mất dữ liệu.

Computed fields:

```

1 fact["line_revenue"]      = (fact["quantity"] * fact["unit_price"]).round(2)
2 fact["line_cost"]        = (fact["quantity"] * fact["cogs"]).round(2)
3 fact["line_gross_profit"] = (fact["line_revenue"] - fact["line_cost"]).round(2)
4 fact["line_margin_pct"]   = np.where(
5     fact["line_revenue"] > 0,
6     (fact["line_gross_profit"] / fact["line_revenue"] * 100).round(2), 0)
7
8 fact["has_promo"]         = (~fact["promo_id"].isna() & (fact["promo_id"] != '')).astype(int)
9
10 fact["discount_pct"]     = np.where(
11     fact["line_revenue"] + fact["discount_amount"] > 0,
12     (fact["discount_amount"] / (fact["line_revenue"] + fact["discount_amount"]) * 100).
13         round(2), 0)

```

np.where(condition, true, false) tương đương IF-ELSE, tránh chia cho 0.

Date dimensions:

```

1 fact["order_year"]       = fact["order_date"].dt.year
2 fact["order_month"]      = fact["order_date"].dt.month
3 fact["order_quarter"]    = fact["order_date"].dt.quarter
4 fact["order_dow"]        = fact["order_date"].dt.dayofweek    # 0=Monday
5 fact["order_dow_name"]   = fact["order_date"].dt.day_name()
6 fact["order_ym"]         = fact["order_date"].dt.to_period("M").astype(str)

```

IV.7. Step 6: fact_returns_enriched

```

1 fact_ret = returns.copy()
2 fact_ret = fact_ret.merge(
3     products[["product_id", "product_name", "category", "segment",
4         "size", "color", "price", "cogs"]],
5     on="product_id", how="left")
6 fact_ret = fact_ret.merge(
7     orders[["order_id", "order_date", "customer_id", "order_source", "device_type"]],
8     on="order_id", how="left")
9 fact_ret["days_to_return"] = (fact_ret["return_date"] - fact_ret["order_date"]).dt.days

```

days_to_return = số ngày từ lúc mua đến lúc trả. Category bị trả sớm (lỗi SP) vs trả muộn (đổi ý).

IV.8. Step 7: RFM Segmentation (phức tạp nhất)

Khái Niệm: RFM Analysis

RFM phân khúc KH dựa trên 3 hành vi: **Recency** (mua gần đây?), **Frequency** (mua thường xuyên?), **Monetary** (mua giá trị?). Mỗi chiều chia thành 5 quintile.

Phần 1–2: Tính R, F, M + gán quintile

```

1 ref_date = orders["order_date"].max() + pd.Timedelta(days=1)
2 rfm = orders.merge(payments[["order_id", "payment_value"]], on="order_id", how="left")
3 rfm = rfm[rfm["order_status"] != "cancelled"]
4
5 rfm_agg = rfm.groupby("customer_id").agg(
6     last_order_date = ("order_date", "max"),
7     first_order_date = ("order_date", "min"),
8     frequency        = ("order_id", "nunique"),
9     monetary         = ("payment_value", "sum"),
10    avg_order_value   = ("payment_value", "mean"),
11 ).reset_index()
12 rfm_agg["recency_days"] = (ref_date - rfm_agg["last_order_date"]).dt.days
13 rfm_agg["tenure_days"]  = (ref_date - rfm_agg["first_order_date"]).dt.days
14
15 rfm_agg["R_score"] = pd.qcut(rfm_agg["recency_days"], q=5, labels=[5,4,3,2,1]).astype(
16                                     int)
17 rfm_agg["F_score"] = pd.qcut(rfm_agg["frequency"].rank(method="first"), q=5, labels=[1,
18                                     2,3,4,5]).astype(int)
19 rfm_agg["M_score"] = pd.qcut(rfm_agg["monetary"].rank(method="first"), q=5, labels=[1,2
20                                     ,3,4,5]).astype(int)
21 rfm_agg["RFM_score"] = rfm_agg["R_score"]*100 + rfm_agg["F_score"]*10 + rfm_agg["
22                                     M_score"]

```

Lưu ý

RFM loại cancelled. fact_orders_enriched giữ lại tất cả đơn. Nếu cần revenue “net”, thêm filter trong worksheet.

Phần 3: Gán segment label

```

1 def rfm_segment(row):
2     r, f, m = row["R_score"], row["F_score"], row["M_score"]
3     if r >= 4 and f >= 4:         return "Champions"
4     elif r >= 3 and f >= 3:       return "Loyal Customers"
5     elif r >= 4 and f <= 2:       return "New Customers"
6     elif r >= 3 and m >= 3:       return "Potential Loyalists"
7     elif r <= 2 and f >= 3:       return "At Risk"
8     elif r <= 2 and f >= 4 and m >= 4: return "Cant Lose Them"
9     elif r <= 2 and f <= 2:       return "Lost"
10    elif r == 3 and f <= 2:        return "About To Sleep"
11    else:                          return "Need Attention"
12
13 rfm_agg["rfm_segment"] = rfm_agg.apply(rfm_segment, axis=1)

```

Phần 4: Merge demographics

```

1 dim_cust = customers.merge(rfm_agg, on="customer_id", how="left")
2 dim_cust = dim_cust.merge(
3     geography[["zip", "region"]].drop_duplicates(subset="zip"),
4     on="zip", how="left")
5 dim_cust["frequency"] = dim_cust["frequency"].fillna(0).astype(int)
6 dim_cust["monetary"] = dim_cust["monetary"].fillna(0)
7 dim_cust["rfm_segment"] = dim_cust["rfm_segment"].fillna("Never Purchased")

```

LEFT JOIN customers (121,930) với rfm_agg (chỉ KH đã mua). KH chưa mua → NaN → segment “Never Purchased”.

Kết quả phân khúc:

Segment	Số KH	%	Ý nghĩa
Never Purchased	33,807	27.7	Đăng ký nhưng chưa mua
Lost	24,996	20.5	Đã lâu không quay lại
Champions	24,490	20.1	Mua gần, thường xuyên
Loyal Customers	18,163	14.9	Khách trung thành
At Risk	10,221	8.4	Từng mua nhiều, giờ thưa
New Customers	4,865	4.0	Mới mua
About To Sleep	4,222	3.5	Đang giảm tương tác
Potential Loyalists	1,166	1.0	Có tiềm năng

IV.9. Step 8: Shipments enriched

```

1 fact_ship = shipments.copy()
2 fact_ship["delivery_days"] = (fact_ship["delivery_date"] - fact_ship["ship_date"]).dt.
3     .days
4 fact_ship = fact_ship.merge(
5     orders[["order_id", "order_date", "zip", "order_status"]],
6     on="order_id", how="left")
7 fact_ship["processing_days"] = (fact_ship["ship_date"] - fact_ship["order_date"]).dt.
8     .days
9 fact_ship["total_lead_time"] = (fact_ship["delivery_date"] - fact_ship["order_date"]).
10    dt.days
11 fact_ship["delivery_bucket"] = pd.cut(
12    fact_ship["delivery_days"],
13    bins=[-1, 2, 5, 7, 14, 999],
14    labels=["1-2 days", "3-5 days", "6-7 days", "8-14 days", "15+ days"])

```

pd.cut() phân nhóm giá trị liên tục vào bucket rời rạc. Tableau dùng delivery_bucket để tạo histogram.

i Lưu ý

Dataset hiện tại chỉ có giá trị ở “1–2 days”, “3–5 days”, “6–7 days”. Hai bucket sau reserve cho data tương lai.

IV.10. Step 9: fact_sales_daily (Revenue + trends)

```

1 fact_sales = sales.copy()
2 fact_sales.columns = ["date", "revenue", "cogs"]
3 fact_sales["gross_profit"] = (fact_sales["revenue"] - fact_sales["cogs"]).round(2)
4 fact_sales["gross_margin_pct"] = (
5     fact_sales["gross_profit"] / fact_sales["revenue"] * 100).round(2)
6 fact_sales["year_month"] = fact_sales["date"].dt.to_period("M").astype(str)
7
8 # Rolling averages
9 fact_sales = fact_sales.sort_values("date")
10 fact_sales["revenue_ma7"] = fact_sales["revenue"].rolling(7, min_periods=1).mean().
11                             round(2)
12 fact_sales["revenue_ma30"] = fact_sales["revenue"].rolling(30, min_periods=1).mean().
13                             round(2)
14
15 # YoY growth
16 fact_sales["revenue_ly"] = fact_sales["revenue"].shift(365)
17 fact_sales["yoy_growth_pct"] = np.where(
18     fact_sales["revenue_ly"] > 0,
19     ((fact_sales["revenue"] - fact_sales["revenue_ly"]) / fact_sales["revenue_ly"] *
20      100).round(2),
21     np.nan)
22 fact_sales.drop(columns=["revenue_ly"], inplace=True)

```

Khái Niệm: Moving Average (MA)

Moving Average tính trung bình N ngày liên tiếp. MA7 loại nhiễu ngày-quá-ngày. MA30 cho xu hướng dài hạn. min_periods=1 giúp N ngày đầu không NULL.

i Lưu ý

yoy_growth_pct null 365 ngày đầu (không có năm trước để so). Nếu overlay YoY trên chart, filter date ≥ 2013-07-04.

IV.11. Step 10: Inventory + health labels

```

1 fact_inv = inventory.copy()
2 fact_inv = fact_inv.merge(
3     products[["product_id", "price", "cogs"]].rename(
4         columns={"price": "product_price", "cogs": "product_cogs"}),
5     on="product_id", how="left")

```

```

6 fact_inv["inventory_value_retail"] = (fact_inv["stock_on_hand"] * fact_inv["
    product_price"]).round(2)
7 fact_inv["inventory_value_cost"] = (fact_inv["stock_on_hand"] * fact_inv["
    product_cogs"]).round(2)
8
9 def inv_health(row):
10     if row["stockout_flag"] == 1: return "Stockout"
11     elif row["overstock_flag"] == 1: return "Overstock"
12     elif row["reorder_flag"] == 1: return "Reorder Needed"
13     else: return "Healthy"
14
15 fact_inv["inventory_health"] = fact_inv.apply(inv_health, axis=1)

```

IV.12. Step 11: Web traffic

```

1 fact_web = web_traffic.copy()
2 fact_web.rename(columns={"date": "traffic_date"}, inplace=True)
3 fact_web["pages_per_session"] = (fact_web["page_views"] / fact_web["sessions"]).round(2)

```

Đổi date → traffic_date vì “date” là keyword trong SQL.

IV.13. Step 12: Cohort Retention

Khái Niệm: Cohort Analysis

Cohort = nhóm KH có cùng tháng mua đầu tiên. **Retention** = % KH quay lại mua ở tháng thứ N. Ma trận cho thấy “KH tháng 3/2020, 6 tháng sau còn bao nhiêu % quay lại?”

```

1 # 1. Xác định cohort = tháng mua đầu tiên
2 first_order = orders[orders["order_status"] != "cancelled"] \
3     .groupby("customer_id")["order_date"].min().reset_index()
4 first_order.columns = ["customer_id", "cohort_date"]
5 first_order["cohort_month"] = first_order["cohort_date"].dt.to_period("M")
6
7 # 2. Tính period_number
8 orders_cohort = orders[orders["order_status"] != "cancelled"] [
9     ["order_id", "customer_id", "order_date"]].copy()
10 orders_cohort["order_month"] = orders_cohort["order_date"].dt.to_period("M")
11 orders_cohort = orders_cohort.merge(
12     first_order[["customer_id", "cohort_month"]], on="customer_id")
13 orders_cohort["period_number"] = (
14     (orders_cohort["order_month"].dt.year - orders_cohort["cohort_month"].dt.year) * 12
15     +
16     (orders_cohort["order_month"].dt.month - orders_cohort["cohort_month"].dt.month))

```

```

16
17 # 3. Tính retention rate
18 cohort_sizes = orders_cohort.groupby("cohort_month")["customer_id"].nunique().
                        reset_index()
19 cohort_sizes.columns = ["cohort_month", "cohort_size"]
20
21 retention = orders_cohort.groupby(["cohort_month", "period_number"]) \
22     ["customer_id"].nunique().reset_index()
23 retention.columns = ["cohort_month", "period_number", "active_customers"]
24 retention = retention.merge(cohort_sizes, on="cohort_month")
25 retention["retention_rate"] = (
26     retention["active_customers"] / retention["cohort_size"] * 100).round(2)
27 retention = retention[retention["period_number"] <= 24]

```

Kết quả: hàng = cohort_month, cột = period_number (0–24), giá trị = retention_rate (%).

IV.14. Step 13–14: Bảng tổng hợp

Step 13: agg_monthly_summary

```

1 monthly = fact_sales.groupby("year_month").agg(
2     revenue      = ("revenue", "sum"),
3     cogs          = ("cogs", "sum"),
4     gross_profit = ("gross_profit", "sum"),
5     avg_daily_rev = ("revenue", "mean"),
6     days_in_month = ("revenue", "count"),
7 ).reset_index()
8 monthly["revenue_prev"] = monthly["revenue"].shift(1)
9 monthly["mom_growth_pct"] = np.where(
10     monthly["revenue_prev"] > 0,
11     ((monthly["revenue"] - monthly["revenue_prev"]) / monthly["revenue_prev"] * 100).
12         round(2),
13     np.nan)

```

.shift(1) lấy giá trị tháng trước. mom_growth_pct = % thay đổi MoM. Dashboard 1 Chart 3 dùng trường này.

Step 14: agg_reviews_summary

```

1 # Enrich reviews voi category tu products
2 reviews_enriched = reviews.merge(
3     products[["product_id", "category", "segment"]],
4     on="product_id", how="left")
5 reviews_enriched["review_ym"] = reviews_enriched["review_date"].dt.to_period("M").
6     astype(str)
7
8 rev_summary = reviews_enriched.groupby(["review_ym", "category"]).agg(
9     avg_rating    = ("rating", "mean"),
10    review_count   = ("review_id", "count"),

```

```
10     pct_5_star    = ("rating", lambda x: (x == 5).mean() * 100),  
11     pct_1_star    = ("rating", lambda x: (x == 1).mean() * 100),  
12 ).reset_index()
```

`lambda x: (x == 5).mean() * 100` = tỷ lệ % review 5 sao. Bảng này (504 dòng) dùng cho Dashboard 3 Rating Trend.

V. Thiết lập Tableau Public

V.1. Tạo Data Source

1. Mở Tableau Public Desktop
2. Chọn **Connect** → **Text File**
3. Chọn file cần dùng (ví dụ `fact_sales_daily.csv`)

Tách data source riêng cho mỗi dashboard thay vì gộp tất cả (Tableau Public giới hạn performance):

Dashboard	Files cần load
1: Revenue	<code>fact_sales_daily</code> + <code>agg_monthly_summary</code> + <code>fact_orders_enriched</code>
2: Customer	<code>dim_customers_rfm</code> + <code>agg_cohort_retention</code>
3: Product	<code>fact_orders_enriched</code> + <code>dim_products</code> + <code>fact_returns_enriched</code> + <code>agg_reviews_summary</code>
4: Marketing	<code>fact_orders_enriched</code> + <code>fact_web_traffic</code> + <code>dim_promotions</code>
5: Operations	<code>fact_inventory</code> + <code>fact_shipments_enriched</code>

V.2. Kiểm tra Data Types

Cột	Kiểu đúng	Lưu ý
<code>order_date</code> , <code>return_date</code>	Date	Nếu hiện String, click icon để đổi
<code>zip</code> , <code>customer_id</code> , <code>order_id</code>	String (Dimension)	Không để Number
<code>year_month</code> , <code>order_ym</code>	String	Dùng làm discrete axis
<code>has_promo</code> , <code>stockout_flag</code>	Number (Boolean)	0/1

V.3. Template layout chung

Mỗi dashboard dùng layout chung:

```

Dashboard ( bg #F6F7F9)
|- Banner (Horizontal, h=72, bg #1F3A5F, text trắng 22pt)
  |- Filter bar (Horizontal, h=44, bg white)
  |- KPI Strip (Horizontal, h=120, 4 KPI cards)
+- Chart Grid 2x2 (Vertical → 2 Horizontal rows, h~382/row)
  
```

💡 Layout panel

Dùng **Layout panel** (trái dưới) để xem cây containers. Click container để chỉnh Padding, Background, Size. Khi kéo worksheet sai vị trí, layout tree dễ dùng hơn click trực tiếp trên canvas.

V.4. Dashboard Actions

Dashboard Action cho phép click/hover vào chart này để filter/highlight chart khác:

1. Menu **Dashboard** → **Actions** → **Add Action**
2. Chọn loại: **Filter** (click drill-down), **Highlight** (hover), hoặc **Go to Sheet** (nav button)
3. Chỉ định Source sheets, Target sheets, Run on (Select/Hover), và Target Filters

V.5. KPI Delta: cách tính “vs prior year”

Thuật ngữ Tableau dùng trong phần này

- **LOD** (Level of Detail): biểu thức `{FIXED ... : ...}` tính aggregation ở cấp khác với chart
- **COUNTD**: đếm giá trị duy nhất (không đếm trùng)
- **Quick Table Calculation**: phép tính nhanh trên cell so với cell khác
- **Marks Card**: bảng điều khiển chứa Color/Size/Text/Label/Detail/Tooltip

Pattern chính (LOD Expression):

```
1 Current Year Rev = { FIXED : SUM(IIF(YEAR([date]) = { MAX(YEAR([date]))
    }, [revenue], 0)) }
2 Prior Year Rev   = { FIXED : SUM(IIF(YEAR([date]) = { MAX(YEAR([date]))
    } - 1, [revenue], 0)) }
3 Revenue YoY Delta = ([Current Year Rev] - [Prior Year Rev]) / [Prior Year
    Rev] * 100
```

`{FIXED : ...}` scope toàn bộ dataset, không phụ thuộc filter. `MAX(YEAR([date]))` = năm mới nhất (2022).

Hiển thị delta trên KPI card:

1. Kéo delta measure vào Text, dòng 2, size 12pt
2. Conditional color: `IIF([Delta] >= 0, "up", "down")` → green/red
3. Custom format: ▲ +0.0%; ▼ -0.0%

 Test delta nhanh

Kéo field vào Marks Card → Text → xem giá trị. Nếu ra NULL: kiểm tra (a) `MAX(YEAR([date]))` hoạt động không, (b) Prior year có dữ liệu không, (c) Divisor có `= 0` không.

VI. Dashboard 1: Revenue & Profitability

Câu hỏi chủ đạo: “10 năm kinh doanh, doanh nghiệp tăng trưởng thế nào? Lợi nhuận có bền vững?”

Chart	Data source
KPI Cards + Chart 1–2	fact_sales_daily.csv
Chart 3 (MoM Heatmap)	agg_monthly_summary.csv
Chart 4 (Category Contribution)	fact_orders_enriched.csv

Dashboard gồm 4 KPI cards + 4 charts, mỗi chart ở 1 cấp phân tích.

VI.1. 4 KPI Cards

KPI 1: Total Revenue font Accent #2E86AB. BAN: SUM([revenue]). Format: K/M/B.

```

1 Current Year Rev = { FIXED : SUM(IIF(YEAR([date]) = { MAX(YEAR([date]))
    }, [revenue], 0)) }
2 Prior Year Rev   = { FIXED : SUM(IIF(YEAR([date]) = { MAX(YEAR([date]))
    } - 1, [revenue], 0)) }
3 Revenue YoY Delta = ([Current Year Rev] - [Prior Year Rev]) / [Prior Year
    Rev] * 100

```

KPI 2: Gross Profit font Accent #6BBF59. BAN: SUM([gross_profit]). Delta: cùng pattern, thay revenue bằng gross_profit.

KPI 3: Gross Margin % BAN: SUM([gross_profit]) / SUM([revenue]) * 100. Format: 0.0%. Delta dùng pp (percentage-point):

```

1 Current Year Margin = SUM(IIF(YEAR([date])={MAX(YEAR([date]))}, [
    gross_profit], 0))
2                               / SUM(IIF(YEAR([date])={MAX(YEAR([date]))}, [revenue
    ], 0)) * 100
3 Prior Year Margin   = SUM(IIF(YEAR([date])={MAX(YEAR([date]))}-1, [
    gross_profit], 0))
4                               / SUM(IIF(YEAR([date])={MAX(YEAR([date]))}-1, [
    revenue], 0)) * 100
5 Margin pp Delta = [Current Year Margin] - [Prior Year Margin]

```

Lưu ý

pp vs %: Nếu margin từ 22% xuống 14%, pp delta = -8 pp (không phải -36%). Format: ▲ +0.0 “pp”; ▼ -0.0 “pp”.

KPI 4: Revenue CAGR (2013 → 2022) font Accent #1F3A5F.

```

1 annual_rev_2013 = { FIXED : SUM(IIF(YEAR([date]) = 2013, [revenue], 0)) }
2 annual_rev_2022 = { FIXED : SUM(IIF(YEAR([date]) = 2022, [revenue], 0)) }
3 CAGR = (POWER([annual_rev_2022] / [annual_rev_2013], 1.0/9) - 1) * 100

```

Format: 0.0%. Không có delta. Dưới BAN ghi chú tính “9-year compound”.

💡 Tại sao 2013–2022, không phải 2012–2022?

fact_sales_daily bắt đầu từ 2012-07-04 (nửa năm cuối 2012), nên tổng 2012 không đại diện. Dừng 2013–2022 = 9 năm đầy đủ.

VI.2. Chart 1: Revenue Trend (Descriptive)

Khái Niệm: Dual Axis

Dual Axis đặt 2 measure trên 2 trục Y. Bật: right-click measure thứ 2 → **Dual Axis**. **Synchronize** khi cùng đơn vị. **KHÔNG Synchronize** khi khác đơn vị (sessions vs bounce_rate).

Loại: Dual-axis Line. **Build:**

1. New Worksheet → rename viz_D1_01_revenue_trend
2. Kéo date vào Columns → **Month (continuous, xanh lá)**. **KHÔNG** chọn discrete
3. Kéo revenue vào Rows → line mỏng
4. Kéo revenue_ma30 vào Rows → right-click → **Dual Axis**
5. Right-click trục phải → **Synchronize Axis**
6. Line revenue: color #2E86AB55 (nhạt), width 1
7. Line revenue_ma30: color #1F3A5F (navy đậm), width 2
8. Reference Line: right-click trục 1 → Add Reference Line → Average, color #5D6D7E dashed

Insight mẫu: “Revenue trung bình giảm dần qua 10 năm, CAGR khoảng –3.8%/năm.”

💡 Lưu ý

yoy_growth_pct null 365 ngày đầu (2012-07-04 → 2013-07-03). Nếu overlay YoY, giai đoạn đầu trống. Workaround: filter date ≥ 2013-07-04.

VI.3. Chart 2: Gross Margin Trend (Diagnostic)

Loại: Area Chart + Trend Line. **Build:**

1. New Worksheet → rename viz_D1_02_margin_trend

2. Kéo `date` vào Columns (Month continuous)
3. Kéo `gross_margin_pct` vào Rows (AVG)
4. Marks card: chuyển sang **Area**
5. Color: #6BBF59 solid green. Area fill opacity: 20%
6. Analytics pane → kéo **Trend Line** → Linear, color #5D6D7E dashed

Insight mẫu: “Gross Margin giảm từ ~22% xuống ~14%. Nguyên nhân: cạnh tranh giá hoặc COGS tăng nhanh hơn revenue.”

VI.4. Chart 3: MoM Growth Heatmap (Predictive)

Loại: Highlight Table (Calendar Heatmap). Data: `agg_monthly_summary`.

Lưu ý

`year_month` là String “YYYY-MM”. Không thể dùng Part → Year/Month trực tiếp. Tách thành 2 field.

Build:

1. New Worksheet → rename `viz_D1_03_mom_heatmap`
2. Tạo calc: MoM Year = LEFT([year_month], 4) (String, discrete)
3. Tạo calc: MoM Month Num = INT(MID([year_month], 6, 2)) (Integer, discrete)
4. MoM Year vào Columns, MoM Month Num vào Rows (sort ascending)
5. `mom_growth_pct` vào Color (AVG). Marks: **Square**
6. Color: Diverging: Red #A8312C ← White #F4F4F4 → Green #2F855A
7. Label: `mom_growth_pct`, format 0%

Insight mẫu: “Q1 mỗi năm có growth âm so với Q4, mùa vụ rõ. Dự kiến Q1/2023 tiếp tục pattern.”

Lưu ý

`mom_growth_pct` của 2012-07 = NULL (không có tháng trước). Ô (Jul, 2012) sẽ trống.

VI.5. Chart 4: Category Revenue Contribution (Prescriptive)

Loại: 100% Stacked Bar. Data: `fact_orders_enriched`.

1. Kéo `order_year` vào Columns (discrete)
2. Kéo `line_revenue` vào Rows (SUM)

3. Right-click → Quick Table Calculation → Percent of Total
4. Right-click → **Edit Table Calculation** → Compute Using: chọn category (KHÔNG để Table)
5. Kéo category vào Color. Marks → Bar. Label: 0.0%

💡 Tại sao Compute Using category?

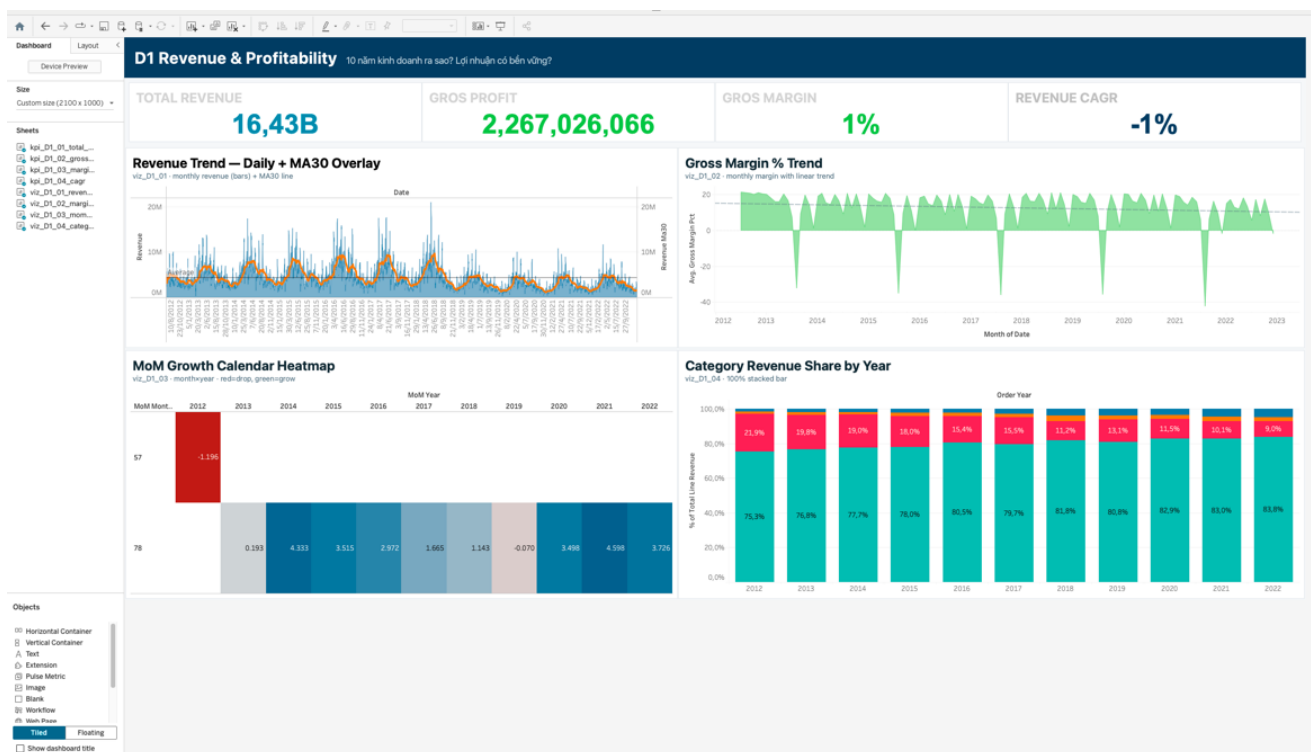
Compute Using category = tính % trong phạm vi mỗi year → tổng 4 category = 100% mỗi năm. Nếu chọn Table (default), % tính trên toàn 11 năm, mất ý nghĩa market share.

💡 Lưu ý

Kiểm chứng nhanh: Hover qua 1 bar → tổng tất cả segment phải đúng **100.0%** cho mỗi năm. Nếu ra khác, Compute Using đang sai.

Insight mẫu: “Nếu category doanh thu cao nhất đồng thời margin thấp nhất, đề xuất pivot mix: giảm tỷ trọng đó, tăng đầu tư category margin cao.”

VI.6. Ghép Dashboard 1



Hình 7: Wireframe layout Dashboard 1: Revenue & Profitability: Fixed 2100 × 1000.

Các bước ghép:

1. Dashboard → New Dashboard → Size Fixed 2100×1000
2. Kéo Horizontal container: Banner h=72, bg #1F3A5F, text 22pt white
3. Kéo Horizontal container: Filter bar h=44, chứa filter `flt_G_year` (multi-select dropdown)
4. Kéo Horizontal container: KPI strip h=120 → kéo 4 KPI worksheet, width 25% mỗi card
5. Kéo Vertical container → 2 Horizontal rows:
 - Row 1: `viz_D1_01` (w=1040) + `viz_D1_02` (w=1040)
 - Row 2: `viz_D1_03` (w=1040) + `viz_D1_04` (w=1040)
6. Text object footer ở đáy

VII. Dashboard D2

Customer Segmentation & Lifecycle

Dashboard D2 tập trung giải quyết bài toán: “Ai là khách hàng giá trị? Ai đang rời đi?”. Phần này hướng dẫn xây dựng hệ thống phân tích dựa trên phương pháp RFM (Recency, Frequency, Monetary) và ma trận Cohort Retention để theo dõi vòng đời khách hàng.

VII.1. Tổ chức & Chuẩn bị Dữ liệu

VII.1.1. Các file CSV cần thiết

Việc phân tích khách hàng đòi hỏi sự kết hợp giữa thông tin định danh và lịch sử giao dịch đã được làm giàu (enriched):

File	Vai trò trong dashboard
<code>dim_customers_rfm.csv</code>	Nguồn chính: Chứa <code>rfm_segment</code> , <code>R/F/M_score</code> , <code>monetary</code> (LTV).
<code>agg_cohort_retention.csv</code>	Dùng cho ma trận giữ chân khách hàng (Retention Matrix).
<code>orders.csv</code>	Thông tin về <code>order_date</code> để tính toán các chỉ số về thời gian.
<code>geography.csv</code>	Cung cấp thông tin <code>region</code> , <code>city</code> để filter theo địa lý.

VII.1.2. Kết nối dữ liệu trong Tableau

1. Mở Tableau → **Connect** → chọn `dim_customers_rfm.csv` làm bảng trung tâm.
2. Kết nối (Relationship) với `geography.csv` qua trường `zip` để hỗ trợ filter vùng miền.

3. Sử dụng `agg_cohort_retention.csv` làm **Data Source riêng biệt** để tránh làm nhiễu grain dữ liệu của bảng RFM.

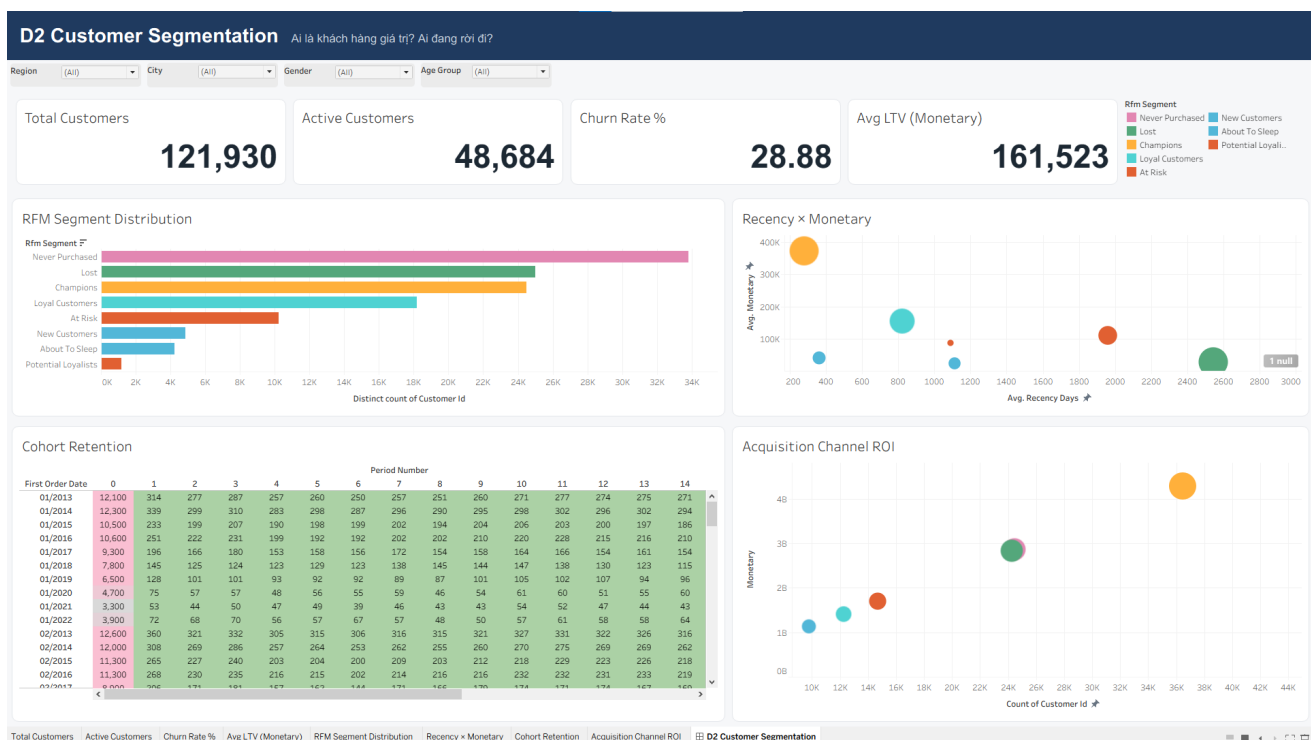
Lưu ý

Grain của dữ liệu khách hàng: Bảng `dim_customers_rfm` có grain là **1 dòng = 1 khách hàng**. Khi tính toán các chỉ số như Total Customers, cần sử dụng `COUNTD([customer_id])` hoặc `COUNT([customer_id])` để đảm bảo tính chính xác.

VII.1.3. Các cột quan trọng trong Dashboard 2

Cột	Kiểu	Dùng cho
<code>rfm_segment</code>	String	Phân lớp khách hàng (Champions, Lost,...).
<code>recency_days</code>	Float	Số ngày kể từ lần mua cuối.
<code>monetary</code>	Float	Tổng giá trị mua sắm trọn đời (LTV).
<code>signup_date</code>	Date	Thời điểm khách hàng đăng ký hệ thống.
<code>active_status</code>	Integer	Flag xác định khách hàng còn hoạt động hay không.

VII.2. Xây dựng dashboard



Hình 8: Dashboard D2: Customer Segmentation & Lifecycle

VII.2.1. Calculated Fields cho 4 KPI Cards

```

1 // [Total Customers]
2 COUNTD([customer\_id]) // Expected ~121.9K
3
4 // [Active Customers]
5 // Khách hàng thuộc nhóm: Champions, Loyal, Potential Loyalists, New
  Customers
6 SUM(
7   IF [rfm\_segment] IN ("Champions", "Loyal Customers", "New Customers",
8     "Potential Loyalists")
9   THEN 1 ELSE 0 END
10 )
11 // [Churn Rate %]
12 // Khách hàng thuộc nhóm: Lost, At Risk
13 SUM(
14   IF [rfm\_segment] IN ("Lost", "At Risk")
15   THEN 1 ELSE 0 END
16 )
17 / COUNTD([customer\_id])
18
19 // [Avg LTV]
20 // Giá trị mua hàng trung bình loại trừ nhóm Never Purchased
21 AVG(
22   IIF([rfm\_segment] != "Never Purchased", [monetary], NULL)
23 )

```

VII.2.2. Calculated Fields cho 4 Visualization

Viz D2_01 - RFM Segment Distribution (Descriptive)

Cấu hình chart: **Horizontal Bar**, Rows = [rfm_segment], Columns = COUNT([customer_id]), Color = [rfm_segment].

Sắp xếp (Sort): Giảm dần theo số lượng khách hàng để thấy rõ các nhóm chiếm tỷ trọng lớn như *Never Purchased* (27.7%) và *Lost* (20.5%).

Viz D2_02 - Recency vs Monetary Scatter (Diagnostic)

```

1 // X-Axis: AVG([recency_days])
2 // Y-Axis: AVG([monetary])
3 // Detail: [rfm_segment]
4 // Size:    COUNT([customer_id])

```

Cấu hình chart: **Scatter Plot**. Lưu ý right-click trục Recency → chọn **Reversed** để khách hàng mua gần đây (Recency thấp) nằm bên phải, đúng với logic của ma trận RFM chuẩn.

Viz D2_03 - Cohort Retention Heatmap (Predictive)

Sử dụng bảng `agg_cohort_retention`.

- Columns: Month Since First Order (0 đến 24).
- Rows: Cohort Month (tháng khách hàng bắt đầu mua).
- Label & Color: Retention Rate %.

Sử dụng dải màu *Sequential* (ví dụ: Teal hoặc Navy) để làm nổi bật các điểm rơi (drop-off) của khách hàng theo thời gian.

Viz D2_04 - Acquisition Channel ROI (Prescriptive)

```
1 // [Avg LTV per Channel]
2 SUM([monetary]) / COUNTD([customer_id])
3
4 // [Total Revenue per Channel]
5 SUM([monetary])
```

Cấu hình chart: **Bubble Chart** hoặc **Scatter**.

X-axis = Số lượng khách hàng (Volume) Y-axis = Avg LTV Size = Total Revenue

Chart này giúp CEO xác định kênh nào mang về khách hàng “chất lượng” (LTV cao) thay vì chỉ nhìn vào số lượng (Volume).

VII.3. Chiến lược ghi điểm cho Dashboard 2

💡 Insight từ dữ liệu thực tế

Dữ liệu cho thấy tỷ lệ khách hàng không hoạt động (Never Purchased + Lost) lên tới gần 48%.

Thay vì tập trung tìm khách hàng mới, hãy đề xuất chiến dịch “*Win-back*” cho nhóm *At Risk* và “*Activation*” cho nhóm *Never Purchased* để tối ưu hóa chi phí vận hành.

Với D1 và D2, chúng ta đã nắm vững quy trình: chuẩn bị data source, tạo calculated field, build chart, ghép dashboard. Giờ chúng ta sẽ nhảy sang **D4 (Marketing)** vì dashboard này mình hoạ thêm 2 kỹ thuật quan trọng: **Data Blending** (dùng web_traffic làm secondary source) và **LOD Expression nâng cao**. D3 (Product) và D5 (Operations) sẽ có gợi ý build ở Section sau.

VIII. Dashboard D4 Marketing & Channel Effectiveness

Dashboard D4 trả lời câu hỏi: “*Kênh nào ROI tốt nhất? Promo có hiệu quả không? Traffic chuyển hoá ra sao?*” Chúng ta sẽ đi qua từng bước: chuẩn bị dữ liệu, tạo calculated field, và build 4 visualization chính.

VIII.1. Tổ chức & Chuẩn bị Dữ liệu

VIII.1.1. Các file CSV cần thiết

Trước khi vào Tableau, cần xác định rõ vai trò của từng file trong bộ dữ liệu được cung cấp:

File	Vai trò trong dashboard
orders.csv	Nguồn chính: order_source, device_type, payment_method
order_items.csv	Doanh thu từng dòng, promo_id để xác định đơn có khuyến mãi
payments.csv	payment_value để tính AOV
web_traffic.csv	Sessions + bounce_rate theo traffic_source
products.csv	Tên category (Casual, GenZ, Outdoor, Streetwear)
promotions.csv	Chi tiết promo (discount_value, promo_type)
geography.csv	Mapping zip → region cho filter vùng

VIII.1.2. Kết nối dữ liệu trong Tableau

1. Mở Tableau → **Connect to Data** → **Text File** → chọn orders.csv làm bảng chính.
2. Kéo order_items.csv vào canvas, dùng **Relationship** (Tableau 2020.2+) liên kết qua order_id.
3. Kéo tiếp payments.csv liên kết qua order_id (quan hệ 1:1).
4. Kéo products.csv liên kết product_id (với order_items) để lấy category.
5. Kéo promotions.csv liên kết promo_id để lấy discount_value, promo_type.
6. Kéo geography.csv liên kết zip (với orders) để lấy region.
7. web_traffic.csv nên đặt làm **secondary data source** riêng, blended với primary qua date = order_date cho viz D4_03.

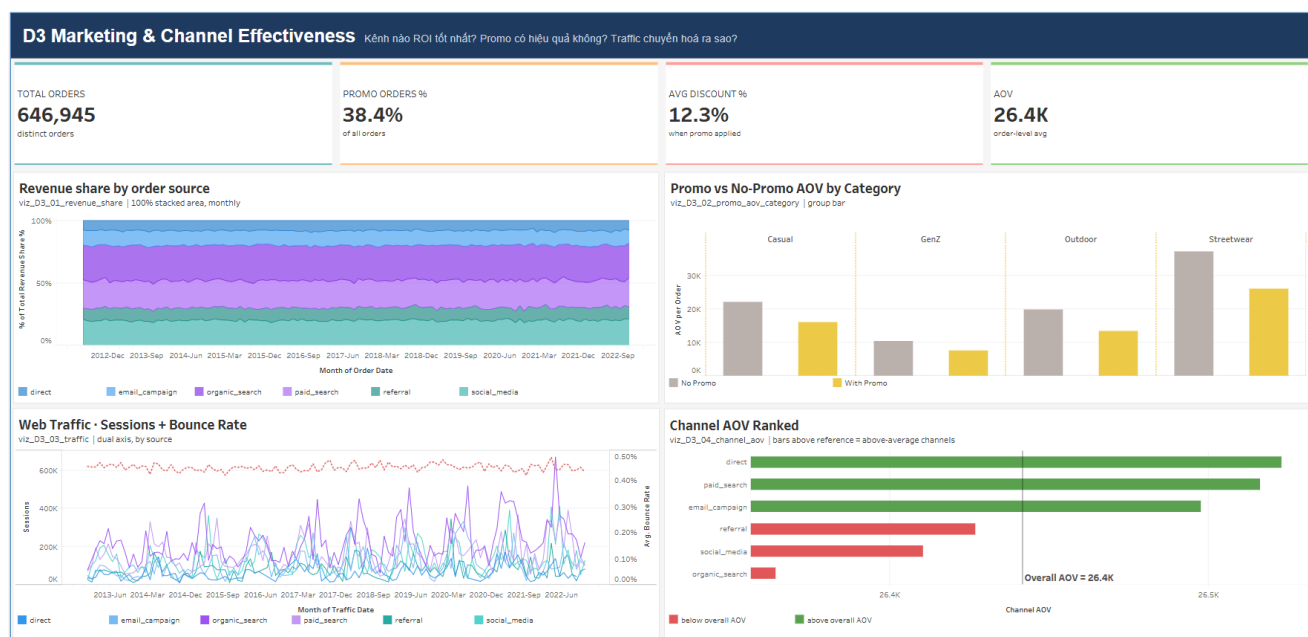
Lưu ý

Relationship vs Join: Với Tableau 2020.2+, nên ưu tiên **Relationship** thay vì Join cổ điển. Relationship giữ granularity của từng bảng riêng biệt, tránh inflate dữ liệu khi join nhiều bảng có cardinality khác nhau (ví dụ: orders 1:N order_items). Nếu dùng Join thông thường, payment_value sẽ bị nhân lên nhiều lần theo số dòng trong order_items.

VIII.1.3. Các cột quan trọng sau khi join

Cột	Kiểu	Dùng cho
order_date	Date	Trục thời gian
order_source	String	Phân tách kênh marketing
device_type	String	Filter thiết bị
payment_method	String	Filter thanh toán
region	String	Filter vùng
payment_value	Float	Tính AOV, Revenue
promo_id	String (nullable)	Xác định đơn có promo
discount_value	Float	Avg discount %
category	String	Phân tách danh mục
order_id	Int	COUNTD → Total Orders

VIII.2. Xây dựng dashboard



Hình 9: Dashboard D4: Marketing & Channel Effectiveness

VIII.2.1. Calculated Fields cho 4 KPI Cards

```

1 // [Total Orders]
2 COUNTD([order_id])
3
4 // [Promo Orders %]
5 COUNTD(IF [has_promo] = 1 THEN [order_id] END)
6 / COUNTD([order_id])
7 // Format: Percentage, 1 decimal --> expected ~38.4%
8
9 // [Avg Discount %]
10 AVG(IF [has_promo] = 1 THEN [discount_value] END)
11 // Chi tinh tren don co promo --> expected ~12.1%
12
13 // [AOV]
14 SUM([payment_value]) / COUNTD([order_id])
15 // Format: #,##0.0K --> expected ~24.2K

```

VIII.2.2. Calculated Fields cho 4 Visualization

Viz D4_01: Revenue Share by Order Source (Descriptive)

```

1 // Dung Quick Table Calculation --> Percent of Total
2 SUM([payment_value])
3 // Partitioning: [order_source]
4 // Addressing: MONTH([order_date])

```

Cấu hình chart: **100% Stacked Area**, Columns = MONTH(order_date) (continuous), Rows = SUM(payment_value) (với Percent of Total Table Calc), Color = [order_source]. Legend hiển thị 6 kênh: *direct, email_campaign, organic_search, paid_search, referral, social_media*.

Viz D4_02: Promo vs No-Promo AOV by Category (Diagnostic)

```

1 // [Promo Label]
2 IF [has_promo] = 1 THEN "With Promo" ELSE "No Promo" END
3
4 // [AOV per Order] -- tu dong aggregate khi dua vao view
5 SUM([payment_value]) / COUNTD([order_id])

```

Cấu hình chart: **Grouped Bar**, Columns = [category], Rows = [AOV per Order], Color = [Promo Label].

Viz D4_03: Web Traffic: Sessions + Bounce Rate (Predictive, Dual Axis)

Dùng web_traffic.csv làm data source riêng, blended với primary qua date.

```

1 // Truc trai: SUM([sessions])
2 // Truc phai: AVG([bounce_rate]) * 100 --> hien thi %
3 // Color by [traffic_source]
4 // Bounce Rate line dung net dut (Dashed), mau do

```

Cấu hình: **Dual-Axis Combo Chart**. Kéo SUM(sessions) vào Rows đầu tiên, rồi kéo AVG(bounce_rate) vào Rows lần thứ 2. Right-click trục phải → **Dual Axis**. **Quan trọng**: right-click trục → **Synchronize Axis = OFF** (hai scale khác nhau hoàn toàn).

Viz D4_04: Channel AOV Ranked (Prescriptive)

```

1 // [Overall AOV] -- Fixed LOD (không bị ảnh hưởng bởi dimension [
  order_source])
2 { FIXED : SUM([payment_value]) / COUNTD([order_id]) }
3
4 // [Channel AOV] -- aggregate theo kênh
5 SUM([payment_value]) / COUNTD([order_id])
6
7 // [Above Average Flag]
8 [Channel AOV] >= [Overall AOV]

```

Cấu hình chart: **Horizontal Bar**, Rows = [order_source] (sorted descending by [Channel AOV]), Columns = [Channel AOV], Color = [Above Average Flag]. Thêm **Reference Line** tại [Overall AOV] (Analytics pane → drag *Reference Line* vào trục X, Value = Overall AOV).

IX. Gợi mở: Dashboard tiếp theo

Sau khi hoàn thành Dashboard 1, 2 và 4, chúng ta đã nắm vững quy trình. Hai dashboard còn lại (D3 và D5) follow cùng pattern.

IX.1. Dashboard D3: Product Analysis

Câu hỏi: “Chiến lược sản phẩm nào hiệu quả? Tại sao hàng bị trả?”. D3 đi sâu vào khía cạnh Product: phân tích lợi nhuận theo danh mục, chất lượng sản phẩm qua reviews, và nguyên nhân trả hàng.

Chart	Loại	Cấp	Gợi ý
Category Contribution	Pie/Donut Chart	Descriptive	Tỷ trọng từng danh mục
Return Rate Hotspot	Highlight Table	Diagnostic	Phân tích return rate theo kích cỡ & danh mục
Review Sentiment	Line Chart	Predictive	Đề xuất hành động từ biểu đồ xu hướng đánh giá
Top & Bottom Products	Bar Chart	Prescriptive	Xếp hạng sản phẩm để đẩy mạnh hoặc loại bỏ

IX.2. Dashboard 5: Operations & Supply Chain

Câu hỏi: “Giao hàng, tồn kho có vấn đề?": Data: fact_inventory + fact_shipments_enriched.

Chart	Loại	Cấp	Gợi ý
Inventory Health	Stacked Bar	Descriptive	3 states: Stockout/Overstock/Healthy
Delivery by Region	Stacked Bar (horizontal)	Diagnostic	processing_days + delivery_days cùng trục
Delivery Bucket	Bar	Predictive	Manual sort: 1-2 → 3-5 → 6-7
Stockout Impact	Scatter	Prescriptive	X = stockout_days, Y = sell_through_rate, Trend line

Insight mục tiêu: “Stockout chiếm 67% snapshots. Top-seller bị stockout nhiều nhất. Đề xuất tăng safety stock top 20 SKUs.”

IX.3. Story: CEO Narrative (6 Story Points)

Sau khi build đủ 5 dashboards, tạo Story ghép thành bản trình bày:

SP	Caption (có con số)	DB
1	Revenue suy giảm: CAGR -3.8% /năm, margin co từ $\sim 22\%$ xuống $\sim 14\%$	D1
2	48% KH đã mất hoặc chưa mua (27.7% Never + 20.5% Lost)	D2
3	Return rate 3.4% tập trung vào hotspot size $\times$ category	D3
4	38% đơn có promo, discount $\sim 12\%$: AOV kênh chênh lệch rõ	D4
5	Stockout 67% snapshots, fill rate vẫn 0.96	D5
6	5 đề xuất hành động cho CEO	Text

X. Phân tích

Phần này áp dụng framework **Descriptive** $\rightarrow$ **Diagnostic** $\rightarrow$ **Predictive** $\rightarrow$ **Prescriptive** cho 3 dashboard đã build chi tiết: D1, D2, D4. Mỗi dashboard được phân tích độc lập, sau đó kết nối lại thành một câu chuyện kinh doanh thống nhất.

X.1. Dashboard D1 Revenue & Profitability Overview

Câu hỏi trung tâm: “10 năm kinh doanh ra sao? Lợi nhuận có bền vững?”

X.1.1. Descriptive: Bức tranh tổng thể 10 năm

Dashboard trình bày 4 KPI tổng quan: **Total Revenue**, **Gross Profit**, **Gross Margin %** và **Revenue CAGR**. Điểm đáng chú ý đầu tiên là Revenue CAGR **âm**, cho thấy tổng doanh thu không tăng trưởng mà có xu hướng *suy giảm kép* qua giai đoạn 9 năm. Đây là tín hiệu cần được đặt ở vị trí trung tâm của mọi phân tích tiếp theo.

Biểu đồ viz_D1_01 (Revenue Trend: Daily + MA30 Overlay) cho thấy doanh thu hàng tháng có **biên độ dao động lớn**, với các đỉnh cao xuất hiện lặp lại theo chu kỳ năm. Đường MA30 (moving average) giúp làm mượt pattern mùa vụ, qua đó lộ ra xu hướng dài hạn: **các đỉnh cao nhất rơi vào giai đoạn giữa (2016–2017), sau đó suy giảm dần đến 2022**. Nói cách khác, doanh nghiệp đã qua “peak” cách đây vài năm và đang ở pha đi xuống của vòng đời kinh doanh.

X.1.2. Diagnostic: Tại sao lợi nhuận mong manh?

Biểu đồ viz_D1_02 (Gross Margin % Trend) tiết lộ một điểm bất thường quan trọng: margin trung bình dao động quanh vùng dương thấp, nhưng **xuất hiện các đợt sụt sâu xuống vùng âm mạnh**. Những đợt âm này không phải ngoại lệ đơn lẻ: chúng rải đều qua các năm với tần suất đáng kể. Điều này đặt ra hai giả thuyết:

- **Giả thuyết 1:** Các đợt khuyến mãi lớn (flash sale, clearance) khiến giá bán thấp hơn COGS trên nhiều đơn hàng, kéo margin cục bộ xuống âm.
- **Giả thuyết 2:** Chi phí hoàn hàng (refund) cao trong một số tháng làm revenue net bị âm cục bộ.

Biểu đồ viz_D1_03 (MoM Growth Calendar Heatmap) củng cố pattern mùa vụ rất rõ: tháng **March mỗi năm đều có growth rất cao** (màu xanh đậm lặp lại qua hầu hết các năm). Ngược lại, các tháng **Jul-Sep đa số có growth âm** (màu đỏ phủ gần như toàn bộ hàng ngang ba tháng này). Đây là pattern “high season: low season” ổn định qua năm, có thể khai thác để lập kế hoạch ngân sách và inventory.

X.1.3. Predictive: Hướng đi của 2023–2024

Từ xu hướng CAGR âm và pattern mùa vụ rõ nét, có thể dự báo:

- Nếu không có can thiệp, tổng doanh thu năm tiếp theo **tiếp tục giảm** theo nhịp suy thoái đã thiết lập.
- **Mùa hè** vẫn sẽ là những tháng cao điểm: nên được ưu tiên về ngân sách và chuẩn bị vận hành.
- Biểu đồ viz_D1_04 (Category Revenue Share by Year) cho thấy **Streetwear chiếm tỷ trọng áp đảo** và ổn định qua các năm, trong khi Casual, GenZ, Outdoor chỉ chiếm phần nhỏ. **Dependency vào một category duy nhất** là rủi ro tập trung cần được nhìn nhận.

X.1.4. Prescriptive: 3 hướng hành động

1. **Kiểm soát margin các đợt khuyến mãi:** Đặt floor price ràng buộc theo COGS để tránh các đợt tụt margin xuống vùng âm. Vấn đề không nằm ở việc có khuyến mãi hay không, mà ở chỗ khuyến mãi đang được áp dụng không có giới hạn dưới về giá.
2. **Khai thác mùa cao điểm March:** Dồn ngân sách marketing và chuẩn bị inventory vào February–March (trước mùa cao điểm) thay vì phân bổ đều cả năm. Đây là “window of opportunity” được dữ liệu lặp lại xác nhận.
3. **Đa dạng hoá category:** Giảm dần sự phụ thuộc vào Streetwear bằng cách đầu tư mở rộng SKU và marketing riêng cho Casual, GenZ, Outdoor. Rủi ro tập trung không chỉ đến từ thị trường mà còn từ chính cấu trúc sản phẩm.

X.2. Dashboard D2: Customer Segmentation & Lifecycle

Câu hỏi trung tâm: “Ai là khách hàng giá trị? Ai đang rời đi?”

X.2.1. Descriptive: Chân dung tập khách hàng

Dashboard trình bày 4 KPI: **Total Customers**, **Active Customers**, **Churn Rate** và **Avg LTV (Monetary)**. Điểm đáng chú ý là tỷ lệ Active chỉ chiếm một phần nhỏ hơn nửa tổng số khách đã đăng ký: phần còn lại thuộc về các segment không còn hoạt động hoặc chưa bao giờ mua. Churn Rate (Lost + At Risk) ở mức **đáng lo ngại cho một doanh nghiệp e-commerce trưởng thành**.

Biểu đồ viz_D2_01 (RFM Segment Distribution) cho thấy phân bố khách hàng rất **mất cân bằng**: nhóm **Never Purchased** chiếm tỷ trọng lớn nhất: tức là một lượng lớn khách đã đăng ký tài khoản **nhưng chưa từng chuyển đổi** thành giao dịch thực. Nhóm **Champions** và **Lost**

có quy mô tương đương nhau: nghĩa là cứ có một khách đăng cấp nhất thì cũng có khoảng một khách đã bỏ đi. Các segment trung gian (Potential Loyalists, About To Sleep, New Customers) chiếm tỷ trọng nhỏ hơn nhiều, cho thấy **phễu chuyển đổi giữa các tầng không mượt**: khách hoặc là ở lại hẳn (Champions/Loyal), hoặc rơi thẳng xuống Lost mà không qua các tầng đệm.

X.2.2. Diagnostic: Tại sao khách rời đi?

Biểu đồ viz_D2_02 (Recency $\times$ Monetary Scatter) cho thấy mối quan hệ rõ ràng giữa thời gian mua gần nhất và giá trị đơn hàng:

- **Champions** (bubble lớn nhất, góc phải trên): Recency rất gần, Monetary cao nhất: segment đăng cấp nhất cần được ưu tiên giữ.
- **Loyal Customers** và **Potential Loyalists**: Recency trung bình, Monetary trung bình: nhóm có tiềm năng nâng cấp lên Champions nếu được kích hoạt đúng.
- **At Risk** và **About To Sleep**: Recency khá xa, Monetary không còn cao: đây là nhóm cần can thiệp *ngay* trước khi chuyển sang Lost.
- **Lost**: Recency rất xa, Monetary thấp: chi phí reactivate cao hơn chi phí acquisition mới, không nên ưu tiên.

Biểu đồ viz_D2_03 (Cohort Retention Heatmap) tiết lộ pattern retention **rất đáng lo**:

- Retention rơi **mạnh ngay từ tháng +1** sau first order: đa số khách không quay lại trong tháng đầu tiên.
- Từ tháng +2 trở đi, retention **ổn định ở mức thấp** và gần như không phục hồi.
- Đây là pattern “**one-and-done customer**” điển hình của e-commerce chưa có chương trình retention bài bản: khách mua một lần rồi biến mất, không có email nurture, không có loyalty program hoạt động.

X.2.3. Predictive: Rủi ro churn tương lai

Từ pattern retention thấp và Churn Rate cao, có thể dự báo:

- Một phần đáng kể khách thuộc **At Risk** sẽ chuyển thành Lost trong 6–12 tháng tới nếu không can thiệp.
- Segment **About To Sleep** là “At Risk tương lai”: cần được đặt trong vòng theo dõi trước khi quá muộn.
- Nhóm **Never Purchased** là pool lớn nhưng *chi phí activate thấp hơn nhiều* so với acquire khách mới: đây là nguồn tăng trưởng đang bị bỏ ngỏ.

X.2.4. Prescriptive: Hành động theo segment

Biểu đồ viz_D2_04 (Acquisition Channel ROI) cho thấy các kênh acquisition khác nhau mang về khách có giá trị LTV khác nhau. Nhìn tổng thể, **organic_search** và **social_media** có bubble lớn nhất (volume cao) với LTV trung bình cạnh tranh: đây là các kênh có khả năng scale. Ngược lại, một số kênh có LTV cao hơn nhưng volume nhỏ (referral, paid_search), cho thấy cơ hội để mở rộng có chọn lọc.

1. **Champions:** VIP program, early access sản phẩm mới, exclusive discount: mục tiêu là *nâng LTV*, không phải tăng tần suất.
2. **At Risk:** Win-back campaign với discount + free shipping: mục tiêu là *ngăn rơi xuống Lost*, chấp nhận biên lợi nhuận thấp hơn trong giao dịch reactivate.
3. **Never Purchased:** Welcome sequence 3-email (Day 1 chào mừng, Day 3 gợi ý sản phẩm, Day 7 first-purchase discount): mục tiêu là *activate first transaction*, kích hoạt pool khách đang nằm yên.
4. **Ưu tiên kênh acquisition:** Đẩy mạnh các kênh đã chứng minh có volume + LTV tốt (organic_search, social_media). Giảm dần đầu tư vào kênh có bubble nhỏ và LTV không vượt trội.

X.3. Dashboard D4: Marketing & Channel Effectiveness

Câu hỏi trung tâm: “Kênh nào ROI tốt nhất? Promo có hiệu quả không? Traffic chuyển hoá ra sao?”

X.3.1. Descriptive: Cấu trúc hoạt động marketing

Dashboard trình bày 4 KPI quan trọng: **Total Orders**, **Promo Orders %**, **Avg Discount %** và **AOV**. Điểm đáng chú ý là gần *hai phần năm* tổng số đơn hàng đều có ít nhất một chương trình khuyến mãi: một tỷ lệ khá cao, cho thấy doanh nghiệp đang phụ thuộc đáng kể vào công cụ giá để thúc đẩy giao dịch. Mức chiết khấu trung bình khi có promo cũng không nhỏ, làm cho câu hỏi “promo có thực sự mang lại giá trị tương xứng không” trở thành trọng tâm của toàn dashboard.

Biểu đồ viz_D4_01 (Revenue Share by Order Source: 100% Stacked Area) cho thấy cơ cấu doanh thu theo kênh **tương đối ổn định suốt 10 năm**. **organic_search** (xanh dương) chiếm tỷ trọng lớn nhất và duy trì vai trò xương sống, tiếp theo là **paid_search** và **social_media**. Các kênh còn lại (direct, email_campaign, referral) đóng góp phần nhỏ hơn nhưng ổn định. Điều này cho thấy thương hiệu đã xây dựng được nền tảng nhận diện tự nhiên vững chắc, **không phụ thuộc quá mức vào quảng cáo trả phí đơn lẻ**: một cấu trúc kênh đáng mong muốn về mặt rủi ro.

X.3.2. Diagnostic: Hiệu quả khuyến mãi theo danh mục

Biểu đồ viz_D4_02 (Promo vs No-Promo AOV by Category) tiết lộ một kết quả **đáng chú ý** và **phản trực giác**: AOV của đơn hàng có promo **đều thấp hơn** so với đơn không promo ở *tất*

cả 4 danh mục. Đây không phải hiện tượng riêng của một category mà là pattern lan toả toàn hệ thống.

Danh mục **Streetwear có mức chênh lệch tuyệt đối lớn nhất**: do bản thân AOV của Streetwear đã cao hơn nhiều so với các category khác, nên mỗi đơn có promo mất đi một khoản giá trị đáng kể. Ba category còn lại (Casual, GenZ, Outdoor) có AOV thấp hơn nhưng *tỷ lệ giảm tương đối cũng không nhỏ*.

Giải thích 1 (tiêu cực): Khuyến mãi đang “ăn mòn” giá trị đơn hàng của những khách vốn đã có ý định mua: họ chỉ chờ có promo để mua rẻ hơn, không tăng volume mua.

Giải thích 2 (tích cực): Promo thu hút thêm nhóm khách mới có giỏ hàng nhỏ hơn (price-sensitive segment): nhóm này kéo AOV trung bình xuống nhưng tăng tổng số đơn, nên tổng doanh thu vẫn tăng.

Để phân biệt, cần phân tích thêm: **promo có tăng số đơn/khách không?** và **những khách mua có promo có quay lại mua tiếp không?** Đây là hướng mở rộng cần thực hiện ở bước phân tích tiếp theo.

X.3.3. Predictive: Tín hiệu từ lưu lượng truy cập

Biểu đồ viz_D4_03 (Web Traffic: Sessions + Bounce Rate, dual-axis) cho thấy hai pattern song song:

- **Sessions có xu hướng tăng theo thời gian**, đặc biệt mạnh trong giai đoạn gần cuối (khoảng 2021). Điều này trùng với giai đoạn COVID-19 và xu hướng mua sắm online bùng nổ: có thể là hiệu ứng thị trường chứ không hoàn toàn đến từ nỗ lực marketing.
- **Bounce rate dao động ở vùng rất thấp** và tương đối đồng đều giữa các kênh: cho thấy chất lượng landing page **không phải là nút thắt**.
- Tín hiệu tích cực: traffic tăng nhưng engagement không giảm: khi ngân sách tăng, conversion không bị kéo xuống. **Đây là điều kiện lý tưởng để scale ngân sách marketing** mà không lo overspend.

X.3.4. Prescriptive: Xếp hạng kênh và hành động

Biểu đồ viz_D4_04 (Channel AOV Ranked) với reference line tại AOV trung bình chia 6 kênh thành 2 nhóm rõ rệt:

- **Trên trung bình (xanh lá: above-average)**: direct, paid_search, email_campaign: 3 kênh có AOV vượt ngưỡng trung bình.
- **Dưới trung bình (đỏ: below-average)**: referral, organic_search, social_media: 3 kênh có AOV thấp hơn ngưỡng trung bình.

Khi đối chiếu D4_01 và D4_04, pattern rõ ràng hiện ra: **organic_search** và **social_media** chiếm tỷ trọng **doanh thu lớn nhất** nhưng **AOV thấp nhất**. Nghĩa là: 2 kênh này mang lại nhiều đơn nhưng mỗi đơn có giá trị nhỏ. Ngược lại, **email_campaign** có AOV cao nhưng volume chưa lớn. Đây chính là **đòn bẩy tái phân bổ ngân sách** mà doanh nghiệp đang chưa khai thác.

XI. Tổng kết

Bài hướng dẫn đã trình bày từ **15 file CSV** thô đến **5 dashboard Tableau hoàn chỉnh** kèm theo khung phân tích kinh doanh có chiều sâu. Quy trình có thể gói gọn trong bốn bước then chốt:

1. **Hiểu đề & rubric trước khi đi vào data:** Phần 2 chiếm 60/100 điểm, trong đó “Chiều sâu phân tích” là tiêu chí nặng nhất. Mỗi biểu đồ nên đi đủ 4 cấp **Descriptive** → **Diagnostic** → **Predictive** → **Prescriptive** để tối đa hoá điểm.
2. **Thiết kế data model trước khi build:** 15 file được phân thành 3 lớp (Master, Transaction, Analytical + Operational). Hiểu rõ grain và cardinality giúp tránh những lỗi tính toán bị “inflate” khi join nhiều bảng.
3. **Python tiền xử lý:** 14 bước trong `prepare_tableau_data.py` sinh ra bộ 13 file đã enrich (RFM, cohort, moving averages, margin, delivery buckets). Tableau chỉ còn lo phần trực quan hoá.
4. **Mỗi dashboard kể một câu chuyện có đề xuất hành động:** D1 Revenue, D2 Customer, D3 Product, D4 Marketing, D5 Operations — ghép lại thành Story 6 Story Points cho CEO với các con số cụ thể.

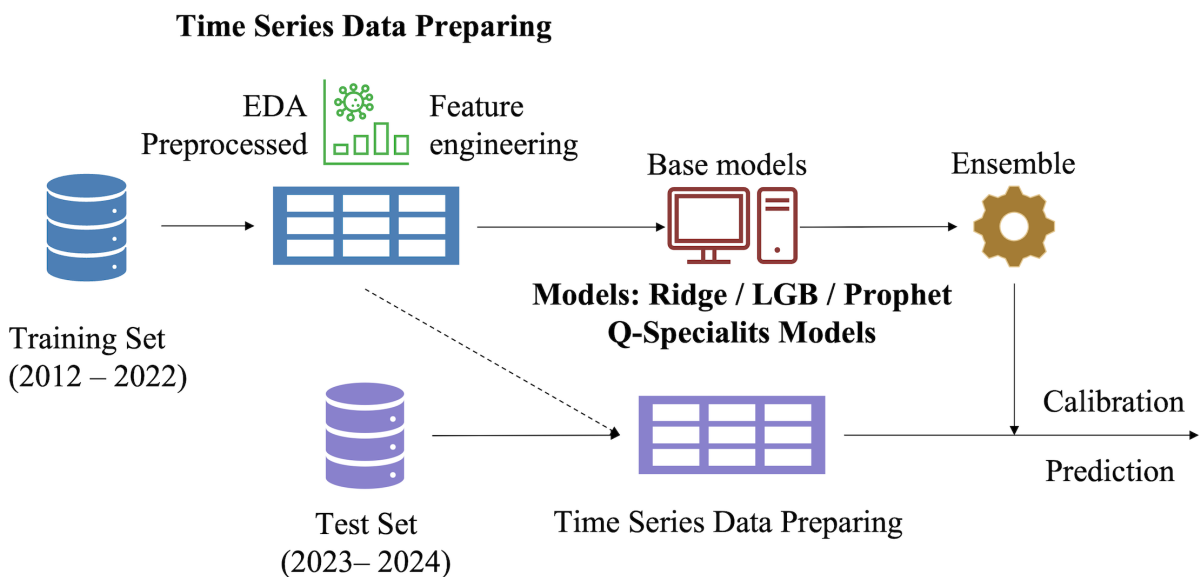
Năm phát hiện kinh doanh nổi bật từ quá trình phân tích:

- Revenue CAGR **âm** (khoảng -3.8% /năm), margin co từ $\sim 22\%$ xuống $\sim 14\%$: doanh nghiệp đã qua đỉnh.
- Gần **48% khách hàng** không hoạt động (Never Purchased + Lost); retention rơi mạnh ngay từ tháng +1: thiếu chương trình giữ chân.
- Promo áp dụng cho $\sim 38\%$ đơn nhưng làm **AOV giảm đều qua mọi category**: cần kiểm soát floor price.
- **Streetwear** chi phối doanh thu nhưng rủi ro tập trung cao: cần đa dạng hoá category.
- `organic_search` + `social_media` có volume lớn nhưng AOV thấp; `email_campaign` có AOV cao nhưng volume nhỏ: **cơ hội tái phân bổ ngân sách** chưa được khai thác.

C. XÂY DỰNG MÔ HÌNH DỰ ĐOÁN

XII. Giới thiệu

Trong cuộc thi VinDatathon 2026 Round 1, tác vụ tập trung với một bộ dữ liệu thương mại điện tử 11 năm và ba nhiệm vụ kế tiếp nhau. Phần 1 là trắc nghiệm, Phần 2 là Visualization & Analysis, và Phần 3 — là phân tích và huấn luyện mô hình **Sales Forecasting**. Yêu cầu của bài toán: từ dữ liệu doanh thu hàng ngày đến hết 2022, hãy dự báo **Revenue** và **COGS** cho 548 ngày kế tiếp (01/01/2023 đến 01/07/2024).



Hình 10: Tổng quan pipeline forecasting.

💡 Quan hệ giữa Phần 2 và Phần 3

Phần 2 yêu cầu *hiểu dữ liệu đã xảy ra trong quá khứ*: trend, segmentation, cohort retention, hiệu quả kênh. Các biểu đồ Revenue, Gross Margin, Category Contribution đã được trình bày chi tiết trong tài liệu “**Datathon 2026: Dashboard Tableau từ A đến Z**”. Phần 3, yêu cầu *dự đoán điều sẽ xảy ra*: từ các pattern đã quan sát, xây mô hình extrapolate sang tương lai. Tài liệu này sẽ sử dụng kế thừa các insight từ Phần 2 nhưng tập trung vào khía cạnh *xây dựng mô hình* (modelling).

Phương pháp tiếp cận theo hướng: Phân tích dữ liệu, kết hợp kỹ thuật Feature Engineering dựa trên calendar, ba mô hình cơ bản học song song (Ridge, LightGBM, Prophet), bốn mô hình chuyên biệt theo quý, và một tầng calibration tune.

XIII. Phát biểu bài toán

XIII.1. Input và Output

Dữ liệu chính từ file `sales.csv` với ba cột `Date`, `Revenue`, `COGS`, phủ liên tục từ ngày 01/01/2012 đến 31/12/2022. Ngoài ra còn mười hai bảng phụ (promotions, inventory, returns, customers, web_traffic, ...) đã được khai thác chi tiết trong Phần 2. Lưu ý rằng *tất cả các bảng phụ trợ đều dừng ở năm 2022*, nên chúng chỉ có thể được dùng để rút ra *pattern có thể lặp lại* (ví dụ: lịch khuyến mãi) chứ không thể dùng trực tiếp như feature cho giai đoạn test.

Mục tiêu huấn luyện mô hình dự đoán tương lai, với output một file `submission.csv` gồm 548 dòng, mỗi dòng mang một `Date` trong khoảng 2023-01-01 đến 2024-07-01 và hai giá trị dự báo `Revenue`, `COGS`.

XIII.2. Hàm mục tiêu

Hàm chấm điểm của cuộc thi là MAE (Mean Absolute Error), RMSE và R2 và được tổng hợp trên cả hai biến `Revenue` và `COGS`.

Một số đặc trưng nhận thấy rằng *một số đặc điểm cùng xuất hiện*. Thứ nhất, **horizon dài 18 tháng** vượt xa khả năng extrapolate của các mô hình tuyến tính đơn giản hay mô hình neural time-series có dự đoán ngắn hạn. Thứ hai, **regime drift** rất mạnh: phân phối giai đoạn sau 2020 khác xa phân phối 2012–2018 cả về mean lẫn variance. Thứ ba, **không có ground truth feedback**: ta không biết sai ở ngày nào nên phải suy luận gián tiếp. Thứ tư, **calibration bias** tồn tại ở hầu hết mô hình học sâu trên log-space: chúng có xu hướng dự đoán thấp hơn thực tế vì hàm loss pull prediction về mean.

Chú ý forecasting horizon dài

Khi horizon quá dài, **feature lag** (ví dụ y_{t-1} , y_{t-7}) không còn truy cập được: ta không biết `Revenue` hôm qua của 548 ngày tương lai. Phương án duy nhất ổn định là dùng *feature calendar thuần* — mọi đặc trưng đều tính được từ ngày tháng, không phụ thuộc vào lịch sử.

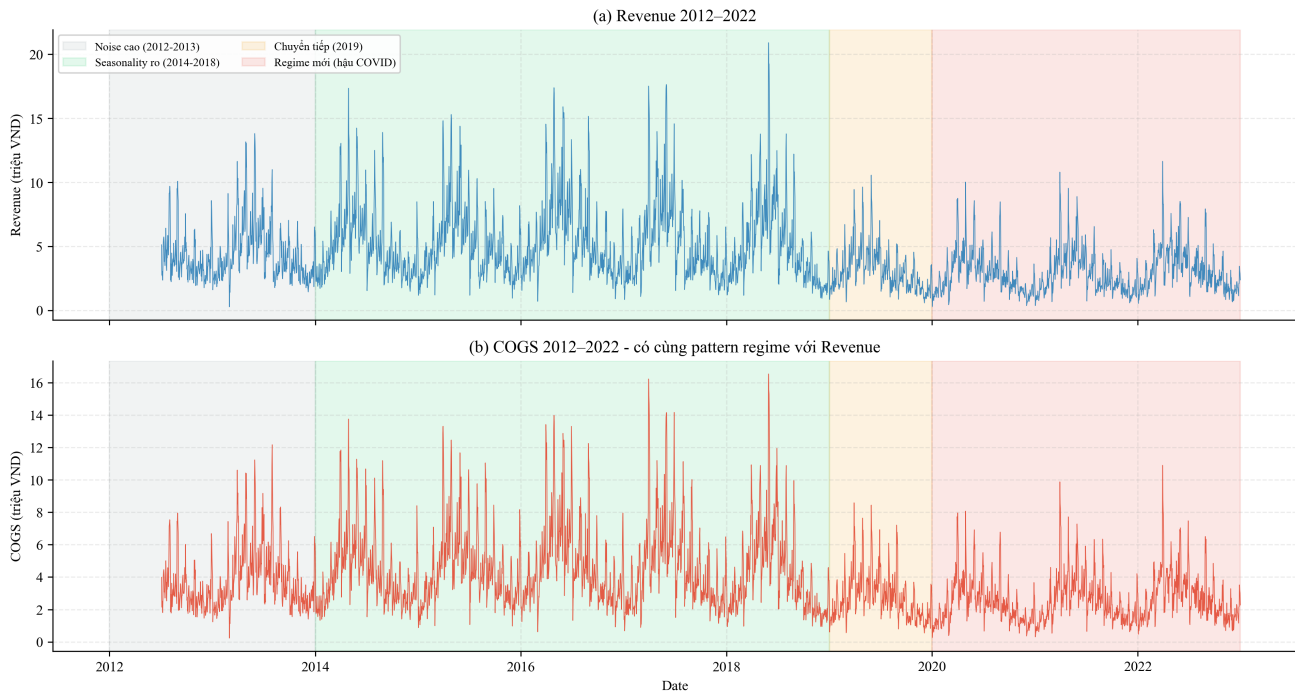
XIV. Phân tích dữ liệu

Trước khi quyết định mô hình, chúng ta cần hình dung rõ dữ liệu đang nói lên điều gì: bắt đầu từ bức tranh tổng thể (regime và mùa vụ dài hạn), các pattern lịch biểu chi tiết hơn (Tết, day-of-week, chu kỳ năm), và tính phân phối cùng mối liên hệ giữa hai biến mục tiêu.

💡 Liên hệ với Phần 2 (Visualization & Analysis)

Phần 2 cũng khám phá dữ liệu nhưng với mục đích khác: “kể chuyện kinh doanh” cho CEO. Ở Mục “Phân tích” cuối Phần 2 có ba kết luận đã được viết ra: (a) Revenue CAGR âm và doanh thu đã qua peak giai đoạn 2016–2017, (b) tháng March mỗi năm đều có growth cao nhất và Jul–Sep đa số growth âm, (c) margin xuất hiện các đợt sụt sâu xuống vùng âm. Ba kết luận này chính là *ba giả định xuất phát* cho Phần 3: chúng củng cố nhu cầu về sample weighting, về Fourier seasonality, và về mô hình chuyên biệt cho quý biến động. Trong Mục này chúng ta định lượng lại ba giả định đó qua biểu đồ, để suy ra các quyết định kỹ thuật cụ thể.

XIV.1. Bức tranh tổng thể: Biểu đồ qua các năm



Hình 11: Timeline Revenue và COGS 2012–2022.

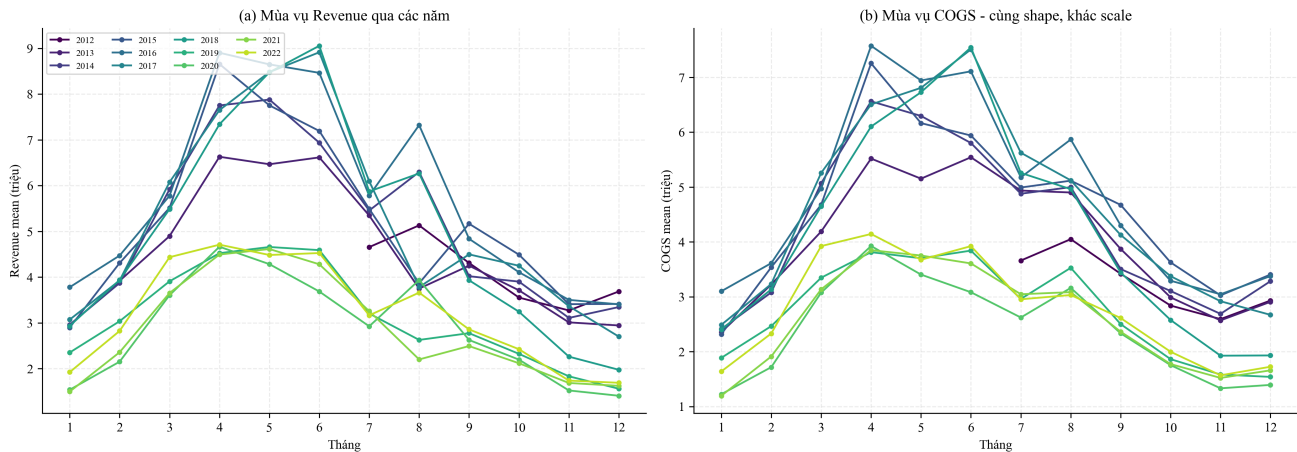
Câu hỏi đầu tiên cần trả lời là: *dữ liệu có đồng nhất theo thời gian hay không?* Đây là câu hỏi then chốt vì nó quyết định chúng ta có thể coi toàn bộ 11 năm như một khối duy nhất, hay phải tách nhỏ và xử lý riêng. Để trả lời, chúng ta nhìn vào Hình 11.

Chuỗi không đồng nhất. Bốn vùng được đánh dấu có những *tính chất* rất khác nhau chứ không chỉ khác về con số. Giai đoạn 2012–2013 có noise trùm lên tín hiệu, mô hình học từ đây sẽ dễ bị phân tán. Giai đoạn 2014–2018 là *vùng peak* với biên độ doanh thu lớn nhất (~ 0 đến 20 triệu/ngày) và seasonality định hình rõ ràng — đây là vùng “sạch” lý tưởng để mô hình học pattern. Năm 2019 là bước nhảy giảm đột ngột. Giai đoạn 2020–2022 chuyển sang một regime mới với *mức doanh thu thấp hơn* $\sim 40\%$ so với 2014–2018 và biên độ thu hẹp (~ 0 đến 10 triệu/ngày). Điều này khớp với kết luận từ Phần 2 về CAGR âm.

Từ quan sát này, hai ý tưởng thiết kế phù hợp. Thứ nhất, khi huấn luyện nên *ưu tiên vùng có seasonality rõ nhất* (2014–2018) bằng trọng số mẫu, để mô hình tập trung học *hình dạng* (shape) của các pattern. Thứ hai, vì vùng học có level khác hẳn vùng test, prediction sẽ bị lệch về phía của vùng học — cần một cơ chế *điều chỉnh mức* (calibration) tách rời để khớp lại với mức thực tế của test. Hai ý tưởng này sẽ được thực thi trong Mục XVII. và Mục XVIII..

XIV.2. Cấu trúc thời gian: Tính mùa vụ có bền vững không?

Giả sử ta đã chấp nhận rằng level thay đổi theo chế độ; câu hỏi tiếp theo là: *shape mùa vụ có thay đổi theo không, hay vẫn giữ ổn định?* Nếu shape ổn định, chiến lược “học shape ở vùng sạch rồi rescale” là khả thi; nếu shape cũng biến đổi, chiến lược đó sụp đổ. Hình 12 giúp ta trả lời.



Hình 12: Doanh thu trung bình tháng qua các năm. Hình dạng đường cong lặp lại rất ổn định giữa các năm: sụt Q1 vì Tết, Q2 và Q4 là đỉnh năm. Sự khác biệt giữa các năm chủ yếu nằm ở *level* (mức trung bình), còn *shape* gần như bất biến.

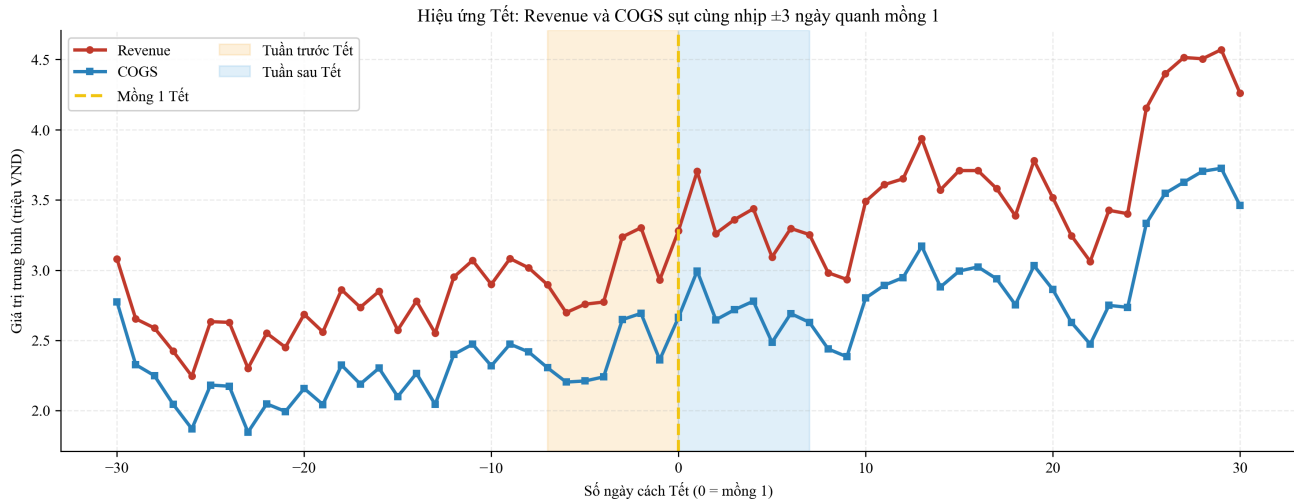
Câu trả lời là *có ổn định*. Tất cả các đường cong đều có cùng một dáng: đi xuống ở tháng 1–2, bật lên ở tháng 3–4, plateau giữa năm, và bung cao ở Q4. Đây là một trong những quan sát quan trọng nhất cho toàn bộ pipeline vì nó xác nhận giả thuyết “**mùa vụ bền vững, level thay đổi**”. Chúng ta có thể dùng Fourier features để mô hình hoá seasonality (Mục XV.).

XIV.3. Các pattern lịch biểu chi tiết

Sau khi đã nắm bức tranh tổng quát, chúng ta phân tích các pattern nhỏ hơn. Có ba pattern xem xét lần lượt: hiệu ứng Tết, pattern theo thứ/ngày-trong-tháng, và chu kỳ năm chẵn/lẻ. Mỗi pattern sẽ dẫn đến một nhóm feature cụ thể trong hàm `build_features()`.

(a) Hiệu ứng Tết Âm lịch

Tết là sự kiện lớn nhất trong năm về mặt hành vi tiêu dùng. Câu hỏi thực tế cần trả lời là: *ảnh hưởng của Tết kéo dài bao lâu, và có đối xứng trước/sau không?*

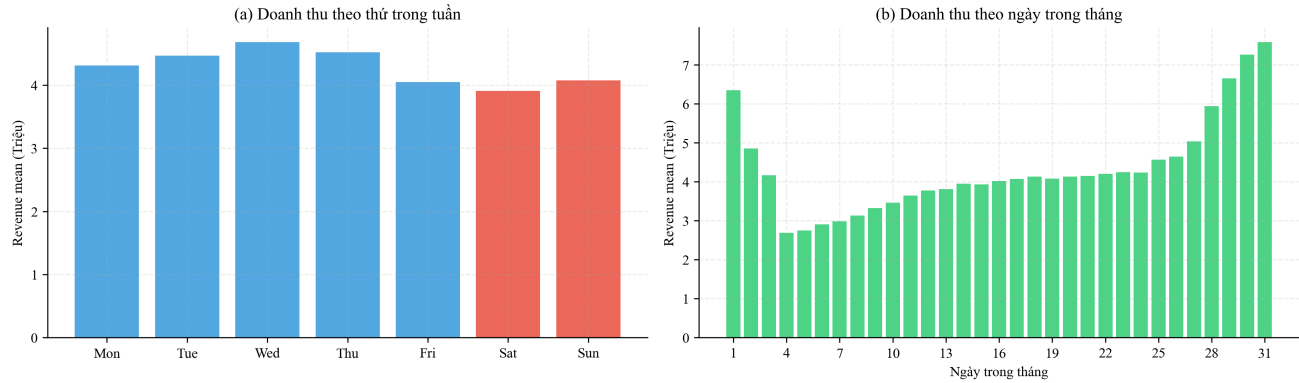


Hình 13: Hiệu ứng Tết Âm lịch, tổng hợp từ mười năm 2013–2022. Trục hoành là số ngày cách mùng 1 Tết, trục tung là doanh thu trung bình. Sụt sâu nhất nằm ở khoảng ± 3 ngày quanh mùng 1, hồi phục sau khoảng một tuần.

Hình 13 cho thấy pattern Tết. Trước Tết ~ 30 ngày, doanh thu thấp ($\sim 2.5\text{--}3.0$ triệu); càng gần mùng 1, doanh thu *tăng dần* chứ không sụt — đỉnh nhỏ ngay quanh mùng 1 ($\sim 3.5\text{--}3.7$ triệu). Sau Tết khoảng 20 ngày, doanh thu mới thực sự bứt lên ($\sim 4\text{--}4.5$ triệu). Revenue và COGS đi cùng nhịp. Đặc điểm quan trọng là Tết rơi vào các ngày dương lịch khác nhau mỗi năm (từ 21/01 đến 19/02), nên feature cố định theo tháng không bắt được. Giải pháp là dùng feature *khoảng cách tới Tết gần nhất* — biến `tet_days_diff` mã hoá chính xác đặc điểm này và áp dụng được cho cả 2023 (Tết 22/01) lẫn 2024 (Tết 10/02).

(b) Day-of-week và Day-of-month

Tiếp theo, chúng ta hỏi: *doanh thu có phụ thuộc vào thứ trong tuần hay vị trí trong tháng không?* Hai pattern này nhỏ hơn nhưng vẫn có ý nghĩa cho mô hình gốc cây.

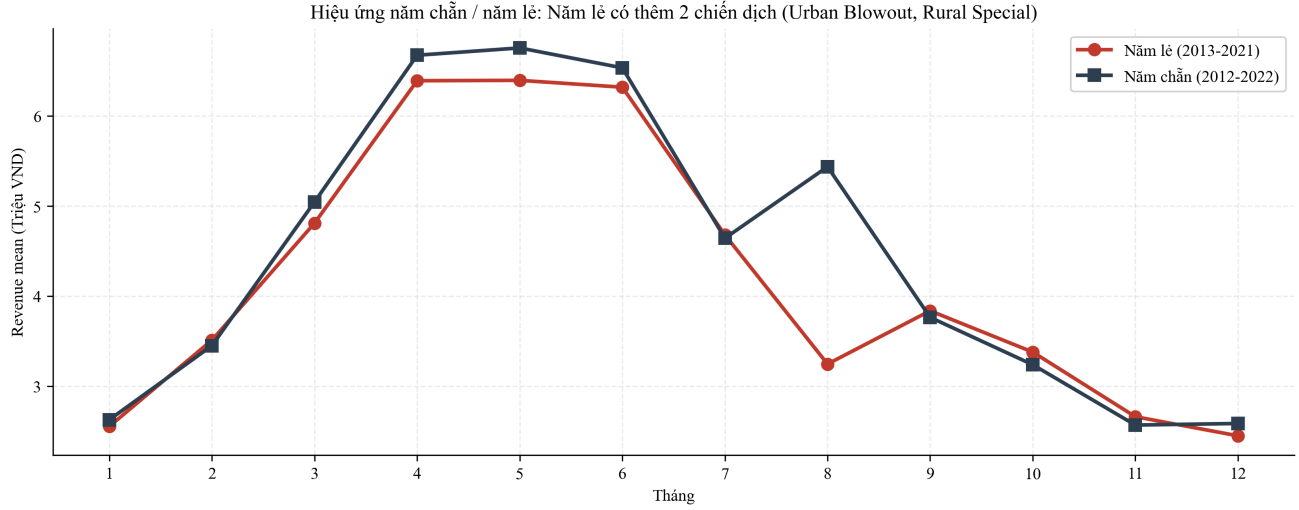


Hình 14: (a) Doanh thu theo thứ trong tuần — khá đồng đều, không có hiệu ứng weekend mạnh điển hình của B2C. (b) Doanh thu theo ngày trong tháng — có spike rõ ở đầu tháng và giữa tháng, gợi ý pattern lương hoặc thói quen mua sắm đầu kỳ.

Hai đặc điểm cần chú ý. Thứ nhất, doanh nghiệp này có hiệu ứng *weekend ngược* — weekday (Mon-Fri ~ 4.0–4.7 triệu) cao hơn weekend (Sat-Sun ~ 3.9–4.1 triệu), khác với retail B2C thông thường. Thứ hai, pattern *day-of-month* rất mạnh (bảng bên phải): ngày 28–31 cuối tháng cao gấp đôi ngày giữa tháng (đỉnh ~ 7.5 triệu vào ngày 31), ngày 1–3 cũng cao (~ 5–6 triệu). Vùng giữa tháng (ngày 4–20) là plateau thấp ~ 3–4 triệu. Quan sát “cuối tháng quan trọng hơn đầu tháng” dẫn đến nhóm feature `is_last1`, `is_last2`, `is_last3` và `days_to_eom`, song song với nhóm `is_first1..3` cho đầu tháng.

(c) Chu kỳ năm chẵn / năm lẻ

Kiểm tra bảng `promotions.csv` cho thấy có hai chiến dịch chỉ xuất hiện ở năm lẻ: Urban Blowout (cuối tháng 7) và Rural Special (cuối tháng 1). Điều này dẫn đến câu hỏi: *hai nhóm năm chẵn / lẻ có thực sự khác nhau trong dữ liệu không?*

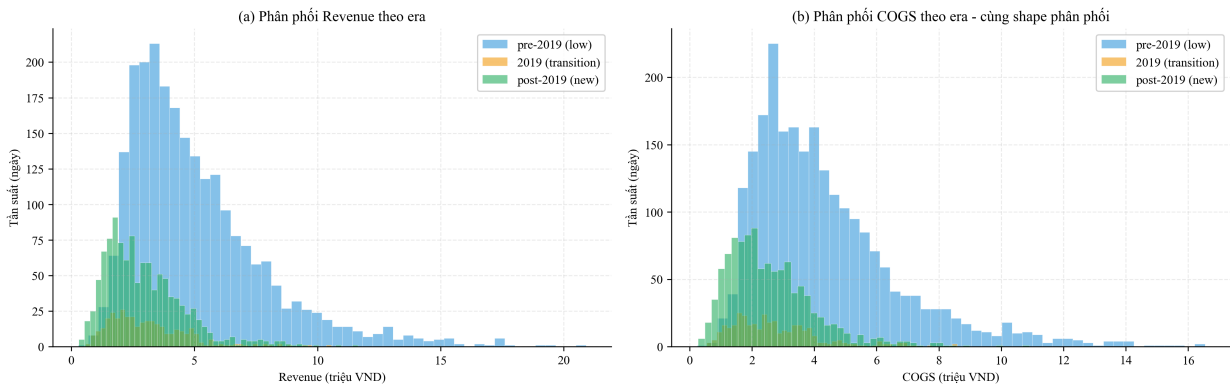


Hình 15: Doanh thu trung bình tháng, tách theo năm chẵn và năm lẻ. Năm lẻ có bump nhỏ nhưng nhất quán ở tháng 1 (Rural Special) và tháng 7–8 (Urban Blowout).

Khác biệt rõ nhất nằm ở *tháng 8*: năm chẵn có doanh thu cao đột biến (~ 5.4 triệu) trong khi năm lẻ chỉ (~ 3.3 triệu) — chênh gần $1.6\times$. Các tháng còn lại gần như trùng nhau. Pattern này nhất quán qua các năm, không phải nhiễu ngẫu nhiên. Vì test gồm 2023 (năm lẻ) và 2024 (năm chẵn), một feature binary `is_odd_year` cho phép mô hình chuyển đổi giữa hai pattern trong từng năm của giai đoạn test.

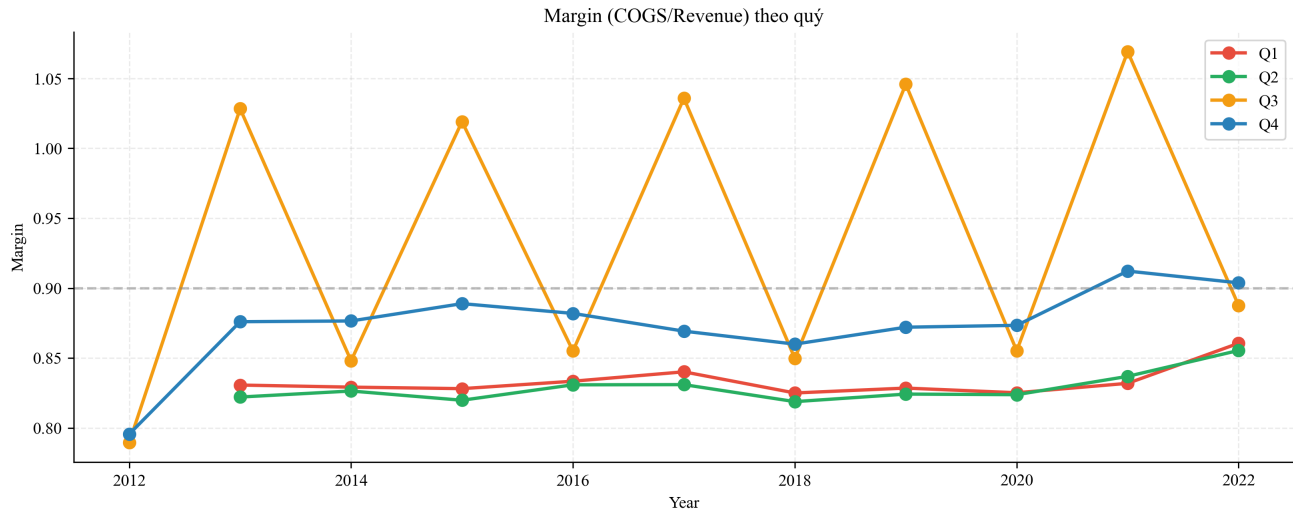
XIV.4. Phân phối và mối liên hệ giữa hai biến mục tiêu

Hai phân tích cuối cùng liên quan đến cấu trúc phân phối của biến mục tiêu. Chúng ta cần biết: *phân phối Revenue có tách biệt giữa các era không*, và *mối liên hệ Revenue–COGS có đặc điểm gì đáng chú ý cho việc dự báo*?



Hình 16: Histogram Revenue chia theo ba era. Ba phân phối gần như không chồng lấp, củng cố nhu cầu về một cơ chế điều chỉnh mức giữa giai đoạn học và giai đoạn dự báo.

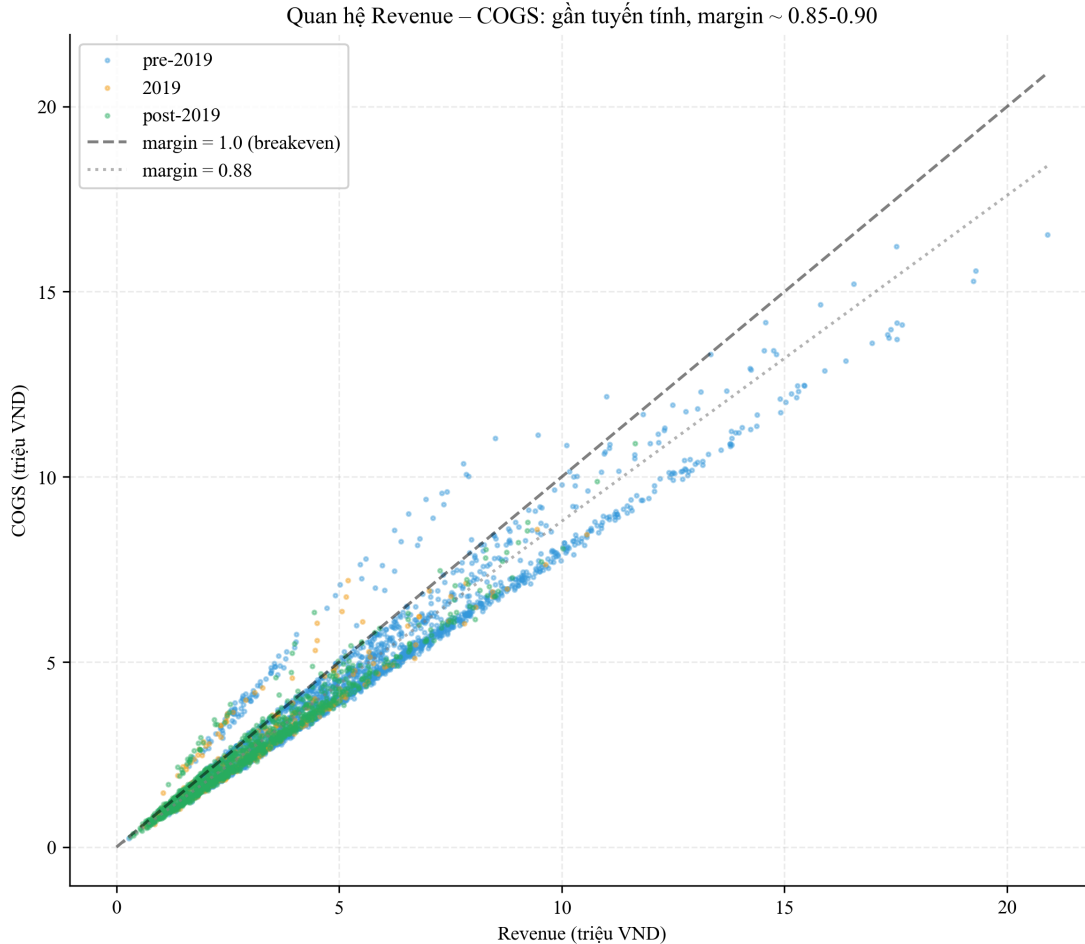
Hình 16 cho thấy ba phân phối tách biệt rõ ràng. Hệ quả định lượng là nếu trainer để mọi mẫu có cùng trọng số, mô hình sẽ thoả hiệp ở một mức trung bình không thuộc về phân phối nào cụ thể — prediction trên 2023 (thuộc phân phối xanh lá) sẽ bị kéo xuống thấp. Đây chính là nguồn gốc của “calibration bias” đã đề cập ở Mục XIII., và là lý do pipeline cần một tầng calibration tách rời.



Hình 17: Margin (COGS / Revenue) theo quý qua từng năm. Q1, Q2, Q4 nằm gọn trong khoảng 0.80–0.92. Riêng Q3 dao động rất mạnh — trong lịch sử từng chạm đến giá trị ngoài 1.0.

Hình 17 cho thấy một pattern rất đặc biệt: Q1, Q2, Q4 có margin ổn định trong khoảng 0.82–0.91, nhưng **Q3 dao động zig-zag rõ rệt theo chu kỳ năm chẵn/lẻ**. Năm lẻ (2013, 2015, 2017, 2019, 2021) margin Q3 vọt lên ~ 1.02–1.07 (COGS vượt Revenue — bán lỗ). Năm chẵn (2014, 2016, 2018, 2020) margin Q3 trở lại bình thường ~ 0.85. Pattern này gần như chắc chắn liên quan tới chiến dịch **urban_blowout** (chỉ chạy năm lẻ, vào cuối tháng 7) — một chiến dịch có vẻ hy sinh margin để đẩy volume. Một mô hình chung sẽ bị “trung bình hoá” và không bắt được pattern Q3-năm-lẻ này. Quan sát này trực tiếp dẫn đến hai ý tưởng: (a) feature **is_odd_year**, và (b) **mô hình chuyên biệt theo quý** (Q-specialist) sẽ xây ở Mục XVII. — mỗi specialist được phép “quan tâm hơn” đến một quý, đặc biệt Spec_Q3 cho hiện tượng phức tạp này.

XIV.5. Scatter Revenue–COGS: cơ sở cho hướng dự báo qua tỉ số



Hình 18: Scatter plot Revenue vs COGS qua ~ 4000 ngày, tách màu theo era. Đường nét đứt là margin = 1.0 (breakeven), đường chấm chấm là margin = 0.88. Phần lớn điểm nằm bên dưới margin = 1.0 với độ dốc gần 0.88.

Hình 18 cho thấy hai điều quan trọng cho việc xây dựng mô hình. Thứ nhất, **COGS gần như tuyến tính theo Revenue** với hệ số góc $\sim 0.85-0.90$. Điều này có nghĩa: nếu ta đã có một mô hình dự báo Revenue tốt, ta có thể lấy COGS bằng $0.88 \times \hat{R}$ (margin trung bình) làm baseline, không cần model riêng. Thứ hai, các **outlier nằm phía trên đường margin = 1.0** (COGS > Revenue) tập trung ở vùng pre-2019 mức thấp — chính là Q3 năm lẻ đã thấy ở Hình 17. Pre-2019 cũng có đuôi Revenue cao hơn nhiều (lên tới ~ 20 triệu) trong khi post-2019 chỉ tới ~ 10 triệu, củng cố lại quan sát về regime drift.

Khi thiết kế: pipeline này chọn dự báo COGS *độc lập* (M2 train trên $\log(\text{COGS})$) thay vì dự báo qua tỉ số. Lý do là Q3 năm lẻ có margin > 1.0 vi phạm giả định “margin ổn định”, nên train trên ratio sẽ truyền lỗi của outlier sang prediction. Tuy nhiên, hướng *cogs_ratio* vẫn được Mục XX.4. đề cập như một hướng mở rộng tiềm năng nếu kết hợp khôn ngoan với Q-specialist.

XIV.6. Tóm tắt: từ EDA đến các quyết định mô hình

Sáu phát hiện ở trên có thể được tổng kết trong bảng sau, để thấy được sự kết nối giữa quan sát dữ liệu và quyết định thiết kế:

Quan sát	Đặc trưng được tạo	Cơ chế bù đắp
Ba regime riêng biệt	–	Sample weight + calibration
Mùa vụ ổn định qua các năm	Fourier annual $k = 1..5$	–
Hiệu ứng Tết & đỉnh sau Tết	<code>tet_days_diff</code> , <code>tet_in_7</code> , ...	–
EOM spike (ngày 28–31 cao nhất)	<code>is_last1..3</code> , <code>days_to_eom</code>	–
SOM spike (ngày 1–3)	<code>is_first1..3</code> , <code>days_from_som</code>	–
Năm chẵn cao đột biến tháng 8	<code>is_odd_year</code>	–
Q3 zig-zag chẵn/lẻ (urban_blowout)	–	Q-specialist + <code>is_odd_year</code>

XV. Feature Engineering

Như đã phân tích ở Mục XIII., mọi feature cần tính được từ `Date` mà không phụ thuộc dữ liệu lịch sử. Điều này loại trừ các feature lag truyền thống ($y_{t-1}, y_{t-7}, y_{t-365}$) nhưng cho phép chúng ta tính sẵn toàn bộ ma trận đặc trưng cho cả 2023 và 2024 ngay từ đầu.

Calendar-only featurization

Với mọi ngày t ta trích ra ba nhóm thông tin. Trước hết là **vị trí trong lịch** (năm, tháng, ngày, thứ, ngày trong năm, quý). Tiếp theo là **quan hệ với các mốc lễ**: khoảng cách tới Tết, có phải Black Friday, có trùng ngày lễ cố định nào không. Cuối cùng là **trạng thái promo**: có đang trong cửa sổ của 4 chiến dịch thường niên và 2 chiến dịch năm lẻ hay không, đã diễn ra được bao nhiêu ngày, còn bao nhiêu ngày đến kết thúc.

Mã nguồn tạo ra các đặc trưng

```

1 import numpy as np, pandas as pd
2
3 # --- Lịch khuyến mãi thường niên (trích tu bảng promotions) ---
4 PROMO_SCHEDULE = [
5     ('spring_sale', 3, 18, 30, 12, True),
6     ('mid_year', 6, 23, 29, 18, True),
7     ('fall_launch', 8, 30, 32, 10, True),
8     ('year_end', 11, 18, 45, 20, True),
9     ('urban_blowout', 7, 30, 33, None, 'odd'), # chi nam le
10    ('rural_special', 1, 30, 30, 15, 'odd'),
11 ]
12
13 # --- Ngày lễ cố định VN ---
14 VN_FIXED_HOLIDAYS = [
15     (1,1,'new_year'), (3,8,'womens_day'), (4,30,'reunification'),
16     (5,1,'labor_day'), (9,2,'national_day'), (10,20,'vn_womens_day'),
17     (11,11,'dd_1111'), (12,12,'dd_1212'),
18     (12,24,'christmas_eve'), (12,25,'christmas'),
19 ]
20
21 # --- Tet lưu cập nhật 2013-2024 ---
22 TET_DATES = TET # dung lai tu cell EDA
23
24 def build_features(dates: pd.DatetimeIndex) -> pd.DataFrame:
25     df = pd.DataFrame({'Date': dates})
26     d = df['Date']
27
28     # (a) Calendar basics
29     df['year'] = d.dt.year
30     df['month'] = d.dt.month
31     df['day'] = d.dt.day
32     df['dow'] = d.dt.dayofweek
33     df['doy'] = d.dt.dayofyear
34     df['quarter'] = d.dt.quarter
35     df['is_weekend'] = (df['dow'] >= 5).astype(int)

```

```

36 df['days_to_eom'] = d.dt.days_in_month - df['day']
37 df['days_from_som'] = df['day'] - 1
38 df['dim'] = d.dt.days_in_month
39
40 # (b) Edge-of-month indicators
41 for k in [1,2,3]:
42     df[f'is_last{k}'] = (df['days_to_eom'] <= k-1).astype(int)
43     df[f'is_first{k}'] = (df['days_from_som'] <= k-1).astype(int)
44
45 # (c) Trend anchor + regime
46 df['t_days'] = (d - pd.Timestamp('2020-01-01')).dt.days
47 df['t_years'] = df['t_days']/365.25
48 df['regime_pre2019'] = (df['year']<=2018).astype(int)
49 df['regime_2019'] = (df['year']==2019).astype(int)
50 df['regime_post2019'] = (df['year']>=2020).astype(int)
51
52 # (d) Fourier (annual k=1..5, weekly, monthly)
53 TAU = 2*np.pi
54 for k in (1,2,3,4,5):
55     df[f'sin_y{k}'] = np.sin(TAU*k*df['doy']/365.25)
56     df[f'cos_y{k}'] = np.cos(TAU*k*df['doy']/365.25)
57 for k in (1,2):
58     df[f'sin_w{k}'] = np.sin(TAU*k*df['dow']/7.0)
59     df[f'cos_w{k}'] = np.cos(TAU*k*df['dow']/7.0)
60 for k in (1,2):
61     df[f'sin_m{k}'] = np.sin(TAU*k*(df['day']-1)/df['dim'])
62     df[f'cos_m{k}'] = np.cos(TAU*k*(df['day']-1)/df['dim'])
63
64 # (e) VN holidays
65 for (m, dd_, name) in VN_FIXED_HOLIDAYS:
66     df[f'hol_{name}'] = ((df['month']==m) & (df['day']==dd_)).astype(int)
67
68 # (f) Tet features
69 tet_lut = {y: pd.Timestamp(v) for y,v in TET_DATES.items()}
70 def nearest_tet_diff(dd):
71     cand1 = tet_lut.get(dd.year), tet_lut.get(dd.year-1), tet_lut.get(dd.year+1)
72     cand1 = [c for c in cand1 if c is not None]
73     valid = [(dd-c).days for c in cand1 if abs((dd-c).days)<=45]
74     return min(valid) if valid else 999
75 diffs = np.array([nearest_tet_diff(dd) for dd in d])
76 df['tet_days_diff'] = diffs
77 df['tet_in_7'] = (np.abs(diffs)<=7).astype(int)
78 df['tet_in_14'] = (np.abs(diffs)<=14).astype(int)
79 df['tet_before_7'] = ((diffs>=-7) & (diffs<0)).astype(int)
80 df['tet_after_7'] = ((diffs>0) & (diffs<=7)).astype(int)
81 df['tet_on'] = (diffs==0).astype(int)
82
83 # (g) Black Friday (Thu 6 cuoi thang 11)
84 def is_black_friday(dd):
85     if dd.month != 11: return 0
86     last = pd.Timestamp(year=dd.year, month=11, day=30)
87     last_fri = last - pd.Timedelta(days=(last.dayofweek - 4) % 7)
88     return int(dd == last_fri)
89 df['hol_black_friday'] = [is_black_friday(dd) for dd in d]

```

```

90
91 # (h) Promo windows
92 yrs = sorted(set(df['year'].tolist()))
93 for (name, sm, sd, dur, disc, recur) in PROMO_SCHEDULE:
94     in_prom = np.zeros(len(df), dtype=int)
95     since = np.full(len(df), -1.0)
96     until = np.full(len(df), -1.0)
97     discount = np.zeros(len(df))
98     for y in range(min(yrs)-1, max(yrs)+2):
99         if recur=='odd' and y%2==0: continue
100         start = pd.Timestamp(year=y, month=sm, day=sd)
101         end = start + pd.Timedelta(days=dur)
102         mask = (d>=start) & (d<=end)
103         in_prom[mask] = 1
104         since[mask] = (d[mask]-start).dt.days
105         until[mask] = (end-d[mask]).dt.days
106         discount[mask] = disc or 0
107         df[f'promo_{name}'] = in_prom
108         df[f'promo_{name}_since'] = since
109         df[f'promo_{name}_until'] = until
110         df[f'promo_{name}_disc'] = discount
111
112 # (i) Odd-year flag
113 df['is_odd_year'] = (df['year'] % 2).astype(int)
114
115 return df
116
117 # Test
118 test_features = build_features(pd.date_range('2023-01-01', '2023-01-10'))
119 print(f'Feature columns: {test_features.shape[1]}')
120 test_features.head()

```

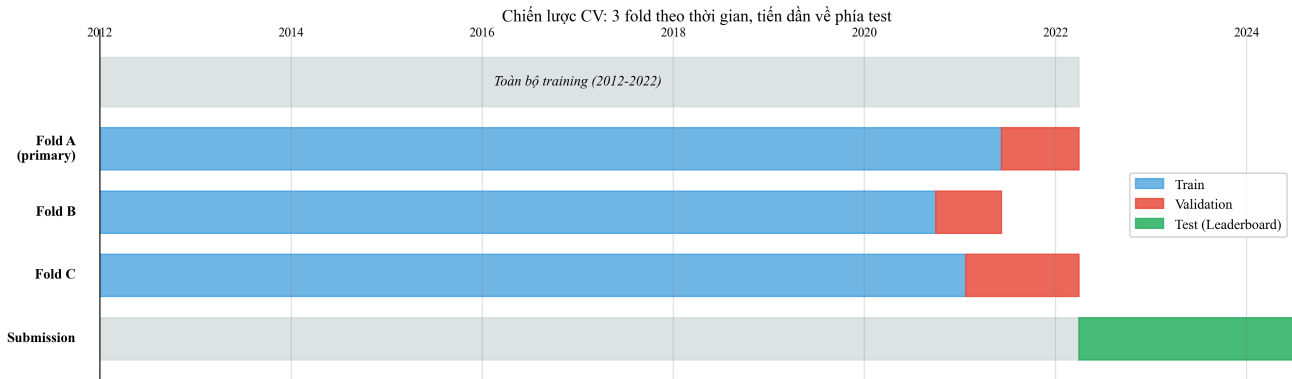
Lưu ý

Xây dựng các **đặc trưng Fourier** (sin/cos của day-of-year, day-of-week, day-of-month) đóng vai trò quan trọng đặc biệt cho mô hình tuyến tính Ridge. Giúp một mô hình tuyến tính học được pattern *lặp định kỳ* theo nhiều tần số khác nhau.

XVI. Chiến lược validation

Vì horizon test rất dài và phân phối lại khác biệt so với training, lựa chọn fold validation như sau.

XVI.1. Ba fold theo thời gian



Hình 19: Chiến lược CV ba fold theo thời gian. Fold A chọn 2022 làm validation, là fold chuẩn chính vì năm 2022 có phân phối gần nhất với test 2023. Fold B (validate 2021) kiểm tra độ ổn định giữa các năm. Fold C có cửa sổ rolling 12 tháng, gần với horizon submission.

Ba fold được thiết kế với ba mục tiêu bổ sung nhau:

- **Fold A (primary).** Train $\leq$ 2021-12, validate 2022. Đây là fold có phân phối validation *gần nhất* với test; mọi quyết định ablation quan trọng nên dựa trên fold này.
- **Fold B (stability).** Train $\leq$ 2020-12, validate 2021. Fold này kiểm tra xem một cải tiến có *ổn định qua năm* hay chỉ đúng cho 2022.
- **Fold C (horizon).** Train $\leq$ 2021-06, validate 2021-07 $\rightarrow$ 2022-06. Fold này mô phỏng cửa sổ 12 tháng liên tục, gần với horizon test nhất.

XVII. Lựa chọn và huấn luyện mô hình

XVII.1. Lựa chọn mô hình như thế nào?

Phân tích lựa chọn các mô hình ensemble học máy là các mô hình cần có tính diversity. Nếu hai mô hình cùng sai một hướng thì trung bình chúng vẫn sai hướng đó, không tạo ra thông tin bổ sung. Ngược lại, nếu một mô hình sai cao và một mô hình sai thấp, trung bình triệt tiêu một phần sai số,

Chúng ta chọn ba mô hình đại diện cho **ba họ giải thuật khác biệt nhất** trong bài toán forecasting cổ điển:

- **Ridge Regression** — đại diện *mô hình tuyến tính tham số*. Ridge học một hàm tuyến tính toàn cục trên không gian feature. Khi được cung cấp đủ Fourier features, Ridge có thể tái hiện dạng mùa vụ theo kiểu *tổng trọng số sin/cos nhiều tần số*. Ridge *ổn định*: train lại nhiều lần cho ra weight rất gần nhau, ít bị nhiễu bởi outlier. Nó đóng vai trò “anchor” — thành phần khó bị lệch bất thường trong ensemble.
- **LightGBM** — *mô hình cây gradient boosting*. Khác với Ridge, LGB không giả định quan hệ tuyến tính; mà học hàm dạng bậc thang (piecewise-constant) qua việc chia feature space. Ưu điểm là nắm được *tương tác* giữa các feature (ví dụ: “nếu là cuối tháng và có promo thì tăng mạnh”). LGB mạnh nhất nhưng cũng variance cao nhất giữa các seed, nên cần regularize bằng trọng số trong ensemble.
- **Prophet** — *mô hình phân rã có cấu trúc* (decomposition-based). Prophet chia chuỗi thành ba thành phần độc lập: trend (piecewise linear có changepoint), seasonality (Fourier đa tần số), và holidays. Vì cấu trúc được định trước, Prophet rất mạnh ở việc extrapolate seasonal component cho horizon dài cần cho bài toán 18 tháng.

Ngoài ra các mô hình hiện đại như neural network hay foundation model có thể bổ sung thêm diversity.

Kiểm chứng diversity bằng số

Một cách đo diversity nhanh: tính *Pearson correlation* của **vector residuals** (sai số) giữa mỗi cặp mô hình trên fold validation. Khi các cặp có correlation khoảng 0.6–0.7 (không quá cao, không quá thấp), blend đem lại cải thiện rõ. Trong thực nghiệm, ba cặp (Ridge, LGB), (Ridge, Prophet), (LGB, Prophet) đều rơi vào khoảng này.

XVII.2. M1: Ridge Regression

```
1 from sklearn.linear_model import Ridge
2
3 def train_ridge(X_train, y_train, alpha=3.0):
4     mu = X_train.mean(axis=0)
5     sigma = X_train.std(axis=0).replace(0, 1)
6     Xs = (X_train - mu) / sigma
```

```

7     model = Ridge(alpha=alpha, random_state=42)
8     model.fit(Xs, y_train)
9     return model, (mu, sigma)
10
11 def predict_ridge(model, X_test, stats):
12     mu, sigma = stats
13     return model.predict((X_test - mu) / sigma)
14
15 # Train tren log(Revenue)
16 y_train_log = np.log(feats["Revenue"].values)
17 ridge, stats = train_ridge(pd.DataFrame(X_train, columns=cols), y_train_log)
18 p_ridge = np.exp(predict_ridge(ridge, pd.DataFrame(X_test, columns=cols), stats))

```

Hai điểm quan trọng: (i) chuẩn hoá z-score trước khi fit vì Ridge nhạy cảm với scale, và (ii) train trên $\log(\text{Revenue})$ rồi exp ngược ra ngoài, vì log-space ổn định hơn khi biên độ dao động lớn.

XVII.3. M2: LightGBM với sample weighting

Cấu hình hyperparameter được chọn khá conservative (learning rate nhỏ, leaves vừa phải, regularization nhẹ) để tránh overfit. Thêm nữa là *sample weight theo era*: chúng ta ưu tiên giai đoạn 2014–2018 khi train.

Era-based sample weighting

Đây là một kỹ thuật cổ điển nhưng rất hiệu quả cho data có regime drift: thay vì để mọi mẫu có trọng số bằng nhau trong hàm loss, ta *upweight* vùng mà model cần học pattern tốt nhất, và *downweight* vùng gây nhiễu. Hiệu ứng là mô hình tập trung học *shape* từ vùng sạch, sau đó chỉ cần một tầng điều chỉnh mức là đủ cho phân phối mới.

```

1 import lightgbm as lgb
2
3 LGB_PARAMS = dict(
4     objective="regression", metric="mae",
5     learning_rate=0.03, num_leaves=63,
6     min_data_in_leaf=30,
7     feature_fraction=0.85, bagging_fraction=0.85, bagging_freq=5,
8     lambda_l2=1.0, seed=42, verbosity=-1,
9 )
10
11 # --- Sample weight: era-focused ---
12 years = feats["Date"].dt.year.values
13 w = np.full(len(feats), 0.01)
14 w[(years >= 2014) & (years <= 2018)] = 1.0 # scheme "high_era"
15
16 # --- Huan luyen 2 giai doan ---
17 # (1) Early stopping tren internal holdout (180 ngay cuoi)
18 intern = feats["Date"].max() - pd.Timedelta(days=180)
19 fit_idx = (feats["Date"] <= intern).values
20 ins_idx = (feats["Date"] > intern).values

```

```

21
22 booster_es = lgb.train(
23     LGB_PARAMS,
24     lgb.Dataset(X_train[fit_idx], y_train_log[fit_idx], weight=w[fit_idx]),
25     num_boost_round=5000,
26     valid_sets=[lgb.Dataset(X_train[ins_idx], y_train_log[ins_idx])],
27     callbacks=[lgb.early_stopping(300, verbose=False),
28               lgb.log_evaluation(0)])
29
30 # (2) Retrain toàn bộ dữ liệu với best_iteration
31 booster_final = lgb.train(
32     LGB_PARAMS,
33     lgb.Dataset(X_train, y_train_log, weight=w),
34     num_boost_round=booster_es.best_iteration)
35
36 p_lgb = np.exp(booster_final.predict(X_test))

```

Lưu ý

Mô hình được huấn luyện hai giai đoạn. Giai đoạn 1 dùng 180 ngày cuối làm validation để xác định `best_iteration`. Giai đoạn 2 huấn luyện lại trên *toàn bộ* dữ liệu với số vòng đã chọn. Cách này giúp model tận dụng được cả 180 ngày cuối trong khi tránh overfit.

XVII.4. M3: Prophet

```

1 from prophet import Prophet
2
3 def fit_prophet(train_df, post_regime_only=True):
4     if post_regime_only:
5         train_df = train_df[train_df["ds"] >= "2020-01-01"]
6     m = Prophet(
7         yearly_seasonality=True, weekly_seasonality=True,
8         daily_seasonality=False,
9         seasonality_mode="multiplicative",
10        changepoint_prior_scale=0.05)
11    for col in [c for c in train_df.columns if c.startswith("promo_")]:
12        m.add_regressor(col)
13    m.fit(train_df)
14    return m
15
16 tdf = pd.DataFrame({"ds": sales["Date"], "y": np.log(sales["Revenue"])} \
17                    .merge(build_promo_regressors(sales["Date"]), on="ds")
18 model_p = fit_prophet(tdf, post_regime_only=True)
19
20 vdf = pd.DataFrame({"ds": test_df["Date"]}) \
21        .merge(build_promo_regressors(test_df["Date"]), on="ds")
22 p_prophet = np.exp(model_p.predict(vdf)["yhat"].values)

```

`post_regime_only=True`: Prophet chỉ học trên dữ liệu 2020–2022. Lý do là component trend

của Prophet xử lý kém với jump lớn (ví dụ bước nhảy 2019–2020): khi train trên toàn bộ lịch sử, trend đoạn cuối có thể bị bóp méo.

XVII.5. Mô hình huấn luyện theo quý

Quan sát từ Hình 17 cho thấy các quý có mức độ biến động rất khác nhau. Một mô hình chung được train với trọng số như nhau cho mọi ngày sẽ bị “trung bình hoá” pattern riêng của mỗi quý — đặc biệt Q3 với biên độ biến động lớn nhất sẽ bị bỏ sót. Giải pháp được thiết kế ở đây là huấn luyện **bốn mô hình cụ thể theo quý** (Q-specialist), mỗi mô hình nhận thêm trọng số boost cho các ngày thuộc quý mà nó “phụ trách”.

Mô hình LightGBM cho các quý

Về *kiến trúc*, bốn specialist hoàn toàn giống nhau và giống mô hình M2 ở Mục XVII.3 (cùng LightGBM, cùng 150+ feature, cùng hyperparameter). Khác biệt duy nhất nằm ở **sample weight**: mỗi specialist có một quý “trọng điểm” được nhân hệ số 2×:

Specialist	Quý trọng điểm	Pattern tập trung học
Spec_Q1	Tháng 1–3	Hiệu ứng Tết Âm lịch, sụt giảm đầu năm
Spec_Q2	Tháng 4–6	Chiến dịch mid-year sale, đỉnh giữa năm
Spec_Q3	Tháng 7–9	Quý có margin biến động nhất, cần bắt chi tiết
Spec_Q4	Tháng 10–12	Đỉnh năm (Black Friday, 11.11, 12.12, Noel)

Cơ chế sample weight

Công thức weight chung cho mỗi specialist là sự kết hợp của hai layer:

```

1 # --- Ví dụ: specialist cho Q3 trên Revenue ---
2 TARGET_Q = 3          # quý trọng điểm
3 QBOOST    = 2.0        # hệ số boost
4
5 # Layer 1: high_era base (giống M2)
6 w = np.full(len(feet), 0.01)
7 w[(years >= 2014) & (years <= 2018)] = 1.0
8
9 # Layer 2: quarter boost - nhân 2x cho quý trọng điểm
10 Q_train = feat["Date"].dt.quarter.values
11 w[Q_train == TARGET_Q] *= QBOOST
12
13 # Huấn luyện 2 giai đoạn (giống M2, chỉ thay weight)
14 y = np.log(feet["Revenue"].values)
15 # ...
16 p_spec_Q3 = np.exp(booster_final.predict(X_test))
17 np.save("v50_specialist_Q3_Revenue.npy", p_spec_Q3)

```

Ví dụ với Spec_Q3, ta có ba mức weight khác nhau:

- Ngày Q3 trong giai đoạn 2014–2018: $\text{weight} = 2.0$ (bằng 1.0×2.0)
- Ngày Q1, Q2, Q4 trong giai đoạn 2014–2018: $\text{weight} = 1.0$
- Các ngày ngoài giai đoạn 2014–2018: $\text{weight} = 0.01$ (gần như bị bỏ qua)

Kết quả là hàm loss phạt *gấp đôi* khi Spec_Q3 dự đoán sai một ngày Q3 so với dự đoán sai một ngày của quý khác. Model học tốt hơn riêng cho Q3 trong khi vẫn có ngữ cảnh của các quý còn lại.

Vì sao không train chỉ trên quý đích?

Một phương án có vẻ đơn giản hơn là *chỉ train Spec_Q3 trên những ngày Q3* ($\text{weight} = 0$ cho các quý khác). Chúng tôi không chọn hướng này vì hai nhược điểm:

1. **Mất ngữ cảnh xuyên quý.** Các feature trend, regime, promo mang ý nghĩa trên toàn năm. Model chỉ thấy Q3 sẽ không học được xu hướng tăng trưởng nội năm.
2. **Giảm kích thước training xuống 1/4.** Chỉ còn ~ 450 ngày hiệu dụng, rất dễ overfit.

Q-boost = 2.0 là compromise: vẫn train trên toàn bộ dữ liệu để giữ ngữ cảnh, nhưng *nhường* loss về phía quý đích.

Vì pipeline cần dự báo hai biến khác nhau (Revenue và COGS), mỗi trong bốn “vị trí” Q-specialist trên thực tế là hai model được train song song

Trong sơ đồ pipeline ở Hình 10, khối “Q-Specialists 4 models” được gộp lại theo số quý cho gọn; số lượng thực tế là **4 quý $\times$ 2 biến target = 8 LightGBM**.

Prediction

Một điểm trong giai đoạn dự báo: chúng ta *không trung bình* prediction của bốn specialist. Thay vào đó, với mỗi ngày của test, ta chọn specialist thuộc đúng quý của ngày đó:

```

1 # Gộp prediction theo quý - Không trung bình
2 Q_test = test_df["Date"].dt.quarter.values
3 spec   = {q: np.load(f"v50_specialist_Q{q}_Revenue.npy") for q in [1,2,3,4]}
4
5 p_spec_composed = np.zeros(548)
6 for q in [1, 2, 3, 4]:
7     mask = Q_test == q
8     p_spec_composed[mask] = spec[q][mask]    # chỉ dùng Spec_Qq cho ngày Q_q
9 # Ví dụ:
10 # 15/02/2023 (Q1) -> lấy giá trị từ spec[1]
11 # 20/08/2023 (Q3) -> lấy giá trị từ spec[3]

```

Sau bước ghép này, `p_spec_composed` là một chuỗi 548 giá trị, được kết hợp từ bốn specialist theo đúng quý của từng ngày. Đây là output sẽ đi vào Tầng 1 của ensemble ở Mục XVIII..

XVIII. Chiến lược ensemble learning

Sau khi đã có nhiều mô hình thành phần (ba mô hình nền và bốn mô hình chuyên biệt theo quý), câu hỏi tiếp theo là: *kết hợp chúng ra sao để được một dự báo cuối cùng?* Có hai hướng tiếp cận chính mà chúng ta sẽ lần lượt xem xét. Hướng đầu đơn giản và trực quan nhưng hạn chế, hướng sau phức tạp hơn nhưng tận dụng được toàn bộ cấu trúc của bài toán.

XVIII.1. Hướng 1: Ensemble đơn giản

Ý tưởng đơn giản nhất là *trung bình trọng số phẳng* trên tất cả mô hình thành phần. Mỗi mô hình được xem như một “vote” bình đẳng, và công thức tổng hợp chỉ là một phép cộng tuyến tính:

$$\hat{y} = w_1 \cdot \text{Ridge} + w_2 \cdot \text{LGB}_{\text{base}} + w_3 \cdot \text{Prophet} + w_4 \cdot \text{LGB}_{\text{Q-spec}},$$

với các trọng số w_i có thể chọn đều nhau (kiểu “trung bình đơn”) hoặc tune qua grid search trên validation.

```

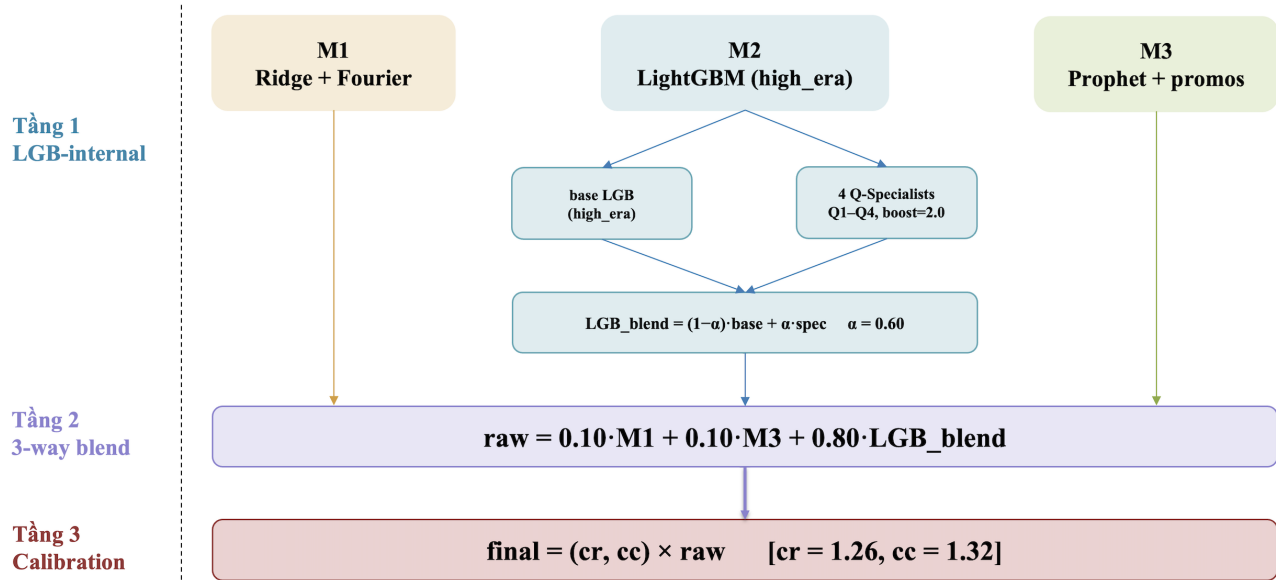
1 # Ensemble đơn giản: trung bình có trọng số
2 w_ridge, w_lgb, w_prophet, w_spec = 0.10, 0.45, 0.10, 0.35 # tune trên CV
3 pred_simple = (w_ridge * p_ridge
4               + w_lgb * p_lgb
5               + w_prophet * p_prophet
6               + w_spec * p_spec_composed) # p_spec đã ghép theo quý
7
8 # Calibration thông nhất
9 final = 1.26 * pred_simple # cho Revenue

```

Cách tiếp cận này có ưu điểm *rất rõ ràng và dễ giải thích*, nhưng trong thực nghiệm nó không đạt kết quả tốt nhất vì hai lý do. Thứ nhất, các mô hình thành phần có bản chất khác nhau rõ rệt: LightGBM (kể cả base và specialist) mạnh hơn đáng kể so với Ridge hay Prophet. Thứ hai, hai thành phần LGB (base và specialist) có *mối liên hệ đặc biệt* với nhau — chúng cùng là gradient boosting, chỉ khác nhau ở trọng số mẫu. Đặt chúng song song bên cạnh Ridge và Prophet không tận dụng được cấu trúc này.

XVIII.2. Hướng 2: Ensemble phân tầng

Thay vì ép tất cả thành phần vào cùng một công thức, chúng ta chia quá trình kết hợp thành *nhiều tầng*, mỗi tầng giải quyết một bài toán con có cấu trúc riêng. Ý tưởng là: *hợp nhất những thành phần gần nhau trước, rồi mới hợp nhất các họ mô hình khác biệt; cuối cùng xử lý riêng vấn đề level*.



Hình 20: Kiến trúc ensemble. Base LGB và các specialist được hợp nhất trước ở Tầng 1 qua hệ số α . Kết quả LGB_blend tiếp tục được pha trộn với Ridge và Prophet ở Tầng 2 bằng trọng số 0.10/0.10/0.80. Tầng 3 áp hai hệ số calibration cr , cc để scale output về mức phù hợp với 2023–2024.

Ba tầng được tổ chức tuần tự theo logic sau:

- Tầng 1 — Hợp nhất trong cùng một họ mô hình.** Base LGB và bốn Q-specialist đều là gradient boosting, chỉ khác nhau ở trọng số mẫu. Vì cùng một họ, chúng có cấu trúc sai số tương tự. Ta cần *một* hệ số α duy nhất để điều phối: $\text{LGB}_{\text{blend}} = (1 - \alpha) \cdot \text{base} + \alpha \cdot \text{Q-spec}$. Giá trị α càng lớn thì specialist càng được trust nhiều hơn.
- Tầng 2 — Hợp nhất giữa các họ mô hình khác nhau.** Sau khi đã gộp LGB nội bộ, kết quả đầu ra được xem như “họ LGB tổng hợp” đặt cùng hàng với Ridge và Prophet. Ba họ này có bản chất rất khác nhau, nên cần ba trọng số riêng. Trong thực nghiệm, tỉ trọng (0.10, 0.10, 0.80) cho Ridge / Prophet / LGB_blend cho kết quả tốt nhất — LGB mạnh hơn hẳn, Ridge và Prophet giữ vai trò “anchor” ổn định, chống overshoot.
- Tầng 3 — Điều chỉnh mức (calibration).** Kết quả raw sau Tầng 2 vẫn nằm ở *level* của vùng học (2014–2018), cần được scale lên mức của 2023–2024. Đây là vấn đề khác hẳn với bài toán blend, nên giải quyết riêng bằng hai hệ số nhân cr (cho Revenue) và cc (cho COGS) tune trực tiếp trên Leaderboard.

Vì sao cần tách calibration khỏi blend?

Cross-validation nội bộ không mô phỏng được *mức thật* của 2023 vì năm 2022 (fold validation) có phân phối khác với 2023. Nếu để calibration được tune bằng CV, kết quả sẽ bị chệch. Do đó chiến lược tối ưu là: các hệ số blend ở Tầng 1 và Tầng 2 được chọn trên CV để tối thiểu hoá MAE của *hình dạng* prediction; hai hệ số calibration ở Tầng 3 được chọn để phủ đúng *mức* của test.

So với Hướng 1, kiến trúc phân tầng ở đây có hai lợi ích định tính: (a) cho phép dùng trọng số khác nhau cho các cặp thành phần có bản chất khác nhau, (b) tách riêng vấn đề level ra khỏi quá trình blend, giúp mỗi bước tối ưu độc lập.

XVIII.3. Mã nguồn ensemble phân tầng

```

1 # --- Tai cac prediction raw da cache ---
2 cache = np.load("v26_high_era_test_raw.npz", allow_pickle=True)
3 p_ridge, p_ridge_c = cache["p_rd_rev"], cache["p_rd_cog"]
4 p_prophet, p_prophet_c = cache["p_pr_rev"], cache["p_pr_cog"]
5 p_lgb, p_lgb_c = cache["p_lgb_rev"], cache["p_lgb_cog"]
6
7 dates = pd.to_datetime(cache["dates"])
8 Q = dates.quarter.values
9
10 # Specialists ghep theo quy
11 spec_rev = {q: np.load(f"v50_specialist_Q{q}_Revenue.npy") for q in [1,2,3,4]}
12 spec_cog = {q: np.load(f"v50_specialist_Q{q}_COGS.npy") for q in [1,2,3,4]}
13 lgb_spec_rev = np.zeros(548); lgb_spec_cog = np.zeros(548)
14 for q in [1,2,3,4]:
15     mask = Q == q
16     lgb_spec_rev[mask] = spec_rev[q][mask]
17     lgb_spec_cog[mask] = spec_cog[q][mask]
18
19 # --- Hyperparameter ensemble (chi tiet cach chon trong Muc VIII) ---
20 ALPHA = 0.60
21 CR, CC = 1.26, 1.32
22
23 # Tang 1: hop nhap LGB noi bo
24 lgb_blend_rev = ALPHA * lgb_spec_rev + (1 - ALPHA) * p_lgb
25 lgb_blend_cog = ALPHA * lgb_spec_cog + (1 - ALPHA) * p_lgb_c
26
27 # Tang 2: hop nhap 3 ho mo hinh
28 raw_rev = 0.10 * p_prophet + 0.10 * p_ridge + 0.80 * lgb_blend_rev
29 raw_cog = 0.10 * p_prophet_c + 0.10 * p_ridge_c + 0.80 * lgb_blend_cog
30
31 # Tang 3: calibration
32 final_rev = CR * raw_rev
33 final_cog = CC * raw_cog
34
35 # --- Xuat submission ---
36 pd.DataFrame({
37     "Date": dates.strftime("%Y-%m-%d"),
38     "Revenue": final_rev,
39     "COGS": final_cog,
40 }).to_csv("submission.csv", index=False)

```

XIX. Parameters Tuning

Pipeline có nhiều hyperparameter, nhưng không phải tham số nào cũng được tune bằng cùng một cách.

XIX.1. Tổng quan: ba nhóm tham số và công cụ tuning tương ứng

Tất cả hyperparameter của pipeline có thể được xếp vào một trong ba nhóm, tùy theo việc chúng ảnh hưởng đến khía cạnh nào của prediction.

Nhóm	Tham số điển hình	Công cụ tuning
(A) Hình dạng	LGB params, sample weight scheme, blend weights	CV trên fold A/B/C
(B) Tinh chỉnh	α , Q-boost	CV $\rightarrow$ xác nhận trên LB

Nhóm (A) — **tham số shape** ảnh hưởng *độ uốn lượn* của prediction theo thời gian. Chúng có thể được đánh giá tin cậy bằng CV vì hình dạng mùa vụ ổn định qua các năm (đã kiểm chứng ở Mục XIV.). Chiến lược: tìm kiếm tham lam trên fold A, chọn điểm tốt nhất, xác nhận tính ổn định trên fold B và C trước khi chốt.

Nhóm (B) — **tham số tinh chỉnh** nằm ở giữa hai nhóm trên. Chúng có thể tune sơ bộ bằng CV (nhận định hướng) nhưng kết quả cuối cần được xác nhận bằng LB. Ví dụ: α có thể đánh giá được trên CV vì nó liên quan đến sai số shape của specialist, nhưng ảnh hưởng đến level cũng có (thay đổi α thay đổi trung bình LGB_blend), nên cần cross-check.

XIX.2. Chiến lược tuning cho từng nhóm

Nhóm (A) — tune bằng Cross-Validation

Các tham số của LightGBM (`learning_rate`, `num_leaves`, `min_data_in_leaf`, `lambda_12`) được tune trên fold A. Vì LGB có variance giữa seed, ta lấy trung bình qua 3 seed để đánh giá ổn định. Giá trị cuối cùng được xác nhận bằng việc rerun trên fold B.

Sample weight scheme (`high_era`, `t1718`, `up_HIGH`, `uniform`, ...) cũng thuộc nhóm (A). Chiến lược: thử mỗi scheme, ghi MAE CV trên fold A. Scheme nào cho CV tốt nhất và ổn định trên fold B là scheme được chọn. Trong thực nghiệm, `high_era` thắng trên cả hai fold.

Trọng số blend 3-way (w_{ridge} , w_{lgb} , w_{prophet}) được thay đổi trên các giá trị $\{0.05, 0.10, 0.15, 0.20\}$ với ràng buộc tổng bằng 1. Điểm tối ưu ổn định rơi vào $(0.10, 0.80, 0.10)$.

Nhóm (B)

Hai hệ số cr và cc được tune *song song* nhưng độc lập: mỗi lần cố định một hệ số ở midpoint và quét hệ số còn lại. Dải thử ban đầu là $\{1.18, 1.22, 1.26, 1.30, 1.34\}$ cho cr và $\{1.28, 1.30, 1.32, 1.34, 1.36\}$ cho cc .

Hệ số	Dải thử	Kết quả trên LB
cr (Revenue)	1.18, 1.22, 1.26, 1.30	Đỉnh tại $cr = 1.26$
cc (COGS)	1.28, 1.30, 1.32, 1.34	Đỉnh tại $cc = 1.32$

Sau khi có dải rộng, tinh chỉnh bước 0.01 quanh điểm đỉnh để chắc chắn. Giá trị $(cr, cc) = (1.26, 1.32)$ là kết quả empirical: tổng hợp giữa raw prediction của blend (đã pha trộn từ nhiều nguồn có level khác nhau — LGB `high_era` dự báo cao, Prophet post-regime dự báo thấp, Ridge ở giữa) và level thực của test 2023–2024. Cần lưu ý rằng cr không có ý nghĩa “scale lên” hay “scale xuống” đơn giản — nó là tham số bù trừ cho mọi thiên lệch hệ thống còn lại sau ensemble, được tune trên LB để đạt MAE thấp nhất. Tham số α (tỉ lệ specialist trong LGB_blend) trước hết được sweep sơ bộ qua $\{0.3, 0.5, 0.7\}$ trên CV fold A để định hướng. Sau đó các giá trị gần đỉnh (0.45, 0.50, 0.55, 0.60, 0.70) được submit lên LB để xác nhận.

Tham số Q-boost (hệ số nhân weight cho quý target trong specialist) tune theo cách tương tự: thử $\{1.5, 2.0, 3.0\}$ trên CV, chọn giá trị tốt nhất, xác nhận bằng một lượt submit. Kết quả chốt: Q-boost = 2.0.

XX. Khả năng giải thích mô hình (Explainability)

Sau khi pipeline đã hoàn chỉnh — ba mô hình gốc, bốn Q-specialist, ensemble phân tầng (Mục XVIII.) và các hệ số calibration đã tune (Mục XIX.) — một câu hỏi đặt ra: *mô hình thực sự đã học được điều gì, và nó dựa vào yếu tố nào để dự báo?* Trả lời câu hỏi này không chỉ để hiểu rõ cơ chế dự báo bên trong, mà quan trọng hơn là để **kiểm chứng ngược**: xác nhận rằng các pattern phát hiện trong Mục XIV. (Tết, cuối tháng, chu kỳ chẵn/lẻ) đã được mô hình sử dụng *đúng*, và *định lượng* mức độ quan trọng của từng pattern.

XX.1. Đối tượng và công cụ giải thích

Ensemble cuối là tổ hợp nhiều mô hình, nhưng trọng số không đồng đều. Gồm tổ hợp các mô hình: họ cây (base LightGBM cùng bốn Q-specialist) chiếm 0.80 tổng trọng số, trong khi Ridge và Prophet mỗi nhánh chỉ 0.10. Vì vậy ta tập trung giải thích **base LightGBM** — mô hình đại diện chi phối ~ 80% dự báo — và dùng **Prophet** làm đối chứng.

Công cụ là **SHAP** (SHapley Additive exPlanations), cụ thể TreeSHAP tính trực tiếp qua LightGBM bằng `predict(pred_contrib=True)` — cho kết quả *chính xác* cho mô hình cây, không cần xấp xỉ và không cần thư viện ngoài. SHAP có tính cộng tính: với mỗi ngày x

$$f(x) = \phi_0 + \sum_{i=1}^n \phi_i(x),$$

trong đó ϕ_0 là giá trị cơ sở (kỳ vọng đầu ra của mô hình) và $\phi_i(x)$ là đóng góp của feature thứ i . Tính cộng tính này được kiểm tra trực tiếp: sai số thiếp lập lại $|\phi_0 + \sum_i \phi_i - f(x)|$ tối đa chỉ $\sim 3 \times 10^{-14}$ trên toàn bộ 548 ngày, xác nhận phân rã là đúng. Vì mô hình học trên $\log(\text{Revenue})$, đóng góp cộng trong không gian log tương đương phép *nhân* trên doanh thu gốc; một đóng góp $\Delta\phi$ quy ra phần trăm theo $e^{\Delta\phi} - 1$. Vì vậy các giải thích chúng ta dùng sẽ phát biểu được trực tiếp bằng %.

Tính TreeSHAP và gom feature thành nhóm nghiệp vụ

```

1 import numpy as np, pandas as pd
2
3 # TreeSHAP cho base LightGBM; cot cuoi cung la base value (ky vong dau ra)
4 def tree_shap(bf, X):
5     contrib = bf.predict(X, pred_contrib=True)          # (n, n_features + 1)
6     return contrib[:, :-1], float(contrib[0, -1])
7
8 X_te = test_df[cols].values.astype(float)
9 sv_rev, base_rev = tree_shap(bf_rev, X_te)
10 sv_cog, base_cog = tree_shap(bf_cog, X_te)
11
12 # Kiem tra tinh cong tinh (log space): base + tong(SHAP) ~ prediction
13 err = np.max(np.abs(base_rev + sv_rev.sum(1) - bf_rev.predict(X_te)))
14 print("Sai so cong tinh:", err)                        # ~ 3e-14
15

```

```

16 # Gom 81 đặc trưng thành nhóm nghiệp vụ (khu nhiều do feature Fourier tương quan)
17 def group_of(c):
18     if c == 'is_odd_year': return 'Year-parity'
19     if c.startswith('tet_'): return 'Tet'
20     if c.startswith('hol_'): return 'Le VN & BF'
21     if c.startswith('promo_'): return 'Promo'
22     if c.startswith(('is_first', 'is_last')) or c in ('days_to_eom', 'days_from_som'):
23         return 'Edge-of-month'
24     if c.startswith(('sin_w', 'cos_w')) or c in ('dow', 'is_weekend'): return 'Weekly'
25     if c.startswith(('sin_y', 'cos_y', 'sin_m', 'cos_m')) or c in ('month', 'doy', 'quarter',
26         'day', 'dim'): return 'Seasonality'
27     if c.startswith('regime_') or c in ('t_days', 't_years', 'year'): return 'Regime &
28         trend'
29     return 'Khac'
30 groups = [group_of(c) for c in cols]
31
32 def group_pct(sv):
33     # đóng góp % mỗi nhóm = tổng
34     # mean|SHAP|
35     mab = np.abs(sv).mean(0)
36     d = {}
37     for c, g, v in zip(cols, groups, mab): d[g] = d.get(g, 0.0) + v
38     tot = sum(d.values())
39     return {g: 100*v/tot for g, v in sorted(d.items(), key=lambda x: -x[1])}
40 gp_rev, gp_cog = group_pct(sv_rev), group_pct(sv_cog)

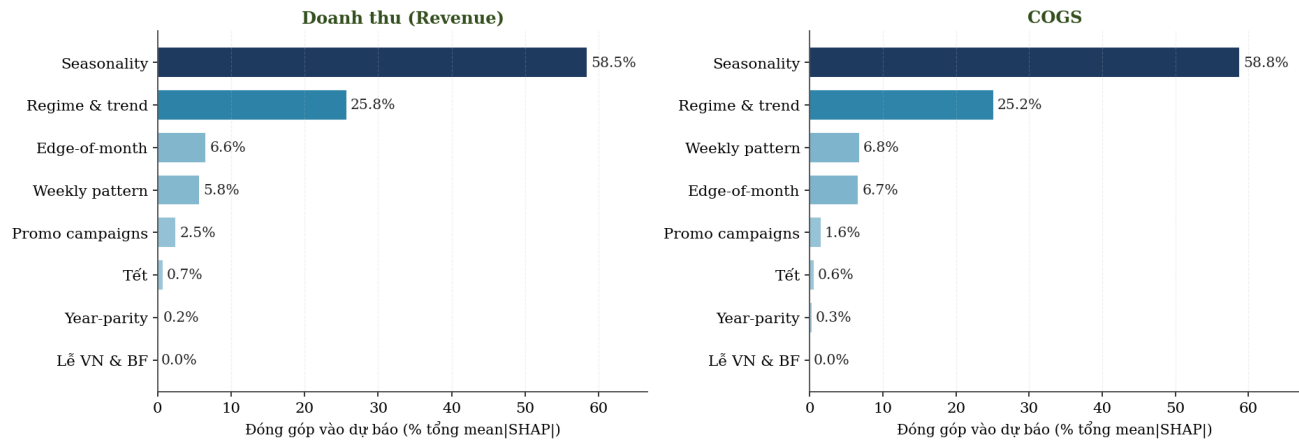
```

Global importance và Local effect là hai góc nhìn khác nhau

Global importance đo trung bình $|\phi_i|$ trên cả 548 ngày — trả lời pattern nào quan trọng xét trên toàn horizon. **Local effect** đo độ lớn đóng góp ngay tại những ngày pattern đó xuất hiện. Một pattern hiếm như Tết (chỉ ~ 2 tuần mỗi năm) sẽ có global importance thấp dù tác động cục bộ rất rõ. Vì vậy hai mục dưới đây báo cáo song song cả hai góc nhìn để tránh kết luận sai.

XX.2. Global importance

Hình 21 gom 81 features đã sử dụng thành tám nhóm nghiệp vụ đặc trưng rồi cộng dồn $|\phi_i|$ theo nhóm. Việc gom nhóm còn có tác dụng kỹ thuật: các feature Fourier vốn tương quan cao nên SHAP chia nhỏ tầm quan trọng cho từng feature, khiến xếp hạng đơn lẻ kém ổn định — cộng theo nhóm khử được nhiễu này.



Hình 21: Đóng góp của tám nhóm pattern vào dự báo, đo bằng tổng mean|SHAP| trên giai đoạn test 2023–2024, cho Revenue (trái) và COGS (phải). Hai nhóm Seasonality và Regime & trend chi phối, chiếm $\sim 84\%$.

Biểu đồ đóng góp theo nhóm và bảng global importance

```

1 import matplotlib.pyplot as plt
2 fig, axes = plt.subplots(1, 2, figsize=(13.5, 4.6))
3 for ax, (gp, title) in zip(axes, [(gp_rev, 'Doanh thu'), (gp_cog, 'COGS')]):
4     names, pct = list(gp.keys()), list(gp.values())
5     ax.barh(names[::-1], pct[::-1],
6             color=[plt.cm.Blues(0.35 + 0.6*(p/max(pct))) for p in pct][::-1])
7     ax.set_title(title); ax.set_xlabel('% tổng mean|SHAP|')
8 plt.tight_layout(); plt.show()
9
10 df_global = pd.DataFrame({'Nhom': list(gp_rev.keys()),
11                           'Revenue (%)': [round(gp_rev[g], 1) for g in gp_rev],
12                           'COGS (%)': [round(gp_cog.get(g, 0.0), 1) for g in gp_rev]})
13 print(df_global)

```

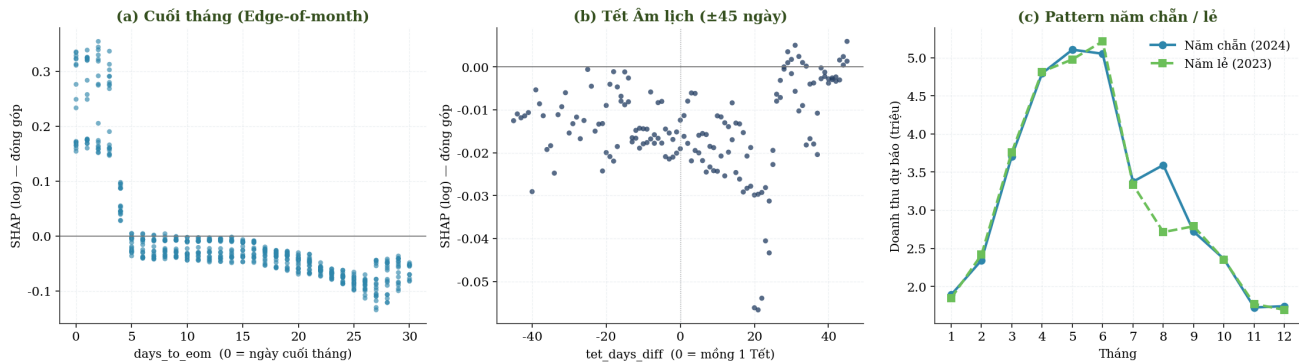
Kết quả nổi bật: dự báo doanh thu dựa **chủ yếu vào mùa vụ năm** (58.5%) và **mặt bằng regime/trend** (25.8%), hai nhóm chiếm $\sim 84\%$ tầm quan trọng. Phần còn lại chia cho cuối tháng (6.6%), thứ trong tuần (5.8%) và promo (2.5%); riêng Tết và chu kỳ chẵn/lẻ đóng góp global rất nhỏ ($\leq 0.7\%$). COGS có cấu trúc gần như y hệt (mùa vụ 58.8%, regime 25.2%) — xác nhận COGS rất gần với Revenue, đúng như quan hệ tuyến tính chặt đã thấy ở Hình 18.

Nhóm pattern	Revenue (%)	COGS (%)	Ý nghĩa
Seasonality (mùa vụ năm)	58.5	58.8	Hình mùa vụ trong năm (Fourier $k = 1.5$)
Regime & trend	25.8	25.2	Mặt bằng theo thời gian, ba regime
Edge-of-month	6.6	6.7	Hiệu ứng cuối/đầu tháng
Weekly pattern	5.8	6.8	Thứ trong tuần (weekday > weekend)
Promo campaigns	2.5	1.6	Sáu chiến dịch khuyến mãi theo lịch
Tết	0.7	0.6	Hiếm nên global thấp
Year-parity	0.2	0.3	Pattern năm chẵn/lẻ
Lễ VN & Black Friday	0.0	0.0	Gần như không đáng kể

Phát hiện này nhất quán với hai thành phần quan trọng khi thiết kế của pipeline: Fourier seasonality (Mục XV.) và sample weighting kết hợp calibration cho regime (Mục XVII., Mục XVIII.). Qua đây, SHAP cho thấy rõ mô hình đã học được những đặc trưng nổi bật và cho thấy những phân tích EDA dự đoán là đúng và có cơ sở.

XX.3. Local effect

Global importance cho biết nhóm nào đóng góp nhiều; để hiểu rõ cơ chế, sử dụng dependence plot cho ta thấy quan hệ giữa giá trị feature và đóng góp SHAP (Hình 22). Để đo độ lớn cục bộ, ta dùng hai phép đo bổ sung cho nhau, được tính trong khối lệnh dưới đây.



Hình 22: Dependence plot của ba driver then chốt. (a) `days_to_eom`: đóng góp SHAP tăng mạnh và dương ở 0–3 ngày cuối tháng. (b) `tet_days_diff`: đóng góp âm quanh mùng 1 Tết. (c) Doanh thu dự báo theo tháng, tách năm chẵn/lẻ: tháng 8 năm chẵn cao hơn rõ rệt.

Hiệu ứng cục bộ: chênh lệch tổng và SHAP cô lập theo nhóm

```

1 def shap_of(name): return sv_rev[:, cols.index(name)], test_df[name].values # cho
                                dependence plot
2
3 # Luoi day du 2023-2024 (bao gom thang 8 nam chan, vi cua so test dung o 2024-07-01)

```

```

4 grid = build_features(pd.date_range('2023-01-01', '2024-12-31', freq='D'))
5 glog_rev = bf_rev.predict(grid[cols].values.astype(float))
6 glog_cog = bf_cog.predict(grid[cols].values.astype(float))
7
8 def gidx(g): return [i for i, gg in enumerate(groups) if gg == g]
9 ti = test_df.reset_index(drop=True)
10 # (1) chênh lệch dự báo TỔNG giữa hai nhóm ngày (gom cả confound mua vụ, dùng cách EDA
    do)
11 def eff_total(sv, a, b): return 100*np.expm1(sv[a.values].sum(1).mean() - sv[b.values].
    sum(1).mean())
12 # (2) SHAP CO LAP: chi công SHAP của riêng nhóm driver
13 def eff_iso(sv, g, a, b):
14     c = sv[:, gidx(g)].sum(1); return 100*np.expm1(c[a.values].mean() - c[b.values].
    mean())
15
16 a_eom, b_eom = ti['is_last1']==1, ti['day'].between(13,17)
17 a_tet, b_tet = ti['tet_in_7']==1, ti['tet_days_diff'].abs(>30)
18 a_yp, b_yp = ti['is_odd_year']==0, ti['is_odd_year']==1
19 augE = (grid['month']==8)&(grid['is_odd_year']==0); aug0 = (grid['month']==8)&(grid['
    is_odd_year']==1)
20 aug_rev = 100*np.expm1(glog_rev[augE.values].mean() - glog_rev[aug0.values].mean())
21 aug_cog = 100*np.expm1(glog_cog[augE.values].mean() - glog_cog[aug0.values].mean())
22 sv_grid = bf_rev.predict(grid[cols].values.astype(float), pred_contrib=True)[: , :-1]
23 yp_grid = sv_grid[:, gidx('Year-parity')].sum(1) # SHAP co lap year-parity,
    riêng tháng 8
24 aug_iso = 100*np.expm1(yp_grid[augE.values].mean() - yp_grid[aug0.values].mean())
25
26 df_local = pd.DataFrame({
27     'Driver': ['Cuoi thang', 'Quanh Tet', 'Thang 8 chan/le'],
28     'Rev tong (%)': [round(eff_total(sv_rev,a_eom,b_eom)), round(eff_total(sv_rev,
    a_tet,b_tet)), round(aug_rev)],
29     'Rev co lap (%)': [round(eff_iso(sv_rev,'Edge-of-month',a_eom,b_eom),1),
    round(eff_iso(sv_rev,'Tet',a_tet,b_tet),1),
    round(aug_iso, 1)],
30     'COGS tong (%)': [round(eff_total(sv_cog,a_eom,b_eom)), round(eff_total(sv_cog,
    a_tet,b_tet)), round(aug_cog)]})
31
32 print(df_local)
33
34
35 gain = bf_rev.feature_importance(importance_type='gain') # kiem tra
    cheo
36 overlap = len({cols[i] for i in np.argsort(gain)[: -1] [:10]} &
37     {cols[i] for i in np.argsort(np.abs(sv_rev).mean(0))[: -1] [:10]})
38 print("SHAP cap gain (top-10):", overlap, "/10")

```

Gọi A và B là hai nhóm ngày được so sánh. Nhờ tính cộng tính của SHAP, tổng đóng góp của mọi đặc trưng trong một ngày bằng log dự báo trừ giá trị cơ sở: $\sum_i \phi_i(x) = \log f(x) - \phi_0$. Ta dùng hai đại lượng bổ sung cho nhau.

(1) Chênh lệch dự báo tổng — cộng SHAP của tất cả đặc trưng:

$$\Delta_{\text{tổng}} = \exp\left(\left|\overline{\sum_i \phi_i}\right|_A - \left|\overline{\sum_i \phi_i}\right|_B\right) - 1.$$

(2) **SHAP cô lập của nhóm g** — chỉ cộng SHAP của các đặc trưng thuộc nhóm g :

$$\Delta_{\text{cô lập}}(g) = \exp\left(\overline{\sum_{i \in g} \phi_i}\Big|_A - \overline{\sum_{i \in g} \phi_i}\Big|_B\right) - 1,$$

trong đó $\overline{(\cdot)}\Big|_A$ là trung bình trên các ngày thuộc nhóm A . Phép $\exp(\cdot) - 1$ đưa hiệu trong không gian log về phần trăm trên doanh thu gốc. Ba cặp nhóm ngày A/B được định nghĩa như sau.

Driver	Nhóm A	Nhóm B (đối chứng)
Cuối tháng	<code>is_last1</code> = 1 (ngày cuối tháng)	ngày 13–17 (giữa tháng)
Tết	trong ± 7 ngày quanh mùng 1	cách Tết > 30 ngày
Tháng 8 chẵn/lẻ	tháng 8 năm chẵn	tháng 8 năm lẻ

Cách đọc hai cột hiệu ứng cục bộ

Chênh lệch tổng trả lời doanh thu dự báo ở nhóm A cao/thấp hơn nhóm B bao nhiêu %, gộp mọi yếu tố khác biệt giữa hai nhóm ngày (kể cả mùa vụ) — đúng cách phân phân tích dữ liệu so sánh mức doanh thu thực. **SHAP cô lập** trả lời mô hình gán trực tiếp bao nhiêu cho riêng nhóm đặc trưng đó. Khi hai số lệch nhau, phần chênh là do các nhóm khác.

Ví dụ tính (Tết). Hiệu hai trung bình trong không gian log là $\Delta = \overline{\sum_i \phi_i}\Big|_A - \overline{\sum_i \phi_i}\Big|_B \approx -0.56$. Quy ra phần trăm: $e^{-0.56} - 1 \approx -0.43$, tức doanh thu quanh Tết thấp hơn ngày thường khoảng 43%.

Hai phép đo cho hai ý nghĩa khác nhau. *Rev tổng* là chênh lệch dự báo giữa hai nhóm ngày, đo theo đúng cách EDA so sánh mức doanh thu thực, nên bao hàm cả thành phần mùa vụ của các ngày đó. *Rev cô lập* chỉ cộng đóng góp của riêng nhóm đặc trưng tương ứng, cho biết mô hình gán trực tiếp bao nhiêu cho chính nhóm đó. Kết quả tổng hợp trong bảng sau:

Driver	Rev tổng (%)	Rev cô lập (%)	COGS tổng (%)	Đối chiếu EDA
Cuối tháng (cuối vs giữa tháng)	+69	+31.7	+79	Hình 14 (gấp đôi)
Quanh Tết (± 7 ngày vs thường)	-43	≈ 0	-44	Hình 13 (sụt sâu)
Tháng 8 năm chẵn vs lẻ	+35	+0.5	+2	Hình 15 ($\sim 1.6\times$)

Hai cột *tổng* và *cô lập* cho thấy một điểm quan trọng. Với hiệu ứng cuối tháng, chênh lệch tổng +69% nhưng riêng nhóm Edge-of-month chỉ đóng góp +31.7%; phần còn lại đến từ vị trí mùa vụ của các ngày cuối tháng. Với Tết, chênh lệch tổng -43% nhưng đóng góp cô lập của nhóm đặc trưng Tết ≈ 0 ; tương tự, chênh lệch tháng 8 năm chẵn/lẻ +35% nhưng đóng góp cô lập của `is_odd_year` chỉ +0.5%. Nghĩa là mô hình tái hiện mức sụt giảm dịp Tết và chu kỳ chẵn/lẻ gần

như hoàn toàn thông qua thành phần mùa vụ năm (Fourier), *không* qua các đặc trưng chuyên dụng cho Tết và năm chẵn/lẻ — các đặc trưng này gần như thừa trong cấu hình hiện tại. Đây là một phát hiện trực tiếp gợi ý hướng cải thiện: tăng trọng số hoặc thiết kế lại để mô hình dựa vào chúng nhiều hơn, qua đó tái hiện đúng *biên độ* (EDA ước lượng sơ bộ tháng 8 chênh $\sim 1.6\times$, mức tổng mô hình tái hiện chỉ $\sim 1.35\times$) thay vì chỉ đúng *chiều*. COGS cho hiệu ứng cục bộ cùng dấu và độ lớn tương đương (cuối tháng $+79\%$, Tết -44%).

Từ các con số này có thể rút ra ba đề xuất vận hành: (i) tập trung khuyến mãi, tồn kho và năng lực giao vận vào tuần cuối tháng; (ii) không tung khuyến mãi lớn đúng dịp Tết mà chuyển ngân sách sang giai đoạn hậu-Tết khi cầu hồi phục; (iii) chuẩn bị tồn kho cho mức tăng doanh thu tháng 8 của năm chẵn (2024).

XX.4. Đối chứng định lượng với Prophet

Prophet là một họ mô hình hoàn toàn khác (phân rã có cấu trúc thay vì cây), nên việc kiểm tra phân tích Prophet độc lập với SHAP sẽ là một cách xác minh hiệu quả.

Phép giải thích trên chỉ dựa vào một mô hình (LightGBM). Để kiểm chứng độc lập, ta đối chiếu với Prophet — một họ mô hình khác hẳn, dự báo bằng cách *phân rã* chuỗi thành xu hướng, thành phần theo tuần và thành phần mùa vụ năm. Việc đối chứng gồm hai phần. **Định tính:** vẽ ba thành phần của Prophet để quan sát trực tiếp hình dạng mùa vụ nó học được (Hình 23).

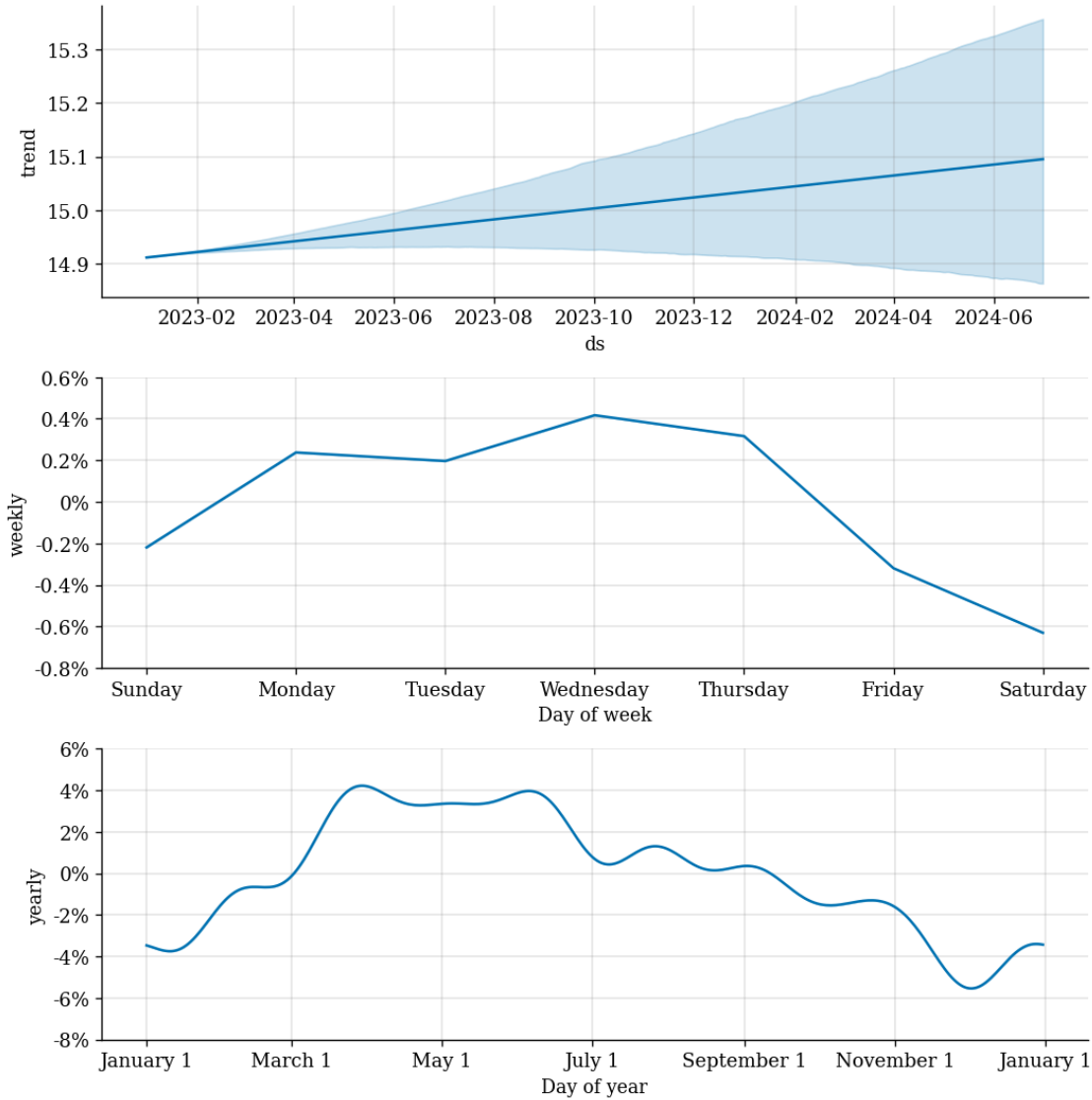
Định lượng: so sánh *đường cong mùa vụ* của hai mô hình theo ngày trong năm (day-of-year, 1–365). Với LightGBM, đóng góp mùa vụ của mỗi ngày là tổng SHAP của nhóm Seasonality; lấy trung bình theo ngày trong năm ta được một đường cong. Với Prophet, thành phần yearly sẵn là đường cong đó. Hệ số tương quan Pearson giữa hai đường cong cho biết hai mô hình có *đồng thuận* về việc doanh thu cao/thấp vào thời điểm nào trong năm hay không — r càng gần 1 thì kết luận giải thích càng đáng tin vì không phụ thuộc một mô hình duy nhất.

Đối chứng định lượng với Prophet

```

1 # Phan ra Prophet (trend / weekly / yearly / regressor promo)
2 vdf = pd.DataFrame({'ds': test_df['Date']}).merge(build_promo_regressors(test_df['Date']
3                                                    ), on='ds')
4 fc = mp_r.predict(vdf)
5 mp_r.plot_components(fc); plt.show()
6 # Tương quan thành phần mùa vụ năm của Prophet vs SHAP mùa vụ của LGB (theo day-of-year)
7 doy = test_df['Date'].dt.dayofyear.values
8 lgb_seas = pd.Series(sv_rev[:, gidx('Seasonality')].sum(1), index=doy).groupby(level=0)
9                                     .mean()
10 pro_year = pd.Series(fc['yearly'].values, index=doy).groupby(level=0).mean()
11 common = lgb_seas.index.intersection(pro_year.index)
12 print("Tương quan r =", round(np.corrcoef(lgb_seas[common], pro_year[common])[0,1], 3))
# ~0.87

```



Hình 23: Phân rã Prophet (Revenue): trend, thành phần theo tuần và thành phần mùa vụ năm. Thành phần mùa vụ năm có dạng trùng với SHAP mùa vụ của LightGBM; thành phần tuần cho ngày thường cao hơn cuối tuần.

Thành phần mùa vụ năm của Prophet (Hình 23) tương quan rất cao với SHAP mùa vụ của LightGBM theo ngày trong năm, hệ số Pearson $r \approx 0.87$; thành phần theo tuần của Prophet cũng cho ngày thường cao hơn cuối tuần, trùng với nhóm Weekly mà SHAP xếp hạng. Một phép kiểm tra chéo thứ hai trong nội bộ LightGBM: 8 trong 10 đặc trưng quan trọng nhất theo **gain importance** trùng với 10 đặc trưng đứng đầu theo SHAP. Cả hai phép kiểm tra đều làm tăng độ tin cậy của kết luận.

Hướng mở rộng

Cuối cùng, phần này trình bày một số các hướng để có thể tiếp tục mở rộng và cải thiện tính hiệu quả của mô hình:

Adaptive per-quarter blend fraction

Phương pháp ở Mục 9.1 dùng hệ số blend $\beta = 0.3$ đồng đều cho cả bốn quý. Nhưng Q1, Q2, Q4 vốn đã có margin hợp lý trong output — áp $\beta = 0.3$ cho chúng là *can thiệp không cần thiết*. Ngược lại, Q3 cần β lớn hơn để kéo margin về đúng vùng. Một hệ số β cố định là tối ưu dưới *trung bình* nhưng chưa tối ưu cho mỗi quý.

CV-optimized ensemble weights (NNLS)

Trong quá trình huấn luyện, chúng ta thử nhiều scheme sample weighting khác nhau (`high_era`, `t1718`, `up_HIGH`, `analog`, `up_2018`), mỗi cái cho ra một LGB khác nhau. Pipeline chỉ dùng `high_era`, vứt bỏ thông tin từ các scheme còn lại. Nhưng liệu một tổ hợp tuyến tính của chúng có thể tốt hơn bất kỳ thành viên đơn lẻ nào?

Phân tích thử nghiệm sự features

Phương pháp tập trung vào tạo các đặc trưng nhưng chưa nghiên cứu sâu vào sự ảnh hưởng thực sự. Nên cần có những ước tính và đo lường cụ thể hơn để xem xét đâu là đặc trưng có nhiều ý nghĩa.

Fine-tuning foundation models

Thử nghiệm, các foundation model (Chronos, NHITS, TTM, PatchTST, TimesFM). Các foundation model mang đến diversity hoàn toàn khác so với họ cây/tuyến tính.

Tổng kết

Đặc trưng dữ liệu cho bài toán

Khi horizon dài, **feature calendar thuần là lựa chọn an toàn nhất**. Việc loại bỏ hoàn toàn feature lag ($y_{t-1}, y_{t-7}, y_{t-365}$) có vẻ hạn chế nhưng lại là điều kiện tiên quyết để pipeline không bị “đứt” khi dự báo xa. Mọi đặc trưng được mã hoá chỉ từ ngày tháng, nhờ vậy ma trận test có thể tính sẵn đầy đủ ngay từ đầu, không phải tính lại trong lúc rolling.

Tách bài toán “học shape” khỏi bài toán “điều chỉnh level”. Khi dữ liệu có regime drift, cố gắng dạy một mô hình duy nhất cả shape lẫn level thường không hiệu quả. Pipeline này chia thành hai pha rõ ràng: sample weight giúp mô hình tập trung học shape từ vùng sạch 2014–2018, rồi hai hệ số calibration tune trên LB phủ khoảng cách level. Cả hai thành phần đơn giản nhưng khi kết hợp đem lại cải thiện lớn.

Diversity có chủ đích trong ensemble. Ba mô hình nền không được chọn ngẫu nhiên mà đại diện ba họ giải thuật khác nhau (tuyến tính, cây, phân rã). Bốn quarter specialist không được train song song vô hướng mà mỗi cái được “kéo” về một quý cụ thể qua Q-boost. Kết quả là các thành viên ensemble không chồng lấp về sai số, cho phép blend tạo ra cải thiện thực sự.

Quy trình tổng quát

Pipeline này có thể được gói thành một *công thức 4 bước* dùng được cho nhiều bài toán forecasting horizon dài khác:

1. **Nhận diện regime và mùa vụ**. Vẽ timeline, phân vùng, kiểm tra tính ổn định của seasonality. Nếu shape ổn định thì cơ chế “học shape + calibrate level” khả thi.
2. **Xây feature calendar thuần**. Mã hoá mọi pattern tự nhiên theo công thức từ Date: Fourier, Tet/holidays, promo schedule, edge-of-month.
3. **Ensemble đa họ với trọng số CV-tune + calibration LB-tune**. Ba họ khác nhau (tuyến tính, cây, phân rã). Blend nội bộ trong từng họ nếu có variant. Tầng calibration riêng xử lý level drift.
4. **Phân tích sai số per-segment, xây specialist cho segment lệch nhất**. Nếu một segment (quý, vùng, kênh) có sai số cao bất thường, train mô hình riêng cho nó và blend lại với mô hình chung.

Pipeline này đạt LB khoảng 680,000 sau khi tuning một số tham số nhưng vẫn còn cách đáng kể so với điểm lý thuyết tối ưu. Một số điểm có thể phân tích tiếp:

- **Chỉ dùng feature từ sales.csv**. Các bảng phụ trợ (web_traffic, inventory, ...) chưa được khai thác, một phần vì chúng dừng ở 2022. Nhưng có thể trích pattern ổn định từ chúng (ví dụ: chu kỳ tồn kho trung bình, correlation giữa lưu lượng truy cập và doanh thu) để thêm feature cho test.
- **Chưa khai thác diversity của neural model**. Foundation model fine-tuned có thể là hướng tốt có sự diversity hoàn toàn khác cho ensemble.

Phụ lục

1. **Hint:** Các file code gợi ý và dữ liệu nếu có được lưu trong thư mục có thể được tải [tại đây](#).
2. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải [tại đây](#) (**Lưu ý:** Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).
3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.

AIO_Family

🔒 Private group · 2.0K members



Hình 24: Hình ảnh group facebook AIO Q&A